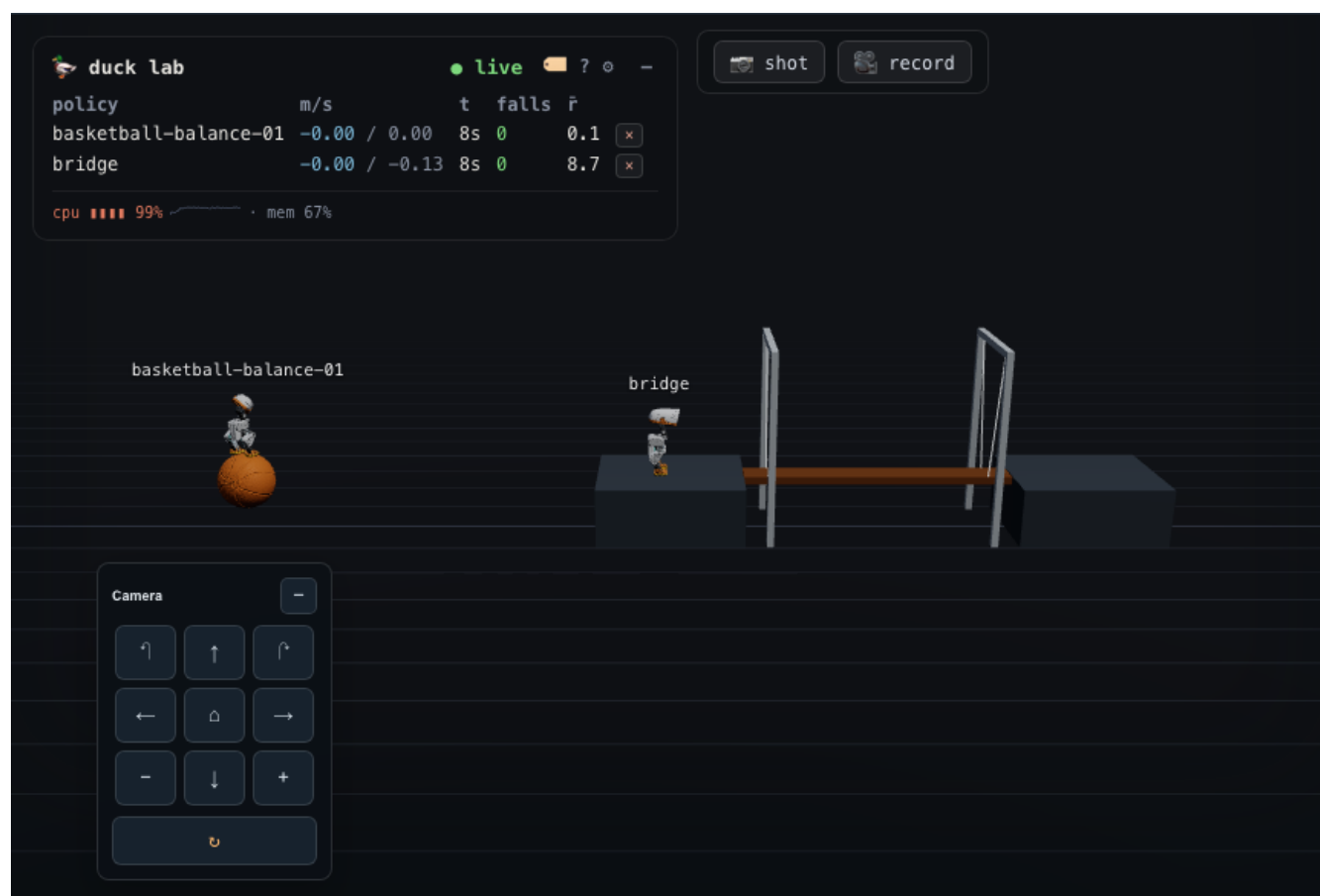


Microduck 平衡实验室

篮球与悬挂独木桥：从建模到 PPO、验证与复现

Author: George Hu

2026-09-11 · v1.0 · Evidence: 2026-09-10



Real simulation evidence. Basketball balance verified; rolling and full bridge crossing remain unaccepted.

PPO / MuJoCo / Curriculum / ONNX / Reproducible code

目录与学习路线

1 两个平衡实验室：本书究竟复现什么	3
1.1 一本有边界的实践教材	3
1.2 学习路线	3
1.3 统一术语	3
1.4 建立实验笔记	4
1.5 仓库地图	4
2 数学和物理：从高中知识出发	5
2.1 先理解单位	5
2.2 为什么平衡和运动分不开	5
2.3 坐标系：谁的前方？	5
2.4 反馈、记忆和部分可观测	6
2.5 算术练习	6
3 搭建可复现的实验室	7
3.1 物料：仿真资产不是实体采购清单	7
3.2 计算机和软件前提	7
3.3 获取工作区，而不只是 PDF	7
3.4 分清预检、smoke、训练和评估	8
3.5 第一步动作前，先看场景	9
3.6 新实验必须使用新输出目录	9
3.7 排错时不要悄悄改题	9
4 PPO：从一小批经验中谨慎学习	10
4.1 用普通语言理解学习循环	10
4.2 Actor、critic 与 advantage 的含义	11
4.3 连续动作使用概率密度	11
4.4 从 log 概率得到 PPO ratio	13
4.5 裁剪后的策略目标	13
4.6 广义优势估计	14
4.7 Terminal 与 time limit 使用不同 mask	15
4.8 Advantage 归一化	17
4.9 Critic 损失	17
4.10 Entropy 以及它为何可以为负	18
4.11 总 loss 不是 reward，两者都不能证明掌握技能	19
4.12 篮球中的循环序列重放	19
4.13 通用 MLX PPO 的前馈 minibatch	20
4.14 KL divergence：MLX 中是诊断，篮球中是停止规则	20
4.15 准确区分默认值与覆盖值	21
4.16 一次完整更新的全过程	22
4.17 常见解释错误	23
4.18 练习	23
4.19 答案	23
4.20 参考文献	24
5 篮球：在自由滚动球体上的盲感知循环平衡	25
5.1 学习目标	25
5.2 证据与结论边界	25

5.3	用数字搭建物理场景	26
5.4	精确的 61 维观测与 14 维动作契约	28
5.5	循环演员网络与状态生命周期	29
5.6	精确的本地奖励	31
5.7	物理辅助课程	33
5.8	训练：热启动、PPO 与从零训练边界	34
5.9	经过参数解析器核对的命令行流程	35
5.10	评估：平衡不等于滚动	37
5.11	阅读产物而不夸大结论	39
5.12	练习	42
5.13	参考答案	42
6	悬索桥：在会移动的支撑面上学习行走	43
6.1	学习目标	43
6.2	证据范围与结论边界	43
6.3	仿真中真正存在什么？	43
6.4	接触、运动与失败条件	45
6.5	观测契约：特权指挥器与普通 actor	46
6.6	奖励：继承行走奖励，没有过桥大奖	47
6.7	只改变物理的课程学习	50
6.8	从已发布行走 actor 迁移	51
6.9	复现实验流程	51
6.10	严格的完整 20 秒过桥规则	53
6.11	当前试验实际完成了什么？	54
6.12	练习	54
6.13	参考答案	55
7	看证据，不看庆祝语	57
7.1	建立证据链	57
7.2	阅读真实 reward 和 loss 曲线	57
7.3	确定性评估：究竟通过了什么？	59
7.4	视觉验证：看连续画面	60
7.5	在 Duck Viewer 运行两个场景	63
7.6	尚未完成技能的研究计划	65
7.7	重建并检查教材	65
7.8	综合实践考核	65
8	参考文献与来源	66
9	附录：完整核心源码与版本指纹	67
9.1	A1. rlx/rlx/environments/basketball.py	67
9.2	A2. rlx/rlx/models/basketball.py	71
9.3	A3. rlx/examples/ppo_microduck_basketball.py	76
9.4	A4. rlx/rlx/environments/bridge.py	86
9.5	A5. rlx/rlx/environments/bridge_evaluation.py	94
9.6	A6. rlx/rlx/models/bridge_bootstrap.py	99
9.7	A7. rlx/rlx/algorithms/ppo.py	101
9.8	Source inventory / 源码清单	110

1 两个平衡实验室：本书究竟复现什么

1.1 一本有边界的实践教材

想象自己站在篮球上：脚移动会改变球的运动，球的运动又会改变脚下的支撑。再想象自己走上由四根绳索悬挂的窄木板：这一步改变了木板的状态，下一步就必须应对新的支撑。这是相互耦合的平衡问题，不是把走路动画放到不同背景上。

本书沿着真实 Microduck 仓库，从物理假设、建模、环境、输入输出，走到 PPO 训练、模型导出、评估和七案例 Duck Viewer。读者只需高中代数、函数图像和初步概率知识。公式出现时会解释符号；代码附录提供真实实现，而不是把一个玩具示例冒充完整训练器。

本书采用 2026 年 9 月 10 日记录的本地实验作为证据；文档版本为 1.0，2026 年 9 月 11 日。版本号表示教材快照，不表示所有技能都已经完成。

问题	证据支持的回答
鸭子能在自由篮球上持续保持平衡吗？	能。固定指令本地候选策略在六次 60 秒名义条件测试中全部存活。
能按指令可靠地驱动篮球前进、转向吗？	尚未证明；三次前进指令测试均未通过滚动门槛。
悬挂独木桥是真实工作的物理仿真吗？	是：自由木板、四根物理绳索 tendon、起终点平台和可见支架。
最终独木桥试训成功完成全程了吗？	没有。新增 PPO 数据 32,768 步，合格过桥次数为零。
两个案例都加入 Duck Viewer 了吗？	是。七案例等于原有五个加两个，不等于七个技能全部学会。
已经在实体机器人上验证了吗？	没有。本书结果全部来自仿真。

成功的篮球行为起点是用户提供的成熟循环策略。本地成功环节包括：导入源策略、准确迁移 actor、进一步 PPO 训练、导出一致性验证、持续平衡以及可视化回放。不能据此说随机初始化网络只用 102,400 步就学会篮球平衡。同样，独木桥训练命令成功运行，不代表鸭子成功过桥。

“从无到有”有两层含义。你可以用源代码和必要资产搭出完整的软件实验；但不能重建未提供完整记录的上游训练历史，也不能用一句话替代缺失的成熟权重。本书完整提供可复现的本地路线，明确将原策略列为外部前提。source/ 与 ZIP 是配套代码快照，不包含所有第三方网格、软件依赖和模型权重。

1.2 学习路线

1. **第一天下午**：阅读假设与建模图，运行环境预检和纯 Python PPO 算术，查看真实连续帧。理解流程不必先训练。
2. **第一次实验**：保留原始资产，建立新的运行目录，执行微型 smoke，逐个检查输出。四步 smoke 只检验管线连通。
3. **测量复现**：执行篮球固定指令微调或独木桥 pilot，导出，关闭辅助后确定性评估，查看完整视频。
4. **研究扩展**：每次只改变一个物理或课程参数，保持评估协议固定，运行多个独立种子，认真记录失败。

调学习率前先读 PPO 章。用曲线判断学习成功前先读验证章。代码附录应与案例章对照阅读，不建议一开始就无背景地看几千行程序。

1.3 统一术语

术语	本书中的含义
状态 State	仿真所有物理变量，包含策略看不到的信息。
观测 Observation	每个控制步提供给 actor 的 61 个数。

术语	本书中的含义
动作 Action	根据机器人控制合同解释的 14 个策略输出。
策略 / Actor	训练时产生动作分布、确定性测试时输出均值的函数。
Critic / 价值	估计未来折扣奖励，用于训练。
回合 Episode	从 reset 开始，到终止或时间上限的一次尝试。
转移 Transition	单环境的一次观测、动作、奖励、下一观测交互。
Rollout	更新网络前收集的一批转移。
Checkpoint	训练权重与该格式实际保存的元数据。
ONNX	已嵌入观测归一化的可移植推理图。
Curriculum	从易到难的物理、出生位置或判定严格程度，不是暗中换奖励。
Parity	同输入、同循环记忆生命周期下两个实现的数值一致性。
技能门槛	可测量的验收条件，不是程序退出码或文件是否生成。

1.4 建立实验笔记

记录源码哈希、命令、Python 路径、包版本、执行器模式、随机种子、出生分布、辅助等级、随机化设置、指令分布、步数和评估时长。保存完整 JSON，而不是只截绿灯。

下面是记录模板，并非新增实测结果：

```
{
  "experiment": "my-independent-basketball-run",
  "initialization": "supplied actor; fresh critic and optimizer",
  "control_dt_s": 0.02,
  "actor_observations": 61,
  "actions": 14,
  "claim_to_test": "60-second free-ball balance",
  "claim_not_yet_supported": "commanded rolling and steering",
  "hardware_tested": false
}
```

实际参数应以 checkpoint 旁的记录为准。文件夹叫 final、full 或 success，本身都不能证明成功。

1.5 仓库地图

除非代码块明确切换目录，本书 shell 命令都从工作区根目录执行。rlx/ 是相对根目录的路径；教材资源和源码快照则位于 docs/basketball-bridge-book/。

```
microduck-lab/
microduck_local/   CPU MuJoCo、机器人合同、lab 后端
microduck_rl/      上游机器人和训练参考仓库
microduck/          上游运行时和现成 walking 策略
microduck-playground/ 用户提供的篮球实验、资产与 checkpoint
rlx/                本地 PPO、环境、ONNX 导出
duck-viewer/        浏览器 Studio 和真实场景回放
scripts/            场景命令与 API 验证
docs/basketball-bridge-book/
  chapters/en/      英文章节源文件
  chapters/zh/      中文章节源文件
  assets/           实测曲线、截屏、示意图
  examples/         可运行的算术与环境探针
  tools/            构建、预检、验证
  source/           选定实现的精确快照
```

执行单位是整个工作区。把一个环境文件复制到空文件夹，不会自动获得机器人合同、网格、执行器实现和训练框架。

2 数学和物理：从高中知识出发

2.1 先理解单位

长度用米，时间用秒，质量用千克，角度用弧度，力用牛顿，力矩用牛顿米。一个没标单位的数字，看起来合理，却可能相差一千倍。

50 Hz 表示每秒向 actor 询问 50 次动作：

$$\Delta t = 1/50 = 0.02 \text{ s.}$$

篮球 60 秒对应 3,000 个控制步；独木桥 20 秒对应 1,000 步。这里是控制步，MuJoCo 内部可能在一次控制期间执行多个更小的物理积分步。环境数增加的是每批数据量，而不是改变一秒有多长。

速度就是位置变化除以时间。0.2 秒前进 0.03 米，平均速度为 0.15 米 / 秒。加速度是速度变化除以时间。先用有限差分，就能理解仿真任务，不必马上学习微分方程。

力改变动量。一维教学近似为：

$$v_{t+1} = v_t + (F/m)\Delta t, \quad x_{t+1} = x_t + v_{t+1}\Delta t.$$

这是说明性近似，**不能替代** MuJoCo 的积分、多关节动力学和接触求解。真实仿真还要处理关节、碰撞、摩擦和约束。力矩是使物体转动的作用，等于力乘以垂直力臂。脚的作用力偏离木板中心，即使鸭子仍站直，也可能把木板转动起来。

2.2 为什么平衡和运动分不开

地面上的支撑脚下方近似不动。在球上，脚施加切向力会使球转动，支撑点随之移动。理想刚性球无滑动滚动时，距离和转角满足：

$$d = r\theta.$$

半径 0.12 米，转过 90 度，即 $\theta = \pi/2$ 弧度，对应 $d \approx 0.1885$ 米。以 0.15 米 / 秒滚动，理想角速度为 $v/r = 1.25$ 弧度 / 秒。这只是尺度检查：滑动、接触形变和转弯都会破坏简单直线公式。

悬挂桥上，重力向下，绳索支撑并约束木板。绳索不是刚性横梁，拉力沿绳方向。代码用 tendon 与限位设置近似所需的绳索行为，具体物理含义由参数决定，不由画面中线条的样子决定。支架可以可见但不参与碰撞。因此视觉图像证明外观，不能完整描述动力学。

“木头”“橡胶”这样的名字不会自动产生正确物理。仿真需要尺寸、质量、惯量、摩擦与接触参数、关节和 tendon 定义。木纹是视觉材质；0.45 千克的木板质量是动力学参数，两者必须分开记录。

2.3 坐标系：谁的前方？

世界坐标系固定在场景中；**机体坐标系**跟着鸭子转动。鸭子转过 90 度，身体前方就不再是世界 x 正方向。如果指令是身体前进，却奖励世界 x 速度，任务就会被错误定义。

平面航向角为 ψ 时，机体速度转换到世界速度：

$$v_x^w = \cos \psi v_x^b - \sin \psi v_y^b,$$

$$v_y^w = \sin \psi v_x^b + \cos \psi v_y^b.$$

上标 w 、 b 分别表示 world 与 body，不是乘方。身体速度 $(0.15, 0)$ 、航向 90 度时，世界速度约为 $(0, 0.15)$ 。篮球评估用初始航向作为方向进度参考；独木桥 commander 把导航信息转换成标准机体指令。这都不意味着 actor 直接获得世界坐标。

2.4 反馈、记忆和部分可观测

开环脚本不管发生什么，都重复预定关节目标。反馈策略根据当前感知选择下一组目标。如果脚打滑，下一次观测会变化，策略就可能纠正。

机器人拿不到所有仿真状态，只拿到角速度、投影重力、指令、关节信息、上一步动作和保留指令槽。它不能直接读取球心或木板姿态。这叫部分可观测：当前观测相同，过去的运动过程可能不同。篮球 actor 用 LSTM 保存记忆，独木桥使用迁移来的前馈 walking actor。记忆网络不是“透视眼”，只能总结实际进入观测的信息。

训练时随机采样动作，用于探索附近行为；确定性测试时使用导出 actor 的均值。随机策略可能偶尔靠噪声救回一次失衡，而均值策略反而摔倒。因此训练 reward 和导出策略测试是不同实验。

2.5 算术练习

```
import math
control_dt = 0.02
radius = 0.12
command_speed = 0.15
steps = round(60 / control_dt)
quarter_turn_distance = radius * math.pi / 2
ideal_angular_speed = command_speed / radius
print(steps, round(quarter_turn_distance, 4), ideal_angular_speed)
assert steps == 3000
assert abs(ideal_angular_speed - 1.25) < 1e-12
```

预期输出：3000 0.1885 1.25。这验证的是算术，不是平衡技能。

练习：四个环境，每次各采 128 步，得到多少 transition？每个环境经过几秒？累计 32,768 步需要多少批？

答案： $4 \times 128 = 512$ 步；单环境经过 $128 \times 0.02 = 2.56$ 秒； $32768 / 512 = 64$ 批。各环境时间相加只是总经验量，不是一次连续 655.36 秒的过桥。

练习：为什么高平均 reward 不能证明持续平衡 60 秒？

答案：多段表现不错但被摔倒和 reset 分隔的短片段，也可能获得高均值。持续平衡要求不间断的完整回合、首次摔倒判定、合法脚支撑，以及完整时长。

3 搭建可复现的实验室

3.1 物料：仿真资产不是实体采购清单

下面是仿真物料表，说明复现软件实验所需的对象与文件，不是经过认证的机械图纸、采购建议，更不是把昂贵机器人放到晃动木板上的施工规范。

项目	数量	用途与限制
Microduck 全碰撞 MJCF 和引用网 格	每场景一个模型	14 个驱动关节；使用配套 XML，不 随意换机器人。
自由篮球	一个	半径 0.12 米、质量 0.62 千克；碰 撞球体与贴图网格分开。
篮球 OBJ、PNG	各一份	使用 playground 资产；外观不同于 接触物理。
悬挂木板	一块	长 1.10、宽 0.13、厚 0.03 米，质量 0.45 千克。
绳索 tendon	四根	物理约束和支撑，不只是装饰线。
起点、终点平台	两个	固定支撑，明确出发与到达区域。
可见支架	场景组件	表示锚点位置；自身不参与碰撞。
地面、灯光、相机	一组	地面接触需要被判失败，不能被镜 头遮住。
已训练篮球 actor 与参考 ONNX	一对	实测本地路线所需的成熟初始化策 略。
现成 walking ONNX	一份	用于精确迁移独木桥 actor 和归一 化。

本书没有测量真实木材等级、绳索断裂载荷、紧固件规格、电池限制、舵机热限制和安全系数。不能从仿真尺寸推导出这些结论。实体测试还需要工程安全绳、隔离区、急停、力矩 / 电流限制、专业监督和独立 sim-to-real 验证。本地训练器是原型工具，不是部署许可。

3.2 计算机和软件前提

记录中的路线使用 Apple Silicon Mac。MuJoCo 在 CPU 上推进物理场景；篮球 PPO 使用 PyTorch CPU；独木桥 PPO 使用 MLX / Metal。“不需要 CUDA”不等于“所有环节在任何纯 CPU 操作系统上都
可原样执行”。本版不声称完成 Linux 上独木桥 learner 的全新复现。

使用 Python 3.12，而不是 PATH 中恰好排第一的 python。Node/npm 运行 viewer；uv 管理 Python 环境。Pandoc、XeLaTeX、pdfinfo、pdftotext、pdftoppm 用于生成和检查教材，训练不需要它们。Pillow 和 Matplotlib 生成图表，不生成机器人动作。

实测 Python 包版本自动记录在 environment.json。它是**环境清单**，不是经过全平台测试的依赖锁。source-manifest.json 用 SHA-256 绑定每个源文件，因为仅靠 Git commit 无法包含尚未提交的新技能实现。

3.3 获取工作区，而不只是 PDF

向项目维护者取得完整工作区，包括 rlx、上游兄弟仓库，以及用户提供的 playground 参考。不要编造权重的公开下载地址。安装前检查：

```
test -f microduck_local/src/microduck_local/contract.py
test -f rlx/rlx/mjlab_microduck/robot/microduck/robot_allcollisions.xml
test -f microduck/policies/alpha_walking.onnx
test -f microduck-playground/artifacts/basketball/checkpoint.pt
test -f microduck-playground/artifacts/basketball/policy.onnx
```



```
test -f microduck-playground/src/mjlab_microduck/robot/assets/basketball/basketball.obj
test -f microduck-playground/src/mjlab_microduck/robot/assets/basketball/basketball.png
command -v uv node npm
```

检查失败就停在缺失资产处。改 actor 架构或把归一化换成零，不会修复缺失权重。独木桥本身使用配套机器人资产；篮球额外需要 playground 的 OBJ / PNG。walking 基线及其他 harness 路径还可能需上游模型仓库。

新环境采用仓库文档中的安装路线：

```
cd rlx
uv sync --python 3.12
uv pip install --python .venv/bin/python -e ../microduck_local
.venv/bin/python examples/ppo_microduck_studio.py --help
cd ..
cd duck-viewer
npm ci
cd ..
```

安装需要网络和可用的软件包。本书验证使用已有的记录环境，没有声称在另一台离线机器上完成全新安装。保留 lockfile；改变解析后的依赖前，比较版本并运行测试。不要跨机器复制虚拟环境。

实测解释器是 `rlx/.venv-microduck/bin/python`；上述安装生成 `rlx/.venv/bin/python`。如果使用新环境，请一致地替换各章命令路径：

```
PYTHON="$PWD/rlx/.venv-microduck/bin/python"
test -x "$PYTHON"
"$PYTHON" docs/basketball-bridge-book/tools/preflight.py
"$PYTHON" docs/basketball-bridge-book/examples/ppo_arithmetic.py
```

预检查文件和包版本，不导入 Metal，因此不能证明后续 Metal / 图形上下文一定可用。受限沙箱可能允许读文件却拒绝 Metal 或 CoreGraphics。这是运行权限问题，不等于策略错误。

3.4 分清预检、smoke、训练和评估

阶段	最小有效证据	不能证明什么
预检 合同测试	必要文件和包可定位。 维度、reset、课程和评估规则在测试中正确。	动力学、绘图和策略表现。 训练技能可靠。
Smoke 训练	少量 transition 能进入更新、 checkpoint 和 ONNX。	技能习得或收敛。
Pilot 试训 确定性评估 视觉检查	有界真实实验改变参数并记录指标。 导出策略满足或不满足明确条件。 接触、姿态、真实运动与指标一致。	未经评估的过桥或转向。 测试条件以外的鲁棒性。 未检查的种子或实体场景。

长期训练前运行针对性合同测试：

```
rlx/.venv-microduck/bin/python -m pytest -q \
  rlx/tests/test_basketball.py \
  rlx/tests/test_basketball_policy.py \
  rlx/tests/test_basketball_evaluation.py \
  rlx/tests/test_balance_adapter.py \
  rlx/tests/test_bridge.py \
  rlx/tests/test_bridge_evaluation.py \
  rlx/tests/test_bridge_bootstrap.py
```

这些测试覆盖所列边界，不代表仓库所有无关测试都通过。本次教材的当前测试与出版检查另存于 `verification/`。

3.5 第一步动作前，先看场景

本书环境探针编译两个模型，检查输入 / 输出维度，并各采四个有限数值 transition。它使用零动作，只验证构造，不加载已训练策略，也不证明平衡：

```
rlx/.venv-microduck/bin/python \
docs/basketball-bridge-book/examples/probe_scenes.py
```

每个环境有私有的 MuJoCo 可变数据；共享不可变模型可以省内存，但不能共享循环记忆或机器人状态。viewer 可以让两只鸭子复用同一 ONNX session，但每只鸭子必须拥有独立 LSTM 记忆。

3.6 新实验必须使用新输出目录

例如 rlx/runs/book/basketball-YYYYMMDD-HHMMSS，把日期占位符换成新的运行标识。检查目录不存在再训练，绝不覆盖本书证据目录。

每次至少保存：完整命令、软件清单、源码与源策略哈希；checkpoint 及元数据；最终 ONNX 和一致性结果；完整 update / episode 日志；包含失败、种子、时长、辅助设置的评估 JSON；与被测策略绑定的视频、连续帧和渲染元数据。

Checkpoint 不自动等于完整优化器恢复。这两条路线都是迁移 actor，但新建 critic 和 optimizer。重跑可以复现协议，不一定得到逐位相同的浮点权重。数值库、版本、调度、仿真器变化都可能改变轨迹。正确承诺是可追溯实验与一致的验收规则，而不是无依据的“训练结果逐位相同”。

3.7 排错时不要悄悄改题

现象	首先检查	不应该做什么
篮球不可见	OBJ / PNG 路径与 group-2 视觉几何。	用装饰动画代替真实碰撞球体。
模型能编译但 viewer 是平地 导出动作不一致	每策略环境参数和 lab 场景提取。 归一化分母、观测顺序、h/c reset、ONNX 元数据。	只凭策略名称声称桥已经显示。 因“看起来正常”忽略 parity。
No Metal device available CoreGraphics 错误 Reward 高但不成功 stdout JSON 格式不匹配	在获准的本地运行时执行 MLX。 图形上下文权限和渲染后端。 完整时长、合法接触和视频。 优先结构化日志和结果文件。	暗中换算法，仍称同一实验。 拿示意图冒充实测截图。 没查探索缺口就加奖励系数。 把解析不到的 loss 画成零。

练习：为什么既保留策略哈希又保留视频？**答案：**好看的视频可能来自另一个策略。绑定哈希的元数据把结论、checkpoint、确定性导出、评估和可见轨迹连成证据链。

4 PPO：从一小批经验中谨慎学习

近端策略优化（Proximal Policy Optimization，通常简称 **PPO**）是 RLX 本地训练器使用的学习算法。PPO 不会直接告诉机器人“篮球”是什么意思。环境负责提供观测、动作、奖励和回合边界。PPO 回答的是一个范围更窄的问题：

根据当前策略刚刚尝试的动作，哪些动作应当稍微提高概率，哪些动作应当稍微降低概率，以及一次更新最多应该改变多大？

本章从高中阶段的代数、概率密度和平均数出发解释这个问题，并把每个概念对应到仓库中的当前实现：

- `rlx/examples/ppo_microduck_basketball.py`: 用于已发布篮球 actor 的循环 PyTorch 训练器；
- `rlx/rlx/algorithms/ppo.py`: 通用的前馈 MLX PPO 实现；
- `rlx/rlx/buffers/rollout_buffer.py`: MLX 实现使用的定长 rollout 缓冲区。

实现细节以代码为准。PPO 论文解释通用方法，而本仓库还加入了超时感知 bootstrap、稳健 critic 损失、log-ratio 数值裁剪，以及篮球训练器专用的 KL 提前停止规则。

4.1 用普通语言理解学习循环

设想一位教练每 20 毫秒必须指导一次运动员。每个控制时刻都会发生以下过程：

1. 策略接收一条观测，例如关节状态、身体朝向、运动命令和近期运动信息。
2. Actor 输出 14 个连续关节命令上的概率分布。
3. 从该分布中采样一个动作。
4. 模拟器向前推进，并返回奖励和边界标志。
5. Critic 预测执行动作之前的观测预计能获得多少折扣未来奖励。

PPO 保存一小段这样的 transition。随后，它把已保存的动作放入更新后的策略，比较：

旧 log 概率 和 新 log 概率。

如果某个动作的结果比 critic 原先预期更好，PPO 就尝试提高该动作的概率；如果结果更差，就尝试降低其概率。近端的含义是更新应当保守：当概率比率在有利方向上变化过大时，PPO 不允许目标函数继续获得同样的收益。

执行若干次优化后，这批 rollout 会被丢弃。系统必须使用新策略重新收集数据。因此 PPO 属于 **on-policy（同策略）** 算法：它不会无限保留来自许多旧策略的动作作为长期 replay 数据库。

本仓库中的两个实现用不同方式组织同一个思想：

路径	Actor 记忆	更新组织方式	主要数值框架
通用 <code>rlx/rlx/algorithms/ppo.py</code>	前馈	展平时间和环境维，打乱后使用 minibatch	MLX
篮球 <code>ppo_microduck_basketball.py</code>	单层 LSTM	每个 epoch 按完整的时间×环境序列重放	PyTorch

这个差异不是表面上的代码风格。前馈样本可以作为独立的数据行训练，而 LSTM 在当前时刻的动作依赖先前观测产生的隐藏状态，所以必须按正确顺序重建历史。

4.2 Actor、critic 与 advantage 的含义

PPO 训练两个预测系统。

Actor 负责选择动作。在连续控制中，它输出动作均值 $\mu(o)$ 和标准差 σ 。篮球 actor 有 14 个动作维度。它先用隐藏维度为 256 的 LSTM 处理 61 维观测，再通过宽度依次为 512、256、128 的 MLP，把循环网络输出映射成 14 个均值。

Critic 预测一个标量 value：

$$V(o_t) \approx \mathbb{E}[r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots].$$

折扣因子 γ 让更远的奖励权重更小。当 $\gamma = 0.99$ 时，下一步奖励乘以 0.99，下两步奖励乘以 0.99^2 ，依此类推。

Actor 并不直接使用原始奖励训练，而是使用 **advantage (优势)** 估计：

$$\hat{A}_t \approx \text{动作 } a_t \text{ 之后的结果} - \text{执行 } a_t \text{ 前 critic 的预期}.$$

- $\hat{A}_t > 0$ ：采样动作优于基线，应提高其概率；
- $\hat{A}_t < 0$ ：采样动作劣于基线，应降低其概率；
- $\hat{A}_t \approx 0$ ：该样本几乎没有提供改变其概率的证据。

“正 advantage”并不等于“正奖励”。如果 critic 预计得到 5，而实际奖励相关结果只有 2，那么 advantage 可以为负。如果 critic 预计为 -4，而结果为 -1，advantage 反而可以为正。Advantage 表示与基线的比较。

虽然篮球 actor 是循环网络，但篮球 critic 是前馈网络。它读取当前经过归一化的 61 维向量并预测一个 value。Actor 和 critic 共享 actor 中冻结的观测归一化器，但篮球续训会创建一个全新的 critic 和全新的 Adam 优化器。这属于 actor warm start，而不是对 critic 和优化器状态的完整恢复。

4.3 连续动作使用概率密度

对于一个动作坐标，高斯策略的密度为

$$p(a \mid \mu, \sigma) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(a - \mu)^2}{2\sigma^2}\right).$$

对应的 log 密度为

$$\log p(a \mid \mu, \sigma) = -\frac{1}{2} \left[\frac{(a - \mu)^2}{\sigma^2} + 2 \log \sigma + \log(2\pi) \right].$$

对于 14 个相互独立的高斯坐标，实现会把 14 个 log 密度相加：

$$\log \pi(a \mid o) = \sum_{j=1}^{14} \log p(a_j \mid \mu_j(o), \sigma_j).$$

因此每条 transition 最终只得到一个联合 log 概率。

4.3.1 密度可以大于 1

学生常常学过“概率不能大于 1”。这对事件的概率是正确的，但连续高斯公式返回的是**密度**，不是某个精确点本身的概率。窄分布的曲线高度可以超过 1，只要整条曲线下面积仍然等于 1。

当 $\mu = 0$ 、 $\sigma = 0.1$ 时，均值位置的密度为

$$p(0) = \frac{1}{0.1\sqrt{2\pi}} \approx 3.989.$$

落入某个区间的概率是该区间下的面积。在均值附近取宽度为 0.01 的很小区间，概率可近似为密度乘宽度，即 $3.989 \times 0.01 = 0.03989$ ，仍然小于 1。

因此 log 密度也可以为正。这完全合法。连续分布中的正 log 密度不代表概率大于 1。

下面这个完整的纯标准库程序可以复现该计算：

```
import math

def gaussian_log_density(x: float, mean: float, std: float) -> float:
    if std <= 0.0:
        raise ValueError("std must be positive")
    variance = std * std
    return -0.5 * (
        (x - mean) ** 2 / variance
        + 2.0 * math.log(std)
        + math.log(2.0 * math.pi)
    )

def main() -> None:
    log_density = gaussian_log_density(0.0, 0.0, 0.1)
    density = math.exp(log_density)
    approximate_interval_probability = density * 0.01

    print(f"log density: {log_density:.6f}")
    print(f"density: {density:.6f}")
    print(f"approximate probability in width 0.01: "
          f"{approximate_interval_probability:.6f}")

    assert density > 1.0
    assert 0.0 < approximate_interval_probability < 1.0

if __name__ == "__main__":
    main()
```

除最后一位浮点舍入外，预期输出为：

```
log density: 1.383647
density: 3.989423
approximate probability in width 0.01: 0.039894
```

在 ppo_microduck_basketball.py 中，gaussian_log_prob() 使用 Torch 张量实现同一表达式，并沿动作维求和。gaussian_entropy() 同样沿动作维求和。

4.4 从 log 概率得到 PPO ratio

缓冲区中的动作由旧策略采样。PPO 要比较该动作在新策略下的密度发生了怎样的变化：

$$r_t(\theta) = \frac{\pi_{\theta}(a_t | o_t)}{\pi_{\theta_{\text{old}}}(a_t | o_t)}.$$

直接除以极小的密度在数值上不方便，因此代码使用 log 概率：

$$r_t(\theta) = \exp \left(\log \pi_{\theta}(a_t | o_t) - \log \pi_{\theta_{\text{old}}}(a_t | o_t) \right).$$

由于指数函数始终为正，ratio 也始终为正。

- $r = 1$ ：已保存动作的密度没有变化；
- $r = 1.2$ ：密度提高了 20%；
- $r = 0.7$ ：密度降低了 30%。

Ratio 不是“这个动作的概率”，而是同一个已记录动作在新旧密度下的比值。

通用 MLX 代码在求指数之前把 **log ratio** 裁剪到 $[-20, 20]$ 。这是防止数值溢出的保护措施，与 PPO 通常使用的 $[0.8, 1.2]$ 等较窄策略裁剪区间不是同一件事。篮球训练器目前直接对无限制的 log-ratio 求指数，然后检查 KL 或损失是否出现非有限值。

4.5 裁剪后的策略目标

PPO 对单个样本使用的 surrogate 为

$$L_t^{\text{clip}} = \min \left(r_t \hat{A}_t, \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right).$$

算法希望最大化该值，而实现通常最小化它的负数。通用 MLX 源码用两个负项的 maximum 表达；篮球源码先对两个正项取 minimum，再对平均值取负。两种写法在代数上等价。

当 $\epsilon = 0.2$ 时，裁剪区间为 $[0.8, 1.2]$ 。

4.5.1 正 advantage

设 $\hat{A} = +2$ 。提高 ratio 是有利方向，因为它让好动作更可能被采样。

- 当 $r = 1.1$ 时，两项都等于 2.2；
- 当 $r = 1.3$ 时，普通乘积为 2.6，裁剪乘积为 $1.2 \times 2 = 2.4$ ，PPO 选择 2.4；
- 当 $r = 0.7$ 时，普通乘积为 1.4，裁剪乘积为 1.6，PPO 选择较小的 1.4。

目标函数不再奖励过大的有利增幅，但不会抹掉有害的概率下降。

4.5.2 负 advantage

设 $\hat{A} = -2$ 。降低 ratio 是有利方向，因为它让坏动作更不容易被采样。

- 当 $r = 0.7$ 时, 普通乘积为 -1.4 , 裁剪乘积为 $0.8 \times (-2) = -1.6$, PPO 选择较小的 -1.6 ;
- 当 $r = 1.3$ 时, 普通乘积为 -2.6 , 裁剪乘积为 -2.4 , PPO 选择 -2.4 。

对于负 advantage, 有利方向是向下, 因此平台出现在 0.8 以下, 而不是 1.2 以上。

下面这个独立程序打印正负 advantage 的结果:

```
def clipped_surrogate(ratio: float, advantage: float, epsilon: float = 0.2) -> float:
    clipped_ratio = min(max(ratio, 1.0 - epsilon), 1.0 + epsilon)
    return min(ratio * advantage, clipped_ratio * advantage)

def main() -> None:
    for advantage in (2.0, -2.0):
        print(f"advantage = {advantage:+.1f}")
        for ratio in (0.7, 1.0, 1.3):
            value = clipped_surrogate(ratio, advantage)
            print(f"  ratio={ratio:.1f}, surrogate={value:+.1f}")

    assert clipped_surrogate(1.3, 2.0) == 2.4
    assert clipped_surrogate(0.7, -2.0) == -1.6

if __name__ == "__main__":
    main()
```

裁剪并不保证新策略一定更好。它只改变当前采样 batch 上的训练目标。网络仍可能在本批次没有覆盖的观测上发生变化; 多个参数会相互作用; critic 可能不准确; 奖励也可能没有正确描述目标篮球技能。

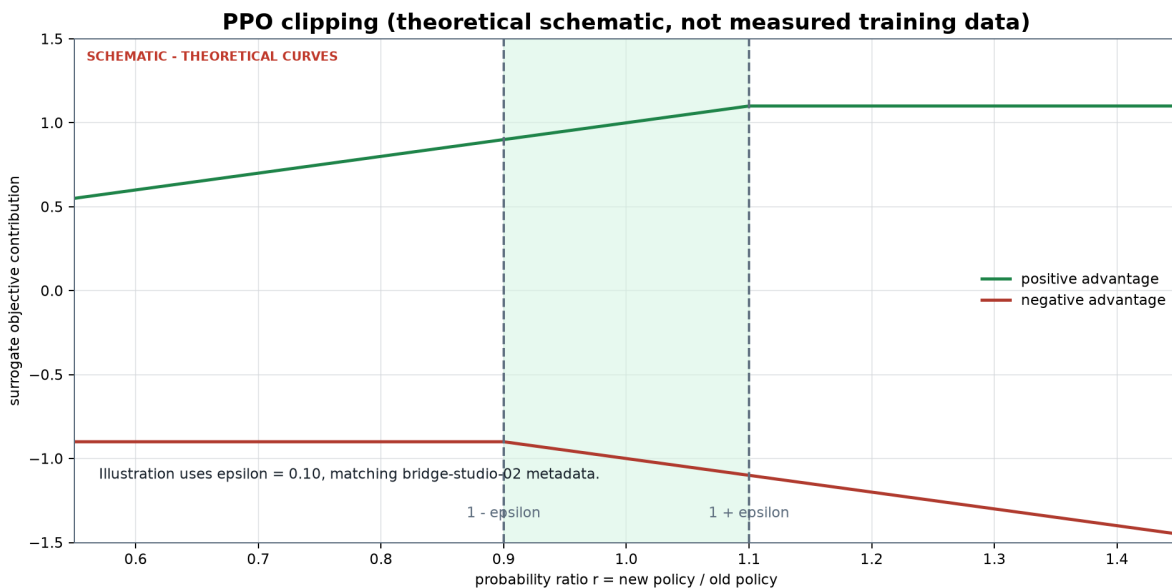


图 1: PPO 裁剪曲线示意图。该图用于解释概念, 不是实测训练证据。

4.6 广义优势估计

一步 temporal-difference residual 为

$$\delta_t = r_t + \gamma b_t - V(o_t),$$

其中 b_t 表示该 transition 之后允许使用的 bootstrap value。广义优势估计（Generalized Advantage Estimation, GAE）按时间反向组合 residual：

$$\hat{A}_t = \delta_t + \gamma \lambda m_t \hat{A}_{t+1}.$$

其中：

- γ 对未来 return 进行折扣；
- λ 控制后续 residual 对当前估计的影响强度；
- m_t 是 continuation mask，用于阻止 trace 穿过回合重置。

使用仓库默认值 $\gamma = 0.99$ 、 $\lambda = 0.95$ 时，trace 乘数为

$$\gamma \lambda = 0.9405.$$

较小的 λ 更依赖 critic 的一步预测；较大的 λ 会把更多采样到的未来信息向前传播，通常也带来更大的采样方差。不存在对所有任务都最佳的固定取值。

Critic 的训练目标为

$$\hat{R}_t = \hat{A}_t + V(o_t).$$

Actor 使用 $\hat{A}_t$ ，critic 拟合 $\hat{R}_t$ 。

4.7 Terminal 与 time limit 使用不同 mask

Gymnasium 区分两类边界：

- **termination**：所建模的任务真正结束，例如定义中的失败条件；
- **truncation**：由于外部原因停止收集，常见情况是时间上限，但底层任务本来仍可继续。

两类边界都会导致环境 reset，但它们不应使用相同的 value bootstrap。

对 transition t ，定义：

$$b_t = \begin{cases} 0, & \text{terminated,} \\ V(o_{t+1}^{\text{final}}), & \text{truncated 但未 terminated,} \\ V(o_{t+1}), & \text{回合继续.} \end{cases}$$

对于 termination 或 truncation，GAE trace mask 都为零：

$$m_t = 1 - \mathbf{1}[\text{terminated} \vee \text{truncated}].$$

这里存在两个不同的问题：

1. **当前 residual 能否 bootstrap 一个 value?** 纯超时可以，真正 terminal 不可以。
2. **Advantage 能否继续传播到下一条已保存 transition?** 两种边界都不可以，因为下一条观测可能来自 reset 后的新回合。

篮球实现为每条 transition 保存 `bootstrap_values`。收集过程中，它在 reset 前的 final observation 上调用 `critic`，然后在 `terminated` 为 true 的位置显式把 value 替换成零。它的 `compute_gae()` 使用：

- `~terminated` 作为 bootstrap mask;
- `~(terminated | truncated)` 作为 trace continuation mask。

通用 MLX 实现通过另一种方式得到相同语义。`_truncation_values()` 只选择 `truncated` & `~terminated` 的环境，并计算 `info["terminal_observation"]` 的 value。共享 GAE helper 在 truncation 时使用 timeout value，在 termination 时使用零，其余情况使用普通 next value。它绝不会从 autoreset observation 进行 bootstrap。

4.7.1 边界计算示例

假设：

$$r_t = 1, \quad V(o_t) = 2, \quad V(o_{t+1}^{\text{final}}) = 5, \quad \gamma = 0.99.$$

如果 transition 是真正 terminal:

$$\delta_t = 1 + 0 - 2 = -1.$$

如果它只是时间上限：

$$\delta_t = 1 + 0.99(5) - 2 = 3.95.$$

如果 autoreset observation 的 value 为 99，错误使用它会得到

$$1 + 0.99(99) - 2 = 97.01,$$

这会把新回合的 value 错误分配给旧回合。

下面的程序计算三步 trace，并证明 reset 后回合中的奖励不能向前泄漏：

```
from typing import Sequence
```

```
def gae(
    rewards: Sequence[float],
    values: Sequence[float],
    bootstraps: Sequence[float],
    terminated: Sequence[bool],
    truncated: Sequence[bool],
    gamma: float = 0.99,
    gae_lambda: float = 0.95,
) -> tuple[list[float], list[float]]:
    size = len(rewards)
    if not all(len(x) == size for x in (
        values, bootstraps, terminated, truncated
    )):
        raise ValueError("all inputs must have equal length")

    advantages = [0.0] * size
    trace = 0.0
    for step in range(size - 1, -1, -1):
        bootstrap_mask = 0.0 if terminated[step] else 1.0
```

```

        continuation_mask = (
            0.0 if terminated[step] or truncated[step] else 1.0
        )
        delta = (
            rewards[step]
            + gamma * bootstraps[step] * bootstrap_mask
            - values[step]
        )
        trace = delta + gamma * gae_lambda * continuation_mask * trace
        advantages[step] = trace

    returns = [a + v for a, v in zip(advantages, values)]
    return advantages, returns

def main() -> None:
    advantages, returns = gae(
        rewards=[0.5, 0.2, 0.0],
        values=[1.0, 1.2, 1.1],
        bootstraps=[1.2, 1.1, 0.9],
        terminated=[False, False, False],
        truncated=[False, False, True],
    )
    print([round(x, 6) for x in advantages])
    print([round(x, 6) for x in returns])

    changed_next_episode_reward = 1000.0
    assert changed_next_episode_reward > 0.0 # It is deliberately unused.
    assert round(advantages[-1], 6) == -0.209
    assert [round(x, 6) for x in advantages] == [
        0.586836, -0.107565, -0.209
    ]

if __name__ == "__main__":
    main()

```

预期 advantage 为:

```
[0.586836, -0.107565, -0.209]
```

超时 transition 从 0.9 bootstrap, 但其 continuation mask 为零, 因此 reset 后的新回合 advantage 不会进入该 trace。

4.8 Advantage 归一化

不同任务或 batch 中的原始 advantage 可能具有非常不同的尺度。两个实现都会归一化 advantage:

$$\hat{A}'_t = \frac{\hat{A}_t - \text{mean}(\hat{A})}{\text{std}(\hat{A}) + 10^{-8}}.$$

篮球训练器在进入 epoch 循环前, 对完整 rollout 统一归一化一次。通用 MLX 训练器在选择 minibatch 后, 于 `_loss_and_metrics()` 内归一化, 因此每个实际 minibatch 使用自己的均值和标准差。

归一化会改变尺度, 也可能改变某个样本相对于所选数据组均值的正负号。它不会修改奖励、return 或旧 log 概率。因此教学表格中的 +2 advantage 并不意味着训练时一定仍然是 +2。

4.9 Critic 损失

篮球续训使用普通的二分之一平方误差:

$$L_V = \frac{1}{2} \text{mean} \left[(V_{\text{new}} - \widehat{R})^2 \right].$$

它不会裁剪 value 更新。

通用 MLX PPO 使用阈值为 10 的 Huber 风格误差：

$$h(e) = \begin{cases} \frac{1}{2}e^2, & |e| \leq 10, \\ 10|e| - 50, & |e| > 10. \end{cases}$$

当 `clip_value_loss=True` 时，它还会构造一个裁剪后的预测：

$$V_{\text{clip}} = V_{\text{old}} + \text{clip}(V_{\text{new}} - V_{\text{old}}, -\epsilon, \epsilon),$$

随后取未裁剪 Huber 误差与裁剪分支 Huber 误差中的较大值。选择较大值可以避免裁剪分支把变化过大的 value 预测伪装成较好的结果。

这是一个重要的实现差异。“RLX PPO 使用 value clipping”对 `rlx/rlx/algorithms/ppo.py` 成立，但对篮球专用的 `ppo_update()` 不成立。

4.10 Entropy 以及它为何可以为负

Entropy 用于鼓励探索。对一个高斯坐标，

$$H = \log \sigma + \frac{1}{2} \log(2\pi e).$$

对于相互独立的多个坐标，实现会把这些值相加。

这里使用的是微分熵 (**differential entropy**)，不是离散 Shannon entropy。微分熵可以为负。例如当 $\sigma = 0.03$ 时：

$$H_{\text{单维}} \approx -2.087,$$

因此 14 个相同维度的总 entropy 约为 -29.2 。

负值并不表示出现错误。它表示连续密度集中在较窄区域内，不代表负概率。

两个损失函数都减去 entropy bonus：

$$L_{\text{total}} = L_{\text{policy}} + c_V L_V - c_H H.$$

如果 H 为负，那么 $-c_H H$ 为正。优化器仍然按照微分熵的准确目标工作，不能用离散 entropy 的直觉简单判断这个符号。

篮球 actor 保存一个直接训练的标准差参数，使用时只把它下限裁剪到 $1e-6$ 。加载源 actor 后，代码会把该参数填充为请求的 `initial_std`，命令行默认值为 0.03 。因此续训会有意重置探索尺度，而不是保留源 checkpoint 中保存的标准差。

4.11 总 loss 不是 reward，两者都不能证明掌握技能

篮球默认总损失为

$$L = L_{\text{policy}} + 1.0L_V - 0.01H.$$

通用 PPO 默认使用 value 系数 **0.5** 和 entropy 系数 **0.01**，但具体示例程序可以覆盖这些值。

训练 loss 下降并不等于 episode reward 上升：

- policy loss 衡量最新 batch 上的裁剪概率目标；
- value loss 衡量 critic 预测误差；
- entropy 衡量分布的扩散程度；
- reward 由环境提供；
- mastery 是外部行为结论。

总 loss 可能在行为改善时上升，因为 critic 正在拟合幅度更大的 return target。总 loss 也可能下降，而机器人只是学会利用奖励漏洞。高 reward 仍可能表示错误持球、利用辅助物理，或者在日志时间范围之后失败。PPO 只会优化它收到的奖励和数据。

因此篮球策略需要独立评估：完整回合、物理任务指标、确定性导出策略测试，以及渲染后的人工检查。任何一个 PPO 标量指标都不能单独证明技能已经掌握。

4.12 篮球中的循环序列重放

篮球 actor 是 LSTM。时刻 t 的动作均值依赖：

$$\mu_t = f(o_t, h_{t-1}, c_{t-1}).$$

如果训练时打乱单个时间步，隐藏状态将不再与生成该动作时的隐藏状态一致。因此篮球 rollout 保存：

- 形状为 [time, environments, 61] 的观测；
- 采样动作和旧 log 概率；
- 旧 critic value、奖励、termination、truncation 和 bootstrap value；
- 每个时间步的 episode_starts；
- rollout 开始时的 LSTM hidden state 和 cell state。

每个 PPO epoch 中，`actor.forward_sequence()` 从保存的初始隐藏状态开始，并按时间顺序向前执行。在处理 `episode_starts` 为 true 的时间步之前，它会把对应环境的 hidden state 和 cell state 乘以零。这样既能重建循环状态，又能防止记忆跨越 reset 边界。

更新每个 epoch 使用完整序列。`ppo_update()` 中没有篮球 minibatch 循环。使用默认的 32 步和 4 个环境时，一个 rollout 包含：

$$32 \times 4 = 128 \text{ 条 transition.}$$

五个 epoch 最多会把同样的 128 条 transition 提交给优化器五次，除非 KL 提前停止先结束循环。这仍然只收集了 128 条新环境 transition，而不是 640 条。

Critic 是前馈网络，所以代码把输入展平为 [time*environment, 61] 进行计算，再把输出恢复成 [time, environment]。

4.13 通用 MLX PPO 的前馈 minibatch

`RolloutBuffer.reset()` 分配形状为 `[num_steps, num_envs, ...]` 的定长数组。它保存：

字段	用途
<code>observations</code>	选择每个动作时使用的输入
<code>next_observations</code>	环境返回的下一个输入
<code>actions</code>	采样动作
<code>rewards</code>	标量奖励
<code>terminations, truncations</code>	两类独立边界
<code>values</code>	旧 critic 预测
<code>log_probs</code>	旧策略的联合 log 概率
<code>advantages, returns</code>	已分配字段，但当前 PPO 路径把计算结果直接作为参数传给 <code>update()</code>

通用更新会展平前两个维度。使用库级默认值时：

$$16 \text{ 步} \times 4096 \text{ 个环境} = 65,536 \text{ 个样本.}$$

分成 16 个 minibatch 后，每个 minibatch 有 4,096 个样本。两个 epoch 会在每个 rollout 上产生 32 次优化器更新。

通用 Microduck 示例并不使用这些巨大的库级默认值。`rlx/examples/ppo_microduck.py` 定义 16 个环境、24 步、4 个 minibatch 和 5 个 epoch，再把这些值传给 `PPOConfig`。因此它的 rollout 有 384 条 transition，每个 minibatch 有 96 条，每个 rollout 产生 20 次优化器更新。

MLX 更新在每个 epoch 创建新的随机排列。对于前馈网络，这是合理的，因为每条保存的 observation 已经包含网络所需的完整输入。`RolloutBuffer` 不保存循环 hidden state，也不保存 sequence-start mask；如果不加修改就用于 LSTM，会丢失必要历史。

4.14 KL divergence: MLX 中是诊断，篮球中是停止规则

两个实现都计算

$$\widehat{D}_{\text{KL}} = \text{mean} [(r - 1) - \log r].$$

因为样本来自旧策略，这是一种常用的样本近似，与 $D_{\text{KL}}(\pi_{\text{old}} \parallel \pi_{\text{new}})$ 相关。其期望应当非负，但有限样本和浮点运算可能产生很小的不规则值。

两个实现对它的用途不同。

4.14.1 通用 MLX 行为

`rlx/rlx/algorithms/ppo.py`:

- 求指数前把 log ratio 裁剪到 `[-20, 20]`;
- 请求 metrics 时报告 `approximate_kl`;
- 报告 `clip_fraction`;
- 不定义 `target_kl`;
- 不根据 KL 停止 epoch 或 minibatch。

这里的 KL 是诊断指标，不是训练门控。

4.14.2 篮球行为

`ppo_microduck_basketball.py`:

- `target_kl` 默认值为 `0.02`;
- 在每个 epoch 的优化器更新前计算 KL;
- 如果保存的旧 log 概率已经与当前 actor 不一致，可以在 `epochs_completed == 0` 时停止;
- 每次优化器更新后重新计算完整循环序列;
- 计算更新后的 KL，并在超过目标时停止后续 epoch;
- 记录 `kl_early_stop`。

更新前的检查可以发现陈旧或不匹配的 rollout 数据。更新后的检查限制同一小批循环 rollout 被重复使用的次数。即使如此，它仍不保证每个状态上的策略变化都小于某个严格上限。

4.15 准确区分默认值与覆盖值

描述“PPO 默认值”时，必须说明具体是哪一层。

4.15.1 通用 `PPOConfig` 默认值

设置	值
<code>num_envs</code>	4096
<code>num_steps</code>	16
<code>gamma</code>	0.99
<code>gae_lambda</code>	0.95
<code>num_minibatches</code>	16
<code>update_epochs</code>	2
<code>normalize_advantages</code>	true
<code>clip_coefficient</code>	0.2
<code>clip_value_loss</code>	true
<code>entropy_coefficient</code>	0.01
<code>value_coefficient</code>	0.5
<code>max_grad_norm</code>	0.5

4.15.2 通用 Microduck 示例覆盖值

`rlx/examples/ppo_microduck.py` 把 `num_envs`、`num_steps`、`num_minibatches` 和 `update_epochs` 覆盖为 16、24、4、5。它保留 `gamma 0.99`、`lambda 0.95`、`clip 0.2` 和 `entropy` 系数 `0.01`。优化器学习率默认值为 `1e-3`，它属于示例参数，不属于 `PPOConfig`。

4.15.3 篮球训练器默认值

篮球训练器不会实例化 `PPOConfig`。其命令行与函数默认值共同得到：

设置	值
环境数	4
rollout 步数	32
update 次数	5

设置	值
每个 rollout 的 PPO epoch	5
gamma	0.99
GAE lambda	0.95
clip 系数	0.2
value 系数	1.0
entropy 系数	0.01
梯度范数上限	1.0
Adam 学习率	2e-5
target KL	0.02
重置后的探索标准差	0.03

环境构造还显式覆盖了训练条件：

- 回合上限：10 秒；
- actuator：默认 "bam"；
- observation noise：关闭；
- domain randomization：关闭；
- pushes：关闭；
- curriculum：关闭；
- hold assistance：0.0；
- command：默认固定为 (0.08, 0.0, 0.0)，除非选择 `--randomized-commands`。

这些选择描述的是默认本地续训实验，不是通用 PPO 建议，也不等于完整复现上游训练配方。

4.16 一次完整更新的全过程

对于默认篮球运行：

1. 使用确定性的 seed 偏移重置四个环境。
2. 创建形状为 [1, 4, 256] 的 LSTM hidden tensor 和 cell tensor。
3. 收集 32 个控制步，共得到 128 条 transition。
4. 在每一步，对当前 observation 属于新回合的环境清零循环状态。
5. 采样 14 维高斯动作，并保存其联合旧 log 概率。
6. 分别推进每个环境。
7. 在 reset 前的 final observation 上计算 bootstrap value，并把 terminal 位置强制设为零。
8. 重置已结束环境，并把下一条 observation 标记为 episode start。
9. 使用 termination 和 truncation mask 反向计算 GAE。
10. 对 128 个 advantage 统一归一化。
11. 从已保存的初始 LSTM 状态重放整个 actor 序列。
12. 检查更新前的 approximate KL。
13. 计算裁剪 policy loss、平方 critic loss 和高斯 entropy。
14. 反向传播，把 actor 与 critic 的联合梯度范数裁剪到 1.0，再执行 Adam 更新。
15. 再次重放序列，计算更新后的 KL，并判断是否允许进入下一 epoch。

命令行默认请求五次这样的 rollout/update 循环：

$$4 \times 32 \times 5 = 640 \text{ 条本地环境 transition.}$$

这个很小的默认规模适合短续训或 smoke-scale 运行，不能作为篮球技能已经掌握的证据。

4.17 常见解释错误

错误：“高斯返回了 3.9 的概率，不可能。”

纠正：返回的是密度。概率是区间下的面积。

错误：“正 log 概率无效。”

纠正：连续 log 密度可以为正，因为密度可以大于 1。

错误：“裁剪会把所有 ratio 都限制在 0.8 到 1.2。”

纠正：目标函数使用裁剪后的比较项，实际 ratio 仍可位于区间外。

错误：“环境 reset 了，所以 timeout 的未来 value 应当是零。”

纠正：纯时间上限从 reset 前的 final observation bootstrap，然后停止 trace。

错误：“负 entropy 表示负概率。”

纠正：连续分布的微分熵可以为负。

错误：“总 loss 更低就表示 reward 更高。”

纠正：总 loss 混合了一个 batch 上的 policy、critic 和 entropy 项。

错误：“reward 高就表示技能已经掌握。”

纠正：reward 是人为设计的训练信号，mastery 需要独立物理标准和可视检查。

错误：“篮球 updater 使用 MLX minibatch。”

纠正：它是独立的 PyTorch 循环 updater，每次重放完整序列。

错误：“两个 PPO 实现都会在 KL 0.02 时停止。”

纠正：只有篮球路径具有该目标和提前停止；通用 MLX PPO 只报告 KL。

4.18 练习

1. 一个高斯分布的均值为 0、标准差为 0.2。计算均值位置的密度。结果可以大于 1 吗？
2. 某动作的旧 log 密度为 -3.0 ，新 log 密度为 -2.8 。计算 PPO ratio。
3. Advantage 为 $+3$ 、ratio 为 1.4 、clip 系数为 0.2 。计算未裁剪乘积、裁剪乘积和最终选中的 surrogate。
4. 把第 3 题的 advantage 改为 -3 ，重新计算。
5. 某 transition 的 reward 为 2，当前 value 为 4，final-observation value 为 5，gamma 为 0.9。分别计算真正 terminal 与纯 timeout 的 TD residual。
6. 为什么 LSTM rollout 必须保存初始 hidden state 和 episode-start mask？
7. 使用通用 PPOConfig 默认值时，每个 rollout 会产生多少次优化器更新？
8. 使用篮球默认值时，每个 rollout 收集多少条新 transition？这些数据最多会提交给优化器多少次？
9. 训练日志显示总 loss 下降、reward 上升。给出两个仍然不能证明篮球 mastery 的理由。
10. 用一句话说明通用 MLX 与篮球实现的 KL 行为差异。

4.19 答案

1. 均值位置密度为 $1/(0.2\sqrt{2\pi}) \approx 1.995$ 。可以大于 1，因为它是密度，总面积仍然为 1。
2. $r = \exp(-2.8 - (-3.0)) = \exp(0.2) \approx 1.2214$ 。
3. 未裁剪乘积为 $1.4 \times 3 = 4.2$ ；裁剪 ratio 为 1.2；裁剪乘积为 3.6；PPO 选择较小的 3.6。
4. 未裁剪乘积为 $1.4 \times (-3) = -4.2$ ；裁剪乘积为 $1.2 \times (-3) = -3.6$ ；PPO 选择更小的 -4.2 。裁剪不会保护有害方向上的变化。
5. Terminal: $2 - 4 = -2$ 。Timeout: $2 + 0.9(5) - 4 = 2.5$ 。

6. 同一个 observation 在不同 hidden state 下可以产生不同动作均值。序列重放必须重建与已保存动作对应的状态，reset mask 必须阻止记忆跨越回合边界。
7. $\text{update_epochs} \times \text{num_minibatches} = 2 \times 16 = 32$ 次优化器更新。
8. $4 \times 32 = 128$ 条新 transition。最多执行五次完整序列优化，但 KL 提前停止可能更早结束。
9. Reward 可能漏掉重要物理要求或存在可利用的捷径；评估也可能只覆盖短回合或带辅助的回合。此外，loss 与 reward 本来就不是同一个量。
10. 通用 MLX PPO 把 approximate KL 作为指标报告但不会据此停止，而篮球实现会在每个完整序列 epoch 前后检查 KL，并在超过默认目标 **0.02** 时停止后续 epoch。

4.20 参考文献

- John Schulman、Filip Wolski、Prafulla Dhariwal、Alec Radford、Oleg Klimov, **Proximal Policy Optimization Algorithms**, arXiv:1707.06347, 2017。 <https://arxiv.org/abs/1707.06347>
- John Schulman、Philipp Moritz、Sergey Levine、Michael Jordan、Pieter Abbeel, **High-Dimensional Continuous Control Using Generalized Advantage Estimation**, arXiv:1506.02438, 2015。 <https://arxiv.org/abs/1506.02438>

论文元数据和摘要已经通过本地保存的 arXiv 页面核对。本章关于仓库实现的陈述则单独来自对开头所列本地源码的核对。

5 篮球：在自由滚动球体上的盲感知循环平衡

5.1 学习目标

学完本章后，你应当能够：

- 根据仓库中的数值重建篮球仿真场景；
- 解释共享的 61 维观测中每一个切片，以及全部 14 个动作；
- 说明为什么动作网络看不到球的显式状态，却仍然需要循环记忆；
- 跟踪隐藏状态和细胞状态在重置、采样、PPO 重放、ONNX 导出、评估与 Studio 播放中的生命周期；
- 使用代码中的公式和系数计算每一个本地奖励项；
- 解释六级物理辅助课程，并明确它不是“奖励课程”；
- 区分发布演员网络热启动、完整优化器续训和真正从零训练；
- 运行经过当前参数解析器核对的训练、评估、渲染与 Studio 适配器命令；
- 应用物理评估门槛，并诚实报告保留结果：**6/6 持续平衡，0/3 指令滚动**；
- 明确边界：转向尚未解决，横向和偏航转向尚未评估，不能据此证明过桥，也没有硬件结论。

5.2 证据与结论边界

本章只使用仓库代码和仓库内保留的产物。主要依据如下：

问题	仓库来源
共享观测、关节顺序、时序和默认姿态	microduck_local/src/microduck_local/contract.py
动作施加、关节速度单控制步延迟和观测拼装	microduck_local/src/microduck_local/walk_env.py
篮球场景、辅助力、终止、奖励和指标	rlx/rlx/environments/basketball.py
LSTM 演员、本地评论家、源文件校验、ONNX 导出和一致性	rlx/rlx/models/basketball.py
本地循环 PPO 和直接命令行	rlx/examples/ppo_microduck_basketball.py
首次跌倒物理评估器	rlx/scripts/eval_basketball_local.py
Studio 训练、评估、渲染和导入适配器	rlx/examples/ppo_microduck_balance.py
Studio 循环策略加载器	microduck_local/src/microduck_local/studio_policies.py
面向人的结果摘要	docs/basketball-showcase/RESULTS.md
保留的直接评估证据	docs/basketball-showcase/evidence/evaluation.json
最新保留的 Studio 证据	rlx/runs/studio/basketball/basketball-balance-01/evaluation.json

这些保留产物生成于 2026 年 9 月 10 日。最新 Studio 报告使用种子 101、102、103，分别在零指令和 **0.15 m/s** 前进指令下评估 60 秒。六个试验全部维持了平衡，但三个带指令试验都没有通过受控滚动。适配器还明确写明：横向和偏航指令试验尚未实现。因此：

已演示的是在自由篮球上的无辅助持续平衡，不是受控滚动、通用转向、过桥、爬上篮球、倒地恢复或硬件验证。

microduck-playground 中的源篮球发布报告了规模大得多的上游评估，但那不是本地训练器的结果。本章严格区分源策略证据、本地证据和 Studio 证据。

本章代码块标签

- **源码节选：**用于学习的真实仓库代码片段。可能省略外围导入或类上下文，因此不是可独立执行的完整程序。

- **可执行命令**：在所需环境和资产齐备时，可从工作区根目录运行的 shell 命令。训练命令会创建新产物。
- **记录文本 / 数据**：仓库证据中的原样路径、序列或元数据值，不是命令。

5.3 用数字搭建物理场景

5.3.1 单位

实现使用 MuJoCo 常见的公制量，变量名和指标名也直接标明了单位：

物理量	单位	例子
长度和位置	米, m	球半径 0.12 m
质量	千克, kg	球质量 0.62 kg
时间	秒, s	控制间隔 0.02 s
线速度	米每秒, m/s	指令 0.15 m/s
角度	内部使用弧度	关节偏移和偏航指令
角速度	弧度每秒, rad/s	偏航和球旋转速度
报告中的倾角	度	评估要求小于 50 deg
力	牛顿, N	辅助弹簧力和阻尼力
力矩	牛顿米, N m	辅助力矩

弧度等于弧长除以半径， $\pi \text{ rad} = 180 \text{ deg}$ 。策略输出的 14 个动作是关节角偏移，单位为弧度，不是电机力矩。

5.3.2 求解器、地面、球和视觉网格

场景从带完整碰撞的 Microduck 模型出发，再加入地面和一个自由刚体：

SCHEMATIC - NOT OBSERVED EVIDENCE

Basketball balance geometry

Dimensions shown are model constants; robot drawing is illustrative.

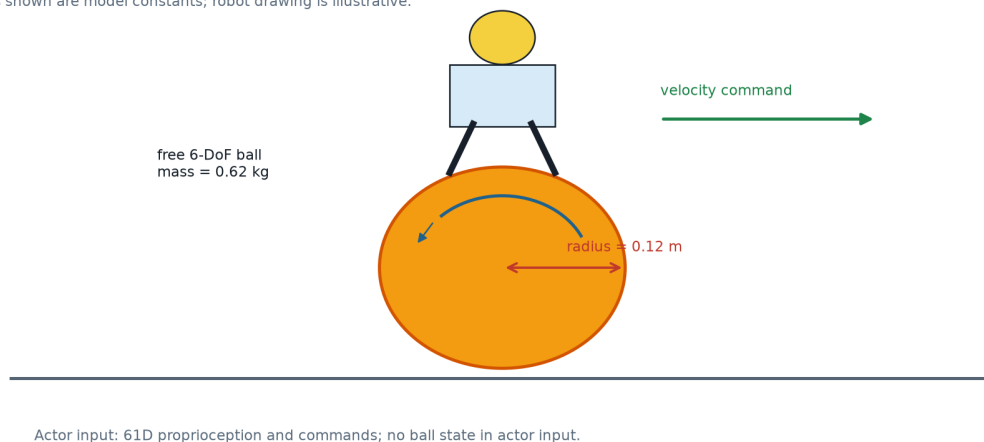


图 2: 使用实现中的半径和初始高度绘制的篮球几何示意图

源码节选（不可独立执行）：`rlx/rlx/environments/basketball.py`

```
# rlx/rlx/environments/basketball.py
BALL_RADIUS = 0.12
BALL_MASS = 0.62

spec.option.timestep = C.PHYSICS_DT
spec.option.integrator = mujoco.mjtIntegrator.mjINT_IMPLICITFAST
spec.option.cone = mujoco.mjtCone.mjCONE_PYRAMIDAL
spec.option.iterations = 10
spec.option.ls_iterations = 20

spec.worldbody.add_geom(
    name="floor", type=mujoco.mjtGeom.mjGEOM_PLANE,
    size=[0, 0, 0.05], material="basketball_ground_mat",
    friction=[1, 0.005, 0.0001],
)

ball = spec.worldbody.add_body(
    name="basketball", pos=[0, 0, BALL_RADIUS + 0.001]
)
ball.add_freejoint(name="basketball_freejoint")
ball.add_geom(
    name="ball_sphere", type=mujoco.mjtGeom.mjGEOM_SPHERE,
    size=[BALL_RADIUS, 0, 0], mass=BALL_MASS,
    friction=[1.2, 0.01, 0.001], priority=1, condim=4,
)
```

球心初始高度是 $z = 0.121$ m，即半径 0.12 m 再加 1 毫米。物理球体几何负责质量和接触。另一个网格几何只负责篮球外观，其 $mass=0$ 、 $contype=0$ 、 $conaffinity=0$ ，不能暗中支撑机器人。纹理和网格来自 `microduck-playground/src/mjlab_microduck/robot/assets/basketball/`。

地面使用棋盘纹理和摩擦参数 $[1, 0.005, 0.0001]$ 。球使用 $[1.2, 0.01, 0.001]$ 、接触维度 4 和优先级 1。这些是仿真接触参数，不是某一种真实地板或篮球的实测认证。

自由关节给篮球七个构型量：三个位置坐标和四元数的四个分量；同时给出六个速度量：三个线速度和三个角速度。测试锁定完整模型为 $nq=28$ 、 $nv=26$ 、 $nu=14$ 。

5.3.3 时间步

物理步长为

$$\Delta t_{\text{physics}} = 0.005 \text{ s.}$$

每个策略动作保持四个物理步：

$$\Delta t_{\text{control}} = 4(0.005) = 0.02 \text{ s,}$$

因此策略频率为

$$f_{\text{control}} = \frac{1}{0.02} = 50 \text{ Hz.}$$

一次 60 秒评估恰好包含

$$\frac{60}{0.02} = 3000$$

个控制步。

5.3.4 初始几何

球心位于 0.121 m。机器人根节点高度设为

$$z_{\text{root}} = 2R + 0.128 = 2(0.12) + 0.128 = 0.368 \text{ m.}$$

因此根节点高出球心

$$0.368 - 0.121 = 0.247 \text{ m,}$$

高出球顶约

$$0.368 - (0.121 + 0.12) = 0.127 \text{ m.}$$

重置测试把 `root_above_ball_m` 锁定为 0.247 。

机器人从球顶附近的直立状态开始。根节点水平位置在 ± 0.01 m 内采样，横滚和俯仰在 ± 2 deg 内采样；启用随机偏航时，偏航在 $[-\pi, \pi]$ 内采样；每个关节在默认姿态的 ± 0.05 rad 内扰动。篮球从静止开始。本任务不训练爬上球或从地面恢复。

5.4 精确的 61 维观测与 14 维动作契约

5.4.1 观测切片

演员网络始终接收一个 `float32[61]` 向量：

Python 切片	索引	宽度	含义
<code>0:3</code>	0–2	3	躯干角速度
<code>3:6</code>	3–5	3	投影到躯干坐标系的重力
<code>6:20</code>	6–19	14	关节位置减去默认姿态
<code>20:34</code>	20–33	14	延迟一个控制步的关节速度
<code>34:48</code>	34–47	14	上一个演员原始动作
<code>48:51</code>	48–50	3	前进、横向和偏航速度指令
<code>51:55</code>	51–54	4	头部指令填充，强制为零
<code>55:61</code>	55–60	6	身体指令填充，强制为零

代码直接按切片拼装：

源码节选（不可独立执行）：`microduck_local/src/microduck_local/walk_env.py`

```
# microduck_local/src/microduck_local/walk_env.py
obs = np.empty(C.OBS_DIM, np.float32)
obs[0:3] = gyro
obs[3:6] = gravity
obs[6:20] = joint_pos
obs[20:34] = joint_vel
obs[34:48] = self.last_action
obs[48:51] = self.twist_cmd
obs[51:55] = self.head_cmd
obs[55:61] = self.body_cmd
```

篮球环境把最后十个指令值设为零。训练器和测试都会拒绝 51:61 中的非零值。聚焦测试中，如果只移动篮球而不改变机器人本体感知，观测完全不变。因此演员网络对球的显式状态是“盲”的：61 个数里没有球位置、球速度、球旋转、球半径、接触标签或世界位置。

“盲”不等于毫无信息。移动球上的接触会改变躯干角速度、投影重力、关节状态和动作响应历史。LSTM 可以从时间序列中推断有用的隐藏动力学。

5.4.2 关节与动作顺序

14 个动作使用固定的舵机顺序：

动作索引	关节	默认姿态, rad
0	左髋偏航	0.0000
1	左髋横滚	-0.0873
2	左髋俯仰	-0.4579
3	左膝	-0.0049
4	左踝	0.4530
5	颈部俯仰	0.3491
6	头部俯仰	0.3491
7	头部偏航	0.0000
8	头部横滚	0.0000
9	右髋偏航	0.0000
10	右髋横滚	0.0873
11	右髋俯仰	0.4579
12	右膝	0.0049
13	右踝	-0.4530

对演员原始输出 $\mathbf{a}_t \in \mathbb{R}^{14}$ ，实际位置目标为

$$\mathbf{q}_t^{\text{target}} = \mathbf{q}^{\text{default}} + \text{clip}(\mathbf{a}_t, -4, 4).$$

观测保存的是原始动作，不是裁剪后的动作。这样，下一个观测和动作变化惩罚都能看到网络真正请求了什么。使用 BAM 执行器时，位置目标驱动电压式舵机模型，并包含继承的 3-6 个物理步总线延迟。四个物理步构成一个控制步，所以该内部延迟短于两个控制周期。

5.5 循环演员网络与状态生命周期

5.5.1 网络结构

本地演员网络为：

$$61 \rightarrow \text{LSTM}(256) \rightarrow 512 \rightarrow 256 \rightarrow 128 \rightarrow 14,$$

多层感知机使用 ELU 激活。对角高斯探索尺度包含 14 个可训练值，通常从 `initial_std=0.03` 开始。这里保存的是直接标准差参数，最低裁剪到 `1e-6`，不是对数标准差。

观测归一化计算

$$\hat{\mathbf{o}} = \frac{\mathbf{o} - \mu}{\sigma + 0.01}.$$

导入的源归一化器被冻结。其最后十个零填充槽的已存标准差为零，因此分母中的 $+0.01$ 保证除法有限。导出的 ONNX 内含该归一化器。

本地评论家不同于源模型卡所描述的上游发布训练评论家。本地 `BasketballCritic` 只接收同样的 61 维归一化观测，是一个全新初始化的前馈网络 $61 \rightarrow 512 \rightarrow 256 \rightarrow 128 \rightarrow 1$ ，没有球状态输入。

5.5.2 单步推理

源码节选（不可独立执行）：`rlx/rlx/models/basketball.py`

```
# rlx/rlx/models/basketball.py
def forward(self, observations, h_in, c_in):
    normalized = self.obs_normalizer(observations)
    recurrent, (h_out, c_out) = self.rnn.rnn(
        normalized.unsqueeze(0),
        (h_in, c_in),
    )
    actions = self.mlp(recurrent.squeeze(0))
    return actions, h_out, c_out
```

ONNX 接口是固定的：

- 输入：obs [1,61]、h_in [1,1,256]、c_in [1,1,256]；
- 输出：actions [1,14]、h_out [1,1,256]、c_out [1,1,256]；
- 数据类型：float32。

5.5.3 何时保留记忆，何时清零

在试验或回合开始时，隐藏状态 `h` 和细胞状态 `c` 都置零。每一个普通的 50 Hz 控制步都执行：

1. 把 obs、h_in、c_in 送入演员；
2. 施加 actions；
3. 把 h_out、c_out 作为下一次调用的输入。

普通指令变化不会清零记忆。试验重置、回合边界、策略启用或切换、恢复边界必须同时清零两个张量。

训练保存 `episode_starts[time, env]` 掩码。在采样和每次 PPO 重放中，代码都会在新回合位置把隐藏状态和细胞状态乘零：

源码节选（不可独立执行）：`rlx/rlx/models/basketball.py`

```
# rlx/rlx/models/basketball.py
for step in range(observations.shape[0]):
    carry = (~episode_starts[step].bool()).to(observations.dtype)
    carry = carry.view(1, -1, 1)
    hidden = hidden * carry
    cell = cell * carry
    actions, hidden, cell = self(observations[step], hidden, cell)
    outputs.append(actions)
```

这一点很重要，因为循环 PPO 不能把保存的观测当成互不相关的行。每个训练轮次都从采样段的初始隐藏状态和细胞状态出发，使用相同的回合掩码重算完整时间序列。评估器也会在每个试验开始时调用一次 `policy.reset()`，然后持续携带状态，直到首次跌倒或到达精确时长。

导出器执行 40 步 PyTorch/ONNX 一致性测试，并在第 20 步清零状态。保留的固定指令演员最大动作绝对误差为 $1.43\text{e-}6$ ，混合指令演员为 $2.15\text{e-}6$ 。

5.6 精确的本地奖励

5.6.1 符号

定义：

- $\mathbf{v}_{xy}$ ：躯干在身体坐标系中的前进和横向速度；
- $\mathbf{c}_{xy}$ ：前进和横向速度指令；
- ω ：躯干陀螺仪值；
- c_ω ：偏航角速度指令；
- $\mathbf{g}$ ：投影到躯干坐标系的重力；
- $\mathbf{d}_{xy}$ ：躯干位置减球位置后的水平分量；
- g_i ：第 i 只脚相对 BALL_RADIUS + 0.012 的径向间隙；
- h ：躯干高出球心的高度；
- $\mathbf{v}_b$ ：篮球水平速度；
- $\mathbf{a}_t - \mathbf{a}_{t-1}$ ：原始动作变化。

每个加权项都乘以 CTRL_DT = 0.02。总奖励为

$$r_t = 0.02 \sum_k w_k f_k.$$

5.6.2 实现中的公式和系数

奖励项	未加权公式 f_k	权重 w_k
线速度跟踪	$\exp(-\ \mathbf{v}_{xy} - \mathbf{c}_{xy}\ ^2 / 0.1)$	1.0
偏航跟踪	$\exp(-(\omega_z - c_\omega)^2 / 0.5)$	0.5
直立	$\exp(-\ (g_x, g_y)\ ^2 / 0.09)$	1.0
居中	$\exp(-\ \mathbf{d}_{xy}\ ^2 / 0.0016)$	2.0
双脚靠近球面	$\frac{1}{2} \sum_i \exp(-g_i^2 / 0.0004)$	1.0
高度	$\exp(-(h - 0.245)^2 / 0.0009)$	1.0
球速惩罚	$-\min(\ \mathbf{v}_b\ ^2, 100)$	0.05
角速度惩罚	$-\min(\omega_x^2 + \omega_y^2, 100)$	0.05
动作变化惩罚	$-\min(\ \mathbf{a}_t - \mathbf{a}_{t-1}\ ^2, 100)$	0.2

源代码足够紧凑，可以直接对照：

源码节选（不可独立执行）：rlx/rlx/environments/basketball.py

```
# rlx/rlx/environments/basketball.py
terms = {
    "linear_tracking": math.exp(
        -float(np.sum((self.body_lin_vel()[2] - self.twist_cmd[2]) ** 2)) / .1
    ),
    "yaw_tracking": math.exp(
        -float((self._gyro[2] - self.twist_cmd[2]) ** 2) / .5
    ),
    "upright": math.exp(-float(np.sum(gravity[2] ** 2)) / .09),
    "centered": math.exp(-float(np.sum(relative[2] ** 2)) / .0016),
    "feet_on_ball": float(np.mean(np.exp(-gaps ** 2 / .0004))),
    "height": math.exp(-float((relative[2] - BALL_RADIUS - .125) ** 2) / .0009),
    "ball_speed_penalty": -float(min(np.sum(ball_velocity ** 2), 100)),
```



```

    "angular_velocity_penalty": -float(min(np.sum(self._gyro[:2] ** 2), 100)),
    "action_rate_penalty": -float(
        min(np.sum((self.last_action - self.prev_action) ** 2), 100)
    ),
}
weighted = {
    key: value * REWARD_WEIGHTS[key] * CTRL_DT
    for key, value in terms.items()
}

```

奖励可以使用球、接触和几何等仿真特权状态，但这些量都不会进入演员观测。

5.6.3 奖励计算例题

假设某一步满足：

- 指令前进速度为 **0.15** m/s，实测前进速度为 **0.01** m/s；
- 横向误差为零；
- 根节点与球心水平偏移为 **0.02** m；
- 高度接近重置值， $h=0.247$ m；
- 原始动作变化平方和为 **0.04**；
- 其他正奖励项都等于 1，其他惩罚为零。

线速度跟踪为

$$f_{\text{lin}} = \exp\left(-\frac{(0.01 - 0.15)^2}{0.1}\right) = \exp(-0.196) \approx 0.8220.$$

其加权贡献为

$$0.8220(1.0)(0.02) \approx 0.01644.$$

居中奖励为

$$f_{\text{center}} = \exp\left(-\frac{0.02^2}{0.0016}\right) = e^{-0.25} \approx 0.7788,$$

因此贡献为

$$0.7788(2.0)(0.02) \approx 0.03115.$$

高度奖励为

$$f_{\text{height}} = \exp\left(-\frac{(0.247 - 0.245)^2}{0.0009}\right) \approx 0.9956,$$

贡献约为 **0.01991**。

动作变化惩罚为

$$(-0.04)(0.2)(0.02) = -0.00016.$$

如果所有正奖励都恰好为 1，所有惩罚都为零，则单步最大总奖励为

$$0.02(1 + 0.5 + 1 + 2 + 1 + 1) = 0.13.$$

这是稠密塑形分数，不是成功判据。评估器有意忽略奖励总和。

5.7 物理辅助课程

5.7.1 阶梯与转换规则

课程只有以下六级：

记录文本 / 数据（不可执行）： 实现中的辅助阶梯。

1.0 -> 0.5 -> 0.25 -> 0.1 -> 0.03 -> 0.0

一个回合结束后：

- 持续至少 6 s：向零辅助方向前进一级；
- 持续少于 1.5 s：向更强辅助方向退一级；
- 其他情况：保持当前级别。

转换在下次重置时应用，而且只有前一个回合确实结束才会更新。奖励权重从不改变。

5.7.2 辅助力做什么

辅助函数首先清空全部外力。hold=0 时立即返回。非零时施加：

源码节选（不可独立执行）： rlx/rlx/environments/basketball.py

```
# rlx/rlx/environments/basketball.py
ball_force = -400 * (ball_pos - self._ball_anchor) - 20 * ball_vel[:3]
ball_force[2] = -20 * ball_vel[2]
self.data.xfrc_applied[self.ball_body_id, :3] = self.hold * ball_force

self.data.xfrc_applied[self.ball_body_id, 3:] = (
    -self.hold * .2 * (ball_rotation @ ball_vel[3:])
)

duck_force = -40 * (
    self.data.xpos[self.trunk_body_id] - ball_pos
) - 4 * self.data.qvel[:3]
duck_force[2] = 0
self.data.xfrc_applied[self.trunk_body_id, :3] = self.hold * duck_force

self.data.xfrc_applied[self.trunk_body_id, 3:] = self.hold * (
    3 * np.cross(up, [0, 0, 1]) - .08 * angular_world
)
```

篮球获得指向锚点的水平弹簧阻尼支撑、没有竖直位置弹簧的竖直阻尼，以及旋转阻尼。躯干获得水平居中力，以及恢复直立和抑制角速度的力矩。

辅助力例题。 如果静止篮球沿正 x 方向偏移 0.02 m，未缩放的 x 力为

$$-400(0.02) = -8 \text{ N}.$$

在 $\text{hold}=0.25$ 时，实际 x 力为 -2 N；在 $\text{hold}=0$ 时严格为零。

保留的课程冒烟运行了 7,168 个环境步，使用两个环境，完成 39 个回合，并实际发生了升级和降级。结束时各环境位于 $\text{hold}=0.25$ 到 $\text{hold}=0.5$ 。它没有走完整个阶梯到零辅助，因此不能证明从零学会自由球平衡。

5.8 训练：热启动、PPO 与从零训练边界

5.8.1 实际加载了什么

直接训练器默认使用：

记录文本 / 数据（不可执行）： 默认源检查点路径。

`microduck-playground/artifacts/basketball/checkpoint.pt`

它用固定 SHA256 校验发布的源检查点和相邻 ONNX，然后加载 `actor_state_dict`，冻结源观测归一化器，并用 `--initial-std` 覆盖演员探索标准差。

它不会恢复源评论家、Adam 优化器、GPU 课程计数器或精确的 mujoco 仿真状态。实际代码是：

源码节选（不可独立执行）： `rlx/examples/ppo_microduck_basketball.py`

```
# rlx/examples/ppo_microduck_basketball.py
actor, source_metadata = load_source_actor(...)
critic = BasketballCritic(actor.obs_normalizer)
optimizer = torch.optim.Adam(parameters, lr=args.learning_rate)
```

保留摘要准确标记为：

记录文本 / 数据（不可执行）： 保留运行的元数据。

```
actor warm-start with fresh local critic and optimizer
full_upstream_resume: false
```

本地检查点也可以作为下一次演员热启动，但必须满足本地格式、冻结归一化契约、相邻 ONNX 和循环一致性要求。这仍然不是完整上游续训。

5.8.2 真正“从零”需要什么

本地命令行没有 `--scratch` 或随机演员选项。`train()` 总是调用 `load_source_actor()`。Studio 适配器也默认使用发布检查点或 `--init-from` 检查点。因此，仓库没有保留证据证明这套本地奖励、课程和 CPU PPO 能从随机演员发现篮球平衡。

成熟的 b11 演员是已演示平衡能力的来源，本地运行只是在它上面适配。辅助课程冒烟同样从该演员开始。真正的从零实验需要明确的随机初始化代码路径、声明过的初始化、冒烟测试和无辅助首次跌倒评估。本章不声称这些已经完成。

5.8.3 采样与 PPO 细节

若有 N 个环境、每段 T 步，则一次更新收集

$$N \times T$$

个转移。保留的固定指令运行使用 $N=8$ 、 $T=128$ 和 100 次更新：

$$8 \times 128 \times 100 = 102,400$$

个新的本地转移。

演员采样

$$\mathbf{a}_t = \mu_t + \epsilon_t \odot \sigma, \quad \epsilon_t \sim \mathcal{N}(\mathbf{0}, \mathbf{I}).$$

GAE 使用 $\gamma=0.99$ 、 $\lambda=0.95$ 。真正终止不进行价值自举；时间上限截断会在单步 TD 差分中使用最终状态的评论家价值，但不会让多步轨迹跨越重置。优势会标准化。

PPO 使用一个完整循环序列小批次、五个训练轮次、裁剪系数 0.2 、价值系数 1.0 、熵系数 0.01 、最大梯度范数 1.0 、目标 KL 0.02 。每个轮次都会重算完整 LSTM 序列。KL 检查可以停止后续轮次，但已经执行的一次优化器更新仍可能让更新后 KL 超过目标。

固定指令运行使用学习率 $2e-6$ 、初始标准差 0.03 、指令 $[0.15, 0, 0]$ 。混合指令运行是独立的 153,600 步热启动，使用学习率 $1e-5$ 、标准差 0.08 和直接训练器的随机采样器：25% 概率精确零指令，否则前进速度在 $[-0.15, 0.15]$ m/s、横向速度在 $[-0.1, 0.1]$ m/s、偏航速度在 $[-0.5, 0.5]$ rad/s 均匀采样。两次运行彼此独立，不是一条累计 256,000 步的训练链。

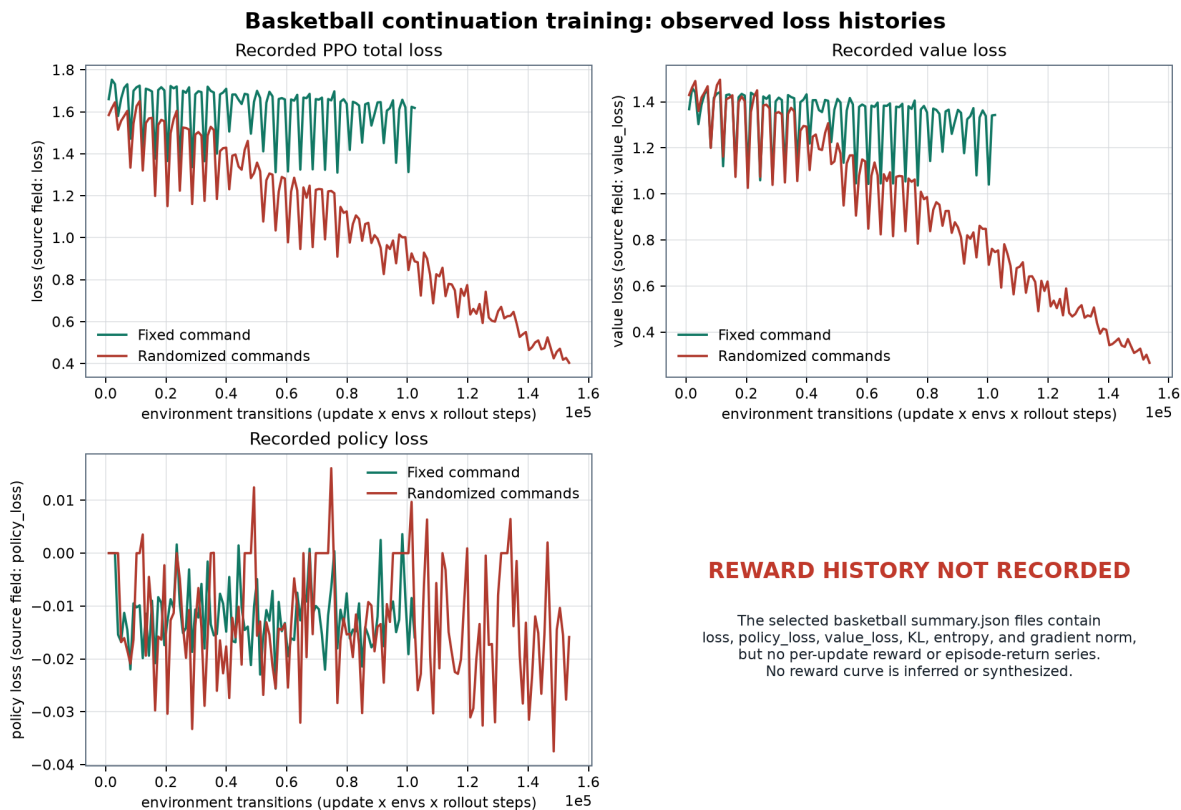


图 3: 两个独立篮球续训运行的实测 PPO 诊断；空奖励面板表示源摘要没有保存逐更新奖励历史

5.9 经过参数解析器核对的命令行流程

以下命令均从工作区根目录运行，其选项拼写已经按当前解析器核对。训练命令会创建新产物，并可能耗费较长时间。

5.9.1 聚焦契约测试

可执行命令（工作区根目录）：

```
rlx/.venv-microduck/bin/python -m pytest \
  rlx/tests/test_basketball.py \
  rlx/tests/test_basketball_policy.py \
  rlx/tests/test_basketball_evaluation.py -q
```

5.9.2 直接五次更新冒烟

必须使用新的输出目录；训练器拒绝覆盖非空运行。

可执行命令（工作区根目录）：

```
bash scripts/train-basketball.sh \
  --output rlx/artifacts/basketball-book-smoke \
  --num-envs 4 \
  --updates 5 \
  --steps 32 \
  --learning-rate 2e-6 \
  --initial-std 0.03 \
  --hold 0 \
  --no-curriculum \
  --command 0.15 0 0
```

该命令请求 $4 \times 32 \times 5 = 640$ 个新转移。它只检查流程接线，不是学习证据。

5.9.3 复现固定指令本地适配的规模

可执行命令（工作区根目录）：

```
bash scripts/train-basketball.sh \
  --output rlx/artifacts/basketball-book-fixed \
  --num-envs 8 \
  --updates 100 \
  --steps 128 \
  --learning-rate 2e-6 \
  --target-kl 0.02 \
  --save-interval 25 \
  --initial-std 0.03 \
  --hold 0 \
  --no-curriculum \
  --command 0.15 0 0
```

5.9.4 诚实的直接评估

重复使用 `--command` 是有意设计。

可执行命令（工作区根目录）：

```
rlx/.venv-microduck/bin/python rlx/scripts/eval_basketball_local.py \
  --policy rlx/artifacts/basketball-book-fixed/policy.onnx \
  --output rlx/artifacts/basketball-book-fixed-eval \
  --seconds 60 \
  --seeds 101 202 303 \
  --command 0 0 0 \
  --command 0.15 0 0 \
  --actuator bam \
  --render
```

聚合物理门槛失败时，评估器返回状态码 1，但仍会写出 `evaluation.json` 和失败媒体。指令滚动失败时，非零状态码是预期行为。

5.9.5 Studio 兼容训练

适配器会把请求步数向上取整到完整的 `num_envs x num_steps` 循环批次。下面的命令能够整除：

可执行命令（工作区根目录）：

```
rlx/.venv-microduck/bin/python rlx/examples/ppo_microduck_balance.py train \
  --recipe basketball \
  --output-dir rlx/runs/studio/basketball/basketball-book-fixed \
  --total-timesteps 102400 \
  --num-envs 8 \
  --num-steps 128 \
  --num-minibatches 1 \
  --learning-rate 2e-6 \
  --initial-std 0.03 \
  --target-kl 0.02 \
  --hold 0 \
  --no-curriculum
```

适配器固定使用指令 `[0.15, 0, 0]`、BAM 执行器、10 秒训练回合、冻结观测归一化、一个完整序列小批次，以及独立训练器规定的 PPO 常数。

5.9.6 Studio 评估和渲染

可执行命令（工作区根目录）：

```
rlx/.venv-microduck/bin/python rlx/examples/ppo_microduck_balance.py eval \
  --recipe basketball \
  --policy rlx/runs/studio/basketball/basketball-book-fixed/policy.onnx \
  --output-dir rlx/runs/studio/basketball/basketball-book-fixed \
  --num-envs 3 \
  --seed 101 \
  --max-episode-s 60 \
  --evaluation-mode skill
```

只要完整转向尚未评估，或滚动失败，适配器就返回状态码 2。它会对种子 101、102、103 分别评估零指令和 `0.15 m/s` 前进指令。

可执行命令（工作区根目录）：

```
rlx/.venv-microduck/bin/python rlx/examples/ppo_microduck_balance.py render \
  --recipe basketball \
  --policy rlx/runs/studio/basketball/basketball-book-fixed/policy.onnx \
  --output-dir rlx/runs/studio/basketball/basketball-book-fixed \
  --output rlx/runs/studio/basketball/basketball-book-fixed/render \
  --episodes 1 \
  --seed 101 \
  --render-seconds 60 \
  --width 1280 \
  --height 720
```

Studio 渲染器目前只录制带前进指令的情况。即使滚动失败，视频仍可作为持续平衡和漂移的有用证据。

5.10 评估：平衡不等于滚动

5.10.1 首次跌倒计数

评估不会自动重置。试验在第一次终止时停止，或者运行到精确时长。完整试验要求：

- 样本数恰好等于要求值；
- 没有无效物理指标；
- 没有终止或截断；

- 自动重置次数为零。

环境会因根节点过低、根节点与球心水平偏移超过 0.16 m、倾角超过 55 deg、机器人接触地面、脚以外的机器人部位接触篮球，或物理状态出现非有限值而终止。评估接受条件对几何更严格：最大倾角必须小于 50 deg，最大根节点与球心偏移必须小于 0.16 m。

5.10.2 持续平衡

只有同时满足以下条件，试验才算持续平衡：

$$\begin{aligned} & \text{完成全部时长} \wedge \text{脚球接触比例} \geq 0.5 \\ & \wedge \text{无身体接触球} \wedge \text{无机器人接触地面} \\ & \wedge \text{辅助始终为零} \wedge \text{几何稳定.} \end{aligned}$$

平衡不要求篮球停在固定世界位置。策略可能在篮球漂移或转动时仍保持平衡。

5.10.3 受控滚动

对于非零平面指令，受控滚动还要求：

1. 沿初始指令方向有足够的有符号球位移；
2. 前进和横向身体速度误差足够小；
3. 偏航角速度误差足够小；
4. 积分球旋转量足够大。

若指令速度为 s 、时长为 T 、球半径为 $R = 0.12$ ，则

$$d_{\min} = \max(0.25, 0.35sT),$$

$$e_{\text{linear,max}} = \max(0.04, 0.6s),$$

$$e_{\text{yaw,max}} = 0.5,$$

$$\theta_{\min} = \frac{\max(0.25, 0.35sT)}{R}(0.5).$$

60 秒门槛例题。当 $s=0.15$ m/s 时：

$$sT = 0.15(60) = 9.0 \text{ m},$$

$$d_{\min} = 0.35(9.0) = 3.15 \text{ m},$$

$$e_{\text{linear,max}} = 0.6(0.15) = 0.09 \text{ m/s},$$

$$\theta_{\min} = \frac{3.15}{0.12}(0.5) = 13.125 \text{ rad.}$$

仅有总路程不够。当沿指令方向的有符号位移相对要求距离或总路程过小时，评估器会把明显运动标记为随机移动。

5.10.4 最新保留的 Studio 结果

最新保留的 Studio 评估使用 BAM、动作延迟和随机初始偏航；不开启推力、观测噪声、质量或摩擦域随机化；辅助为零；不自动重置。

情况	试验数	持续平衡	受控滚动	该情况结论
指令 $[0, 0, 0]$	3	3/3	不是该情况判据	通过
指令 $[0.15, 0, 0]$	3	3/3	0/3	失败
全部试验	6	6/6	三个指令试验 0/3	完整任务失败

三个指令试验都同时因“沿指令方向位移不足”和“速度跟踪误差超限”失败。适配器报告：

源码节选（不可独立执行）：[rlx/examples/ppo_microduck_balance.py](#)

```
# rlx/examples/ppo_microduck_balance.py
forward_rolling_passed = (
    bool(rolling_cases) and all(case["passed"] for case in rolling_cases)
)
rolling_passed = False
success = False
failures.append(
    "full steering is not assessed: lateral and yaw command trials are not implemented"
)
```

这种强制失败是有意的保守设计。即使未来前进滚动通过，也不能证明完整转向。目前证据连前进滚动也没有通过。

5.11 阅读产物而不夸大结论

不同文件回答不同问题：

产物	能证明什么
summary.json	源哈希、设置、本地步数、演员变化、一致性和 PPO 诊断
checkpoint.pt	本地演员、新建本地评论家、优化器和运行元数据
policy.onnx	带冻结归一化器的确定性循环部署图
evaluation.json	每个物理试验、门槛、失败原因和聚合结论
rollout.mp4	一个已渲染种子和指令的可见运动
contact-sheet.png	按时间间隔抽样的整段视觉检查
Studio result.json	适配器级通过或失败以及来源信息

不能据此推断：

- 从篮球纹理旋转推断已学会转向；
- 从总路程推断沿指令方向直线运动；
- 从课程冒烟推断已经从零学习；
- 从演员热启动推断精确上游续训；

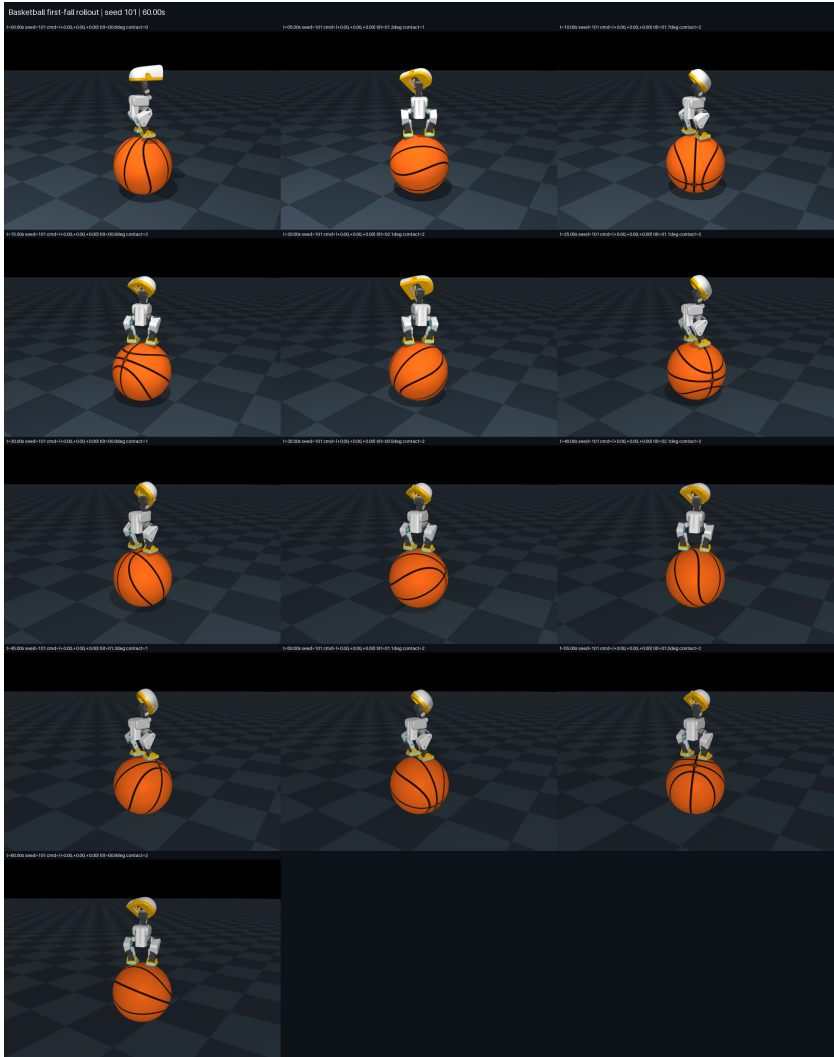


图 4: 种子 101 的实测零指令 60 秒接触表

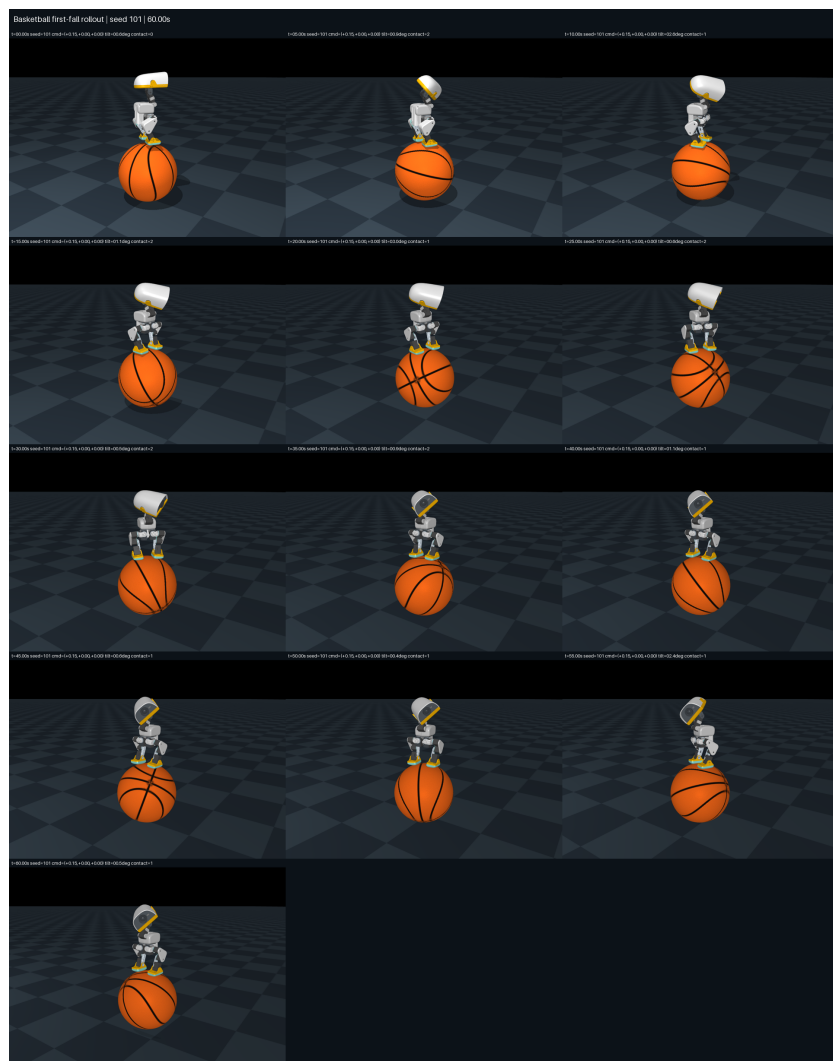


图 5: 种子 101 的实测 0.15 m/s 前进指令尝试; 平衡持续, 但滚动阈值失败

- 从 CPU MuJoCo 推断硬件安全；
- 从篮球平衡推断能够过桥。

对保留策略，正确的结论是：

它在保留的六个 60 秒 Studio 试验中全部维持了无辅助平衡；三个带前进指令的滚动试验全部失败；横向和偏航转向尚未评估；过桥和硬件行为不在这些证据范围内。

5.12 练习

1. 球半径为 0.12 m 。球直径是多少？
2. 在 50 Hz 下， 12 s 包含多少个控制步？
3. 偏航指令位于哪些观测索引？
4. 观测索引 55 是否包含球位置？
5. 第 4 号关节的原始动作为 0.20 rad ，默认踝关节角为 0.4530 rad 。忽略裁剪，目标角是多少？
6. 水平偏移为 0.04 m 时，居中奖励的未加权值是多少？
7. $\text{hold}=0.5$ 时，静止篮球沿 x 偏移 0.01 m 。施加给篮球的 x 辅助力是多少？
8. 某课程环境当前 $\text{hold}=0.25$ ，完成了 6.4 s 回合。下一回合的 hold 是多少？
9. 8 个环境、每段 128 步、25 次更新会收集多少个新转移？
10. 对 0.15 m/s 、60 秒的情况，为什么 1.30 m 有符号位移即使机器人保持直立也会失败？
11. 某试验完成 60 秒，脚球接触比例 90%，辅助为零，没有禁止接触，最大倾角 4 deg ，最大偏移 0.02 m 。篮球在前进指令下横向移动 4 m 。它是否通过持续平衡？是否通过受控滚动？
12. 为什么保留的 7,168 步课程冒烟不能证明从零学习？

5.13 参考答案

1. $2R = 2(0.12) = 0.24\text{ m}$ 。
2. $12/0.02 = 600$ 步。
3. 索引 50，即切片 48:51 的第三个值。
4. 不是。切片 55:61 是全零身体指令填充；演员没有显式球状态。
5. $0.4530 + 0.20 = 0.6530\text{ rad}$ 。
6. $\exp(-0.04^2/0.0016) = \exp(-1) \approx 0.3679$ 。
7. 未缩放力为 $-400(0.01) = -4\text{ N}$ ，乘以一半辅助后为 -2 N 。
8. 阶梯向零移动一级： $0.25 \rightarrow 0.1$ 。
9. $8 \times 128 \times 25 = 25,600$ 个转移。
10. 门槛要求至少 3.15 m 有符号位移，因此 1.30 m 不足。保持直立只能证明平衡，不能证明受控滚动。
11. 它通过持续平衡，因为这些条件满足平衡门槛；它不通过受控滚动，因为横向路程没有提供足够的有符号前进位移，而且通常也会导致速度跟踪失败。
12. 该冒烟从成熟发布演员开始，使用物理辅助，并在高于零的辅助级别结束。本地命令行没有初始化随机演员，也没有评估无辅助的从零策略。

6 悬索桥：在会移动的支撑面上学习行走

6.1 学习目标

学完本章后，你应当能够：

1. 用准确尺寸说明仿真桥面的自由木板、四根肌腱缆索、两端平台和无碰撞门架；
2. 区分“仿真物料表”和“经过认证的实体建造方案”；
3. 追踪不变的 61 维观测、14 维动作部署契约；
4. 解释为什么导航指挥器可以读取世界坐标，而学习到的 actor 仍只接收标准命令槽；
5. 推导继承自行走环境的全部奖励公式及真实权重；
6. 解释课程学习为何只改变物理辅助和出生位置，而不改变奖励；
7. 说明行走策略迁移过程：迁移 actor 与归一化器、冻结观测统计量、重新初始化 critic 和优化器；
8. 使用当前解析器支持的准确命令完成场景构建检查、训练、导出、评估和渲染；以及
9. 按严格的完整 20 秒规则判断过桥，不能把“走了一部分”写成“成功过桥”。

先给出本章最重要的结论：

已记录的 `bridge-studio-02` 试验进行了 32,768 个 PPO transition，但没有穿越悬索桥。它从起点算起的最佳进度为 1.175 m，低于 1.60 m 的进度门槛，也没有出现经过接触验证的双脚终点落地。本章记录的是可运行的仿真与训练协议，不是已经掌握的技能。

6.2 证据范围与结论边界

本章以当前仓库中的以下实现为准：

问题	权威来源
桥梁几何、指挥器、课程、接触与指标 严格过桥判据 行走 actor 迁移 Studio 的 train/eval/render/export 参数解析 共享观测与动作顺序	<code>rlx/rlx/environments/bridge.py</code> <code>rlx/rlx/environments/bridge_evaluation.py</code> <code>rlx/rlx/models/bridge_bootstrap.py</code> <code>rlx/examples/ppo_microduck_studio.py</code> <code>microduck_local/src/microduck_local/contract.py</code>
继承的奖励公式	<code>microduck_local/src/microduck_local/walk_env.py</code>
已记录试验与确定性评估	<code>docs/bridge-showcase/</code> 与 <code>rlx/runs/studio/bridge/bridge-studio-02/</code>

本地训练手册还规定了三条解释规则：

- 不能为了某个任务改变 61/14 接口；
- 不能把奖励值或 PPO 跑完当作行为成功的证据；
- 声称技能成功前，必须检查导出 ONNX 的确定性 rollout。

桥梁模型是一个本地 **MuJoCo 原型**。尺寸、质量、摩擦系数、肌腱刚度、阻尼和长度限制都是仿真参数。它们不是实体桥梁的测量结果，也不是缆索额定载荷、结构计算、安全系数或硬件认证。`bridge_plank_mat` 等材质名称只定义渲染颜色和外观，并没有指定木材树种、金属牌号或制造工艺。

6.3 仿真中真正存在什么？

任务把 Microduck 放在抬高的起点平台上。一块窄板跨过间隙，连接到终点平台。窄板不是焊死的、铰接的，也不是用动画脚本移动的。它带有 MuJoCo 自由关节，因此接触力能够让它平移，并产生滚转、俯仰和偏航。四条空间肌腱把木板上的移动连接点连到世界坐标中的固定锚点。

6.3.1 仿真物料表

下表是仿真对象清单，不是采购清单或施工图。

数量	仿真对象	准确模型参数
1	自由木板	1.10 x 0.13 x 0.03 m, 质量 0.45 kg, 初始中心 (0,0,0.225) m
2	固定平台	每个 0.60 x 0.48 x 0.24 m, 中心 x=-0.85 m 与 x=+0.85 m
4	空间肌腱缆索	刚度 320, 阻尼 2.0, 渲染宽度 0.0015 m, 带长度限制
4	固定锚点	(x,y,z)=(+/-0.45,+/-0.17,0.77) m
4	木板连接点	木板局部坐标 (x,y,z)=(+/-0.45,+/-0.057,0) m
2	门架横梁	0.028 x 0.48 x 0.028 m, 中心位于 x=+/-0.45 m、z=0.784 m
4	门架立柱	0.028 x 0.028 x 0.77 m, 位于 x=+/-0.45 m、y=+/-0.225 m
1	地面平面	世界坐标顶部 z=0, 低于平台顶面

MuJoCo 的 box size 使用半尺寸。例如木板源码中的 (0.55,0.065,0.015) 是半长、半宽和半厚，因此完整尺寸正好是表中的 1.10 x 0.13 x 0.03 m。

起点平台覆盖 x=-1.15 到 -0.55 m；木板覆盖 x=-0.55 到 +0.55 m；终点平台覆盖 x=+0.55 到 +1.15 m。在标称模型中，它们的边界恰好相接。三个表面的顶面都位于 z=0.24 m，所以木板中心高度为

$$0.24 - 0.015 = 0.225 \text{ m.}$$

6.3.2 四根缆索

在 x=-0.45 m 和 x=+0.45 m 两个纵向位置，各有左、右两根缆索。左侧锚点与连接点的横向距离为

$$+0.17 - 0.057 = 0.113 \text{ m,}$$

右侧的距离大小相同。竖直距离为

$$0.77 - 0.225 = 0.545 \text{ m.}$$

因此标称直线长度为

$$L = \sqrt{0.113^2 + 0.545^2} = 0.556591412079 \text{ m.}$$

源码把弹簧长度上界设为 $L-0.0015$ ，约为 0.555091 m；把受限最大长度设为 $L+0.008$ ，约为 0.564591 m。这些肌腱是 MuJoCo 中无质量、带弹性和阻尼参数的约束。代码没有模拟钢丝股、绳结、连接件、疲劳、断裂强度或锚点拔出。

若只做简化的线性弹簧估算，初始伸长量是 0.0015 m，对应

$$F = k\Delta L = 320(0.0015) = 0.48$$

个仿真力单位，尚未考虑阻尼和约束求解。这只是教学估算，不是实体缆索载荷等级。

6.3.3 门架可见，但不能接住机器人

横梁和立柱都使用 `contype=0`、`conaffinity=0`。MuJoCo 会把它们画出来，但碰撞系统不允许机器人站在门架上、靠在门架上或借门架获得支撑。四条 tendon 约束属于物理仿真；矩形门架几何只是无碰撞的视觉背景。

下面是经过删节的核心源码片段。它省略了外层循环，不能作为独立程序直接运行：

```
plank = spec.worldbody.add_body(
    name="bridge_plank",
    pos=(0, 0, 0.225),
)
plank.add_freejoint(name="bridge_freejoint")
plank.add_geom(
    name="bridge_surface",
    type=mujoco.mjtGeom.mjGEOM_BOX,
    size=(0.55, 0.065, 0.015),
    mass=0.45,
    friction=(1.15, 0.006, 0.0002),
    condim=4,
)

tendon = spec.add_tendon(
    stiffness=320.0,
    damping=2.0,
    springlength=(0.0, cable_length - 0.0015),
    limited=True,
    range=(0.0, cable_length + 0.008),
    width=0.0015,
)
tendon.wrap_site(f"{name}_anchor")
tendon.wrap_site(f"{name}_attach")
```

地面摩擦参数为 $(1.0, 0.005, 0.0001)$ ，平台为 $(1.1, 0.005, 0.0001)$ ，木板为 $(1.15, 0.006, 0.0002)$ 。它们是本地训练选择的参数，不是实体材料测试结果。

6.4 接触、运动与失败条件

仿真物理步长为 0.005 s，每个策略动作保持四个物理子步：

$$\Delta t_{\text{control}} = 4(0.005) = 0.02 \text{ s.}$$

因此 actor 的控制频率是 50 Hz。默认 20 秒 episode 包含 1,000 个控制步。

动作向量有 14 个值，顺序与共享关节契约一致。动作表示关节目标偏移：

$$\mathbf{q}^{\text{target}} = \mathbf{q}^{\text{default}} + \text{clip}(\mathbf{a}, -4, 4).$$

木板只通过仿真动力学运动，包括重力、肌腱力、接触力和可选课程辅助。任何控制步都不会把木板或机器人向前瞬移。`reset()` 会按照课程阶段把机器人放到声明过的出生点，并给木板很小的随机滚转、俯仰和角速度。

出现以下任一条件时，桥梁 episode 会终止：

- 任意机器人部位接触地面；
- 躯干倾斜超过 70 度；
- 躯干高度低于 0.10 m；
- 躯干横向位置超过 $|y| > 0.36$ m，并且双脚都没有有效支撑；
- 物理状态出现非有限数值。

达到普通时间上限属于 truncation，不等于跌倒。有效脚部支撑可以来自起点平台、木板或终点平台。如果手、头、躯干或其他非脚部位接触这些支撑面，系统会记录 `nonfoot_support`。

6.5 观测契约：特权指挥器与普通 actor

6.5.1 Actor 仍然只接收 61 个值

桥梁 actor 没有专门的桥面状态输入：

切片	宽度	含义
0:3	3	躯干角速度
3:6	3	投影到躯干坐标系的重力
6:20	14	相对 <code>DEFAULT_POSE</code> 的关节位置
20:34	14	延迟一个控制步的关节速度
34:48	14	上一个原始 actor 动作
48:51	3	机体坐标系前进、横移和偏航角速度命令
51:55	4	头部命令，Bridge 中为零
55:61	6	身体命令，Bridge 中为零

桥面位置、桥面速度、缆索长度、木板滚转/俯仰、世界位置、接触标签、过桥状态和辅助等级都不在观测中。测试会在机器人不动时把木板横向移动 0.03 m，然后确认 61 维观测逐字节不变。

Actor 和 critic 是两个独立的 61 → 512 → 256 → 128 ELU 网络，但二者接收相同的归一化 61 维向量。本实现没有给 critic 增加非对称的特权观测。特权仿真量只用于奖励计算、课程决策、日志和评估。

6.5.2 为什么指挥器是特权模块

导航指挥器运行在学习到的 actor 外部。它读取躯干世界坐标 x 、世界横向偏移 y 和世界航向，再把结果写入标准的三个 `twist_cmd` 槽。Actor 看到的是转换后的命令，而不是生成命令所用的世界测量值。

设水平前向单位向量为

$$\mathbf{h} = (h_x, h_y, 0),$$

机体左向为

$$\mathbf{s} = (-h_y, h_x, 0).$$

到达终点停止区域之前，期望世界速度是

$$\mathbf{d} = (0.25, \text{clip}(-1.2y, -0.12, 0.12), 0).$$

航向误差为

$$\psi = \text{atan2}(h_y, h_x).$$

写入观测的机体坐标命令为

$$\mathbf{c} = (\mathbf{d} \cdot \mathbf{h}, \mathbf{d} \cdot \mathbf{s}, \text{clip}(-1.5\psi, -0.6, 0.6)).$$

实现代码为：

```
reached_destination = (
    trunk_world_x >= END_PLATFORM_X - 0.05
) # 0.85 - 0.05 = 0.80 m

desired_world = np.array((
    0.0 if reached_destination else fixed_command[0],
    np.clip(-1.2 * trunk_world_y, -0.12, 0.12),
    0.0,
))

twist_cmd[:] = (
    desired_world @ heading,
    desired_world @ side,
    np.clip(-1.5 * yaw_error, -0.6, 0.6),
)
```

这个指挥器不是物理辅助：它不施加力，也不写关节动作。但它属于特权协议逻辑，因为它使用了 61 维观测中不存在的世界坐标定位。完整系统的能力声明必须把指挥器算进去，不能把全部导航能力都归功于 actor。

当躯干 $x \geq 0.80$ m 时，当前指挥器把期望世界前进速度设为零，同时保留横向和偏航修正。这个“终点停止”行为区分了最终的 **bridge-studio-02** 协议和更早的 **bridge-studio-01**；后者早于该停止逻辑。两个试验都是独立地从行走策略热启动并训练 32,768 个 transition，不能合并成一个 65,536 transition 的连续训练。

6.6 奖励：继承行走奖励，没有过桥大奖

BridgeEnv 没有覆盖 `_compute_reward`。桥梁 recipe 的可覆盖奖励键集合也是空集。因此所有课程阶段都使用完全相同的基础行走奖励：

$$r_t = r_{\text{lin}} + r_{\text{ang}} + r_{\text{upright}} + r_{\text{pose}} + r_{\text{head}} + r_{\text{air}} + r_{\Delta a} + r_{\omega xy}.$$

奖励中没有世界 x 、桥梁进度、木板角度、缆索张力、终点平台接触或 `bridge_crossed`。

记：

- $\mathbf{c} = (c_x, c_y, c_\omega)$ 为 `twist_cmd`；
- $\mathbf{v}_b = (v_x, v_y, v_z)$ 为躯干在机体坐标系中的真实线速度；
- $\boldsymbol{\omega} = (\omega_x, \omega_y, \omega_z)$ 为真实躯干角速度；

- $\mathbf{g}_b = (g_x, g_y, g_z)$ 为投影重力;
- $\delta q_j = q_j - q_j^{\text{default}}$;
- h_j 为头部命令, 在 Bridge 中为零。

真实实现的奖励项和权重如下。

6.6.1 线速度跟踪

$$e_v = (c_x - v_x)^2 + (c_y - v_y)^2 + v_z^2,$$

$$r_{\text{lin}} = 2 \exp(-e_v/0.1).$$

6.6.2 角速度跟踪

$$e_\omega = (c_\omega - \omega_z)^2 + \omega_x^2 + \omega_y^2,$$

$$r_{\text{ang}} = 2 \exp(-e_\omega/0.5).$$

6.6.3 直立

$$r_{\text{upright}} = 2 \exp[-(g_x^2 + g_y^2)/0.05].$$

6.6.4 默认腿部姿态

对十个腿部关节:

$$r_{\text{pose}} = \exp \left[-\frac{\sum_{j \in \text{legs}} \delta q_j^2}{0.5} \right].$$

6.6.5 头部姿态

对四个头部关节:

$$r_{\text{head}} = 2 \left[\frac{1}{4} \sum_{j \in \text{head}} \exp \left(-\left(\frac{\delta q_j - h_j}{0.5} \right)^2 \right) \right].$$

6.6.6 稠密腾空时间

每只没有支撑的脚每个控制步累加 0.02 s 腾空时间。如果运动命令幅值大于 0.01 ，且

$$0.125 < \tau_f < 0.300 \text{ s},$$

该脚在当前步贡献 1。因此

$$r_{\text{air}} = 3 \sum_{f \in \{L, R\}} \mathbb{1}(0.125 < \tau_f < 0.300).$$

Bridge 覆盖了脚部接触检测，因此起点平台、木板和终点平台都属于该继承奖励项中的有效支撑。

6.6.7 动作变化惩罚

$$r_{\Delta a} = -w_{\Delta a} \sum_{j=1}^{14} (a_{t,j} - a_{t-1,j})^2.$$

权重按照每个环境实例的累计控制步数变化：

环境步数: 0 12,000 18,000 24,000 30,000 36,000
权重: 0.1 0.2 0.4 0.6 0.8 1.0

已记录试验共有四个环境和 32,768 个总 transition，即每个环境 8,192 个控制步，所以整个试验都处于 0.1 权重阶段。

6.6.8 滚转/俯仰角速度惩罚

$$r_{\omega xy} = -0.05(\omega_x^2 + \omega_y^2).$$

两个 penalty 项按构造都不大于零。

6.6.9 数值例题

假设某一步的状态为：

命令 = (0.25, 0, 0)
机体速度 = (0.15, 0, 0.05) m/s
角速度 = (0.10, 0.05, 0.10) rad/s
投影重力 = (0.10, 0, 约 -0.995)
腿部误差平方和 = 0.20
头部误差 = (0.10, -0.10, 0, 0.20) rad
一只脚腾空时间 = 0.20 s, 另一只脚有支撑
动作变化平方和 = 0.08
动作变化惩罚权重 = 0.1

则：

奖励项	数值
track_lin_vel	1.764994
track_ang_vel	1.911995

奖励项	数值
upright	1.637462
pose	0.670320
head_pose	1.886861
feet_air_time	3.000000
action_rate_penalty	-0.008000
ang_vel_xy_penalty	-0.000625
总和	10.863007

这个奖励很高，但仍不能证明过桥。机器人可能在带辅助的早期出生点上获得它，也可能在到达终点前获得它。

6.7 只改变物理的课程学习

课程共有五个阶段：

阶段	辅助 α	机器人出生 x	含义
0	1.0	0.00 m	从木板中心附近开始，木板稳定辅助最大
1	0.6	-0.30 m	更长的木板路径，辅助减弱
2	0.3	-0.55 m	从木板与起点平台边界开始
3	0.1	-0.75 m	从起点平台内部开始
4	0.0	-0.90 m	与无辅助评估完全相同的出生点

辅助只作用于木板。设木板位移 $\Delta \mathbf{p} = \mathbf{p} - \mathbf{p}_0$ ，线速度为 $\mathbf{v}$ ：

$$F_{xy} = \alpha(-45\Delta p_{xy} - 4v_{xy}),$$

$$F_z = \alpha(-25\Delta p_z - 2.5v_z).$$

设木板向上轴为 $\mathbf{z}_p$ ，世界向上轴为 $\mathbf{z}_w$ ，世界角速度为 ω_w ：

$$\tau = \alpha [3.5(\mathbf{z}_p \times \mathbf{z}_w) - 0.18\omega_w].$$

当 $\alpha=0$ 时，外加力数组严格为零。课程不会推着机器人向前，不会修改动作，不会修改奖励系数，也不会直接给予过桥资格。

每个 episode 结束后：

```
if crossed:
    stage = min(stage + 1, 4)
elif elapsed_s < 2.0:
    stage = max(stage - 1, 0)
# 其他情况保持原阶段。
```

这符合训练手册中的探索原则：如果目标状态从未出现，应修改物理或出生条件，让它能够被采样，而不是增加稀疏大奖。最容易的阶段曾在确定性 `alpha_walking.onnx` 诊断中出现一次接触有效的过桥，但该尝试随后跌倒，因此它只是探索证据，不是严格技能通过。

6.8 从已发布行走 actor 迁移

当 Bridge 的完整训练没有指定 `--init-from` 且 `total_timesteps>4` 时，训练器会自动从 `microduck/policies/alpha_walking.onnx` 创建初始化 checkpoint。

Bootstrap 会验证：

- 输入名为 `obs`，形状 `(1,61)`；
- 输出名为 `actions`，形状 `(1,14)`；
- ONNX 节点序列完全匹配；
- tensor 名称、形状和 `float32` 类型匹配；
- 归一化均值有限、标准差为正；
- 迁移期间源策略哈希没有变化。

随后创建新的 MLX actor-critic。四个 actor 线性层从 ONNX actor 精确复制；观测均值与方差从 ONNX 归一化器复制；critic 按种子重新初始化；actor_log_std 根据 `initial_std=0.03` 新建；训练调用方创建全新的 Adam 优化器。

Actor **没有被冻结**。它只是热启动，之后仍会参与 PPO 更新。被冻结的是观测归一化统计量，这样迁移后的 actor 继续使用原行走权重熟悉的输入坐标系。

保存前，代码用 32 个代表性观测比较 MLX actor 和源 ONNX。最大允许动作绝对误差为 $1e-4$ 。bridge-studio-02 初始化器记录的实际最大误差为 $1.0728836e-6$ 。

准确的迁移总结是：

actor 均值网络权重:	迁移后继续训练
观测归一化器:	迁移并冻结
critic 权重:	全新、按种子初始化
actor 探索标准差:	全新，初值 0.03
优化器状态:	全新的 Adam

这不是从上游行走 PPO 训练中恢复完整优化器状态。

6.9 复现实验流程

以下命令都从工作区根目录运行。`scripts/train-bridge.sh` 默认使用已有的 `rlx/.venv-microduck` 环境。

6.9.1 构建并检查仿真场景

Studio 解析器没有 `build` 子命令。MuJoCo 模型由 `bridge_model()` 在运行时构建。下面的已检查命令创建一个环境并验证公开契约：

```
rlx/.venv-microduck/bin/python - <<'PY'
from rlx.environments.bridge import BridgeEnv

env = BridgeEnv(
    actuator="xml",
    curriculum=False,
    obs_noise=False,
    domain_rand=False,
    action_delay=False,
    random_yaw=False,
    seed=7,
)
obs, info = env.reset(seed=7)
print(obs.shape, env.action_space.shape, env.model.ntendon)
print(round(info["bridge_position"][2], 3), info["assistance"])
env.close()
```

PY

预期输出:

```
(61,) (14,) 4  
0.225 0.0
```

6.9.2 训练有界试验 recipe

```
./scripts/train-bridge.sh train \  
  --bridge-curriculum \  
  --total-timesteps 32768 \  
  --num-envs 4 \  
  --num-steps 128 \  
  --num-minibatches 4 \  
  --update-epochs 2 \  
  --learning-rate 1e-5 \  
  --gamma 0.99 \  
  --gae-lambda 0.95 \  
  --clip-coefficient 0.1 \  
  --entropy-coefficient 0 \  
  --max-grad-norm 0.5 \  
  --initial-std 0.03 \  
  --backend dummy \  
  --actuator xml \  
  --max-episode-s 20 \  
  --seed 7 \  
  --no-normalize-rewards \  
  --no-domain-rand \  
  --no-obs-noise \  
  --no-action-delay \  
  --no-random-yaw \  
  --output-dir rlx/runs/studio/bridge/my-bridge
```

训练会自动写入 `bridge.safetensors`、JSON sidecar、`bridge.onnx`、初始 snapshot 和 `training-metrics.jsonl`。是否额外生成周期 snapshot 由 `--checkpoint-interval` 决定。

Batch 数量可以准确计算:

$$4 \text{ 个环境} \times 128 \text{ 步} = 512 \text{ 个 transition/rollout.}$$

$$32768/512 = 64 \text{ 个 rollout.}$$

每个 rollout 有四个 minibatch, 并重复两个 epoch:

$$64 \times 4 \times 2 = 512 \text{ 个优化器 minibatch 更新.}$$

6.9.3 单独导出

训练默认会导出 ONNX。若只想重新生成确定性 ONNX actor:

```
./scripts/train-bridge.sh export \  
  --checkpoint rlx/runs/studio/bridge/my-bridge/bridge.safetensors \  
  --output rlx/runs/studio/bridge/my-bridge/bridge.onnx
```

导出文件包含观测归一化器和 actor 均值网络, 不包含 critic, 也不包含随机动作采样。

6.9.4 运行严格技能评估

```
./scripts/train-bridge.sh eval \
  --backend dummy \
  --num-envs 3 \
  --seed 101 \
  --policy rlx/runs/studio/bridge/my-bridge/bridge.onnx \
  --evaluation-mode skill \
  --eval-steps 1000 \
  --max-episode-s 20 \
  --actuator xml \
  --no-domain-rand \
  --no-obs-noise \
  --no-action-delay \
  --no-random-yaw
```

Bridge 技能评估拒绝 `fork` backend，因为它需要逐控制步物理指标。应使用 `dummy` 或 `subproc`。Skill 模式还要求完整 episode 时长的整数倍，并且每个 20 秒 episode 至少 1,000 步。如果技能失败，即使推理管线全程有限，命令也会以状态码 2 退出。

6.9.5 渲染并观察

```
./scripts/train-bridge.sh render \
  --policy rlx/runs/studio/bridge/my-bridge/bridge.onnx \
  --output rlx/runs/studio/bridge/my-bridge/render \
  --render-seconds 20 \
  --episodes 1 \
  --seed 101 \
  --actuator xml \
  --camera side \
  --width 640 \
  --height 360 \
  --fps 25 \
  --sheet-frames 12
```

渲染会强制关闭域随机化、观测噪声、动作延迟、随机偏航和桥面辅助。如果机器人在 20 秒结束前跌倒，渲染器会 `reset` 后继续录制；画面叠加层会显示尝试编号和 `reset` 次数。因此一个 20 秒视频可能包含多次失败尝试，而不是一个完整 episode。

6.10 严格的完整 20 秒过桥规则

只有当某一个控制步同时满足以下条件时，环境级 `crossing_accepted` 才会变为真并保持为真：

1. 躯干 $x \geq \text{END_PLATFORM_X} - 0.08 = 0.77$ m；
2. 双脚都接触终点平台；
3. 两只脚都没有接触地面；
4. 机器人任何部位都没有接触地面；
5. 没有非脚部位为机器人提供支撑；
6. 之前至少观察到一次脚与悬空木板的接触。

独立的 `BridgeEvaluation` 还要求：

- episode 在进度接近零的位置观察到起点平台脚部支撑；
- 最大进度至少 1.60 m；
- 每一个被测控制步的辅助都为零；
- 全程没有地面接触或非脚支撑；
- 观察到经过接触验证的 `crossing flag`；
- episode 一直运行到 1,000 步 truncation，中途不能 termination；
- 每个评估 lane、每个 episode 都通过。

进度从最终出生点计算：

$$p = x_{\text{trunk}} - (-0.90) = x_{\text{trunk}} + 0.90.$$

因此 1.60 m 进度门槛本身只对应 $x_{\text{trunk}} \geq 0.70$ m。它故意不是充分条件：更严格的 crossing flag 仍要求 $x \geq 0.77$ m、双脚终点平台接触、之前接触木板并且支撑历史干净。

提前过桥但之后跌倒仍然失败，因为没有完成要求的 20 秒时长。直接把机器人放到终点平台会因为缺少木板接触历史而失败。沿地面走到很大的 x 坐标也会失败。

6.11 当前试验实际完成了什么？

最新记录的最终试验是 `rlx/runs/studio/bridge/bridge-studio-02/`，生成于 2026 年 9 月 10 日。Checkpoint 中记录：

项目	数值
新 PPO transition	32,768
环境数	4
Rollout 长度	128
Minibatch 数	4
Update epoch	2
种子	7
执行器	XML
观测归一化器	迁移并冻结
奖励归一化	关闭
课程	只改变物理与出生点

确定性技能评估使用三个 lane，种子分别为 101、102、103；每个 lane 运行 1,000 个控制步；关闭随机化、噪声、延迟和随机偏航；使用 XML 执行器、零辅助和带终点停止的导航指挥器。

报告中有 11 次尝试，因为某些 lane termination 后会 reset，但整个 3,000 transition 评估继续运行。没有一次尝试完成完整 20 秒，也没有一次过桥。八次尝试接触了木板。最佳尝试从起点前进 1.174915875 m，距离评估进度门槛还差

$$1.60 - 1.174915875 = 0.425084125 \text{ m.}$$

没有任何尝试记录到脚站上终点平台。有一次尝试接触了地面。推理管线保持有限，但技能状态为 **failed**。

正确总结是：

该试验学习到或保留了部分接近木板和在木板上行走的行为，但没有观察到成功的无辅助完整过桥。

不能说 32,768 步“完成了课程”，不能用高 return 证明过桥，也不能说保留的视频是一段连续的 20 秒成功过桥。视频明确包含 reset 和不完整尝试。

6.12 练习

6.12.1 练习 1：缆索几何

使用锚点 (0.45, 0.17, 0.77) 和标称连接点 (0.45, 0.057, 0.225)，计算缆索长度。

6.12.2 练习 2：指挥器修正

假设躯干朝向世界 +x，位于 $y=0.08$ m，尚未到达终点。指挥器会发布什么命令？

6.12.3 练习 3：观测中的特权信息

木板滚转 5 度，但机器人状态和命令完全不变。61 维观测中哪些索引会因为木板滚转而改变？

6.12.4 练习 4：课程阶段转移

某环境当前处于阶段 3。分别判断以下情况后的下一阶段：

1. 8 秒时干净过桥；
2. 1.5 秒时失败；
3. 6 秒时失败。

6.12.5 练习 5：PPO 数量计算

总 transition 为 32,768，环境数 4，rollout 步数 128，minibatch 数 4，epoch 数 2。计算 rollout 数、每个 minibatch 的 transition 数和优化器 minibatch 更新总数。

6.12.6 练习 6：诚实的过桥判断

判断下列情况通过还是失败：

1. 进度 1.65 m，第 700 步 crossing flag 变真，无非法接触，第 1,000 步 truncation；
2. 同样的 crossing，但第 850 步跌倒；
3. 双脚在终点平台，但之前没有接触木板；
4. 完整运行到时限，crossing flag 为真，但最大进度只有 1.59 m；
5. 完美运行 20 秒，但其中一个控制步的辅助为 0.1。

6.12.7 练习 7：解释结果

策略的平均 return 高于初始化策略，但 `bridge_crossed.max=0`。可以声称什么？

6.13 参考答案

6.13.1 答案 1

纵向差为零，横向差为 0.113 m，竖直差为 0.545 m：

$$L = \sqrt{0^2 + 0.113^2 + 0.545^2} = 0.556591412079 \text{ m.}$$

6.13.2 答案 2

航向为 (1,0,0)，侧向为 (0,1,0)，并且 $y=0.08$ ：

$$d_y = \text{clip}(-1.2(0.08), -0.12, 0.12) = -0.096.$$

偏航误差为零，所以命令是：

(0.25, -0.096, 0.0)

6.13.3 答案 3

没有任何索引会直接变化。61 维向量中没有木板状态块。接触造成的后续机器人运动可能改变 IMU、关节或上一动作，但孤立的木板姿态属于特权信息。

6.13.4 答案 4

1. 过桥：进入阶段 4；
2. 2 秒前失败：退回阶段 2；
3. 较晚失败：保持阶段 3。

6.13.5 答案 5

每个 rollout 有 $4 \times 128 = 512$ 个 transition，共有 $32768 / 512 = 64$ 个 rollout。每个 minibatch 有 $512 / 4 = 128$ 个 transition。每个 rollout 的两个 epoch 共执行 $4 \times 2 = 8$ 次优化器更新，合计 $64 \times 8 = 512$ 次。

6.13.6 答案 6

1. 通过，前提是起点支撑和指标覆盖也有效；
2. 失败：在要求时长结束前 termination；
3. 失败：必须有悬空木板脚部接触历史；
4. 失败：进度低于 1.60 m；
5. 失败：任何一个带辅助的控制步都会使无辅助评估失效。

6.13.7 答案 7

如果比较条件受控，可以声称继承的行走目标 return 提高。不能声称完成过桥或掌握桥梁技能。最终结论仍由确定性物理评估决定。

7 看证据，不看庆祝语

7.1 建立证据链

SCHEMATIC - NOT OBSERVED EVIDENCE

Evidence preparation pipeline

Source artifacts remain distinct from extracted data, plots, and diagrams.



Rule: missing fields remain missing. No synthetic training series.

图 6: 证据制作示意图: 保留的运行记录经过结构检查与 CSV 提取, 生成实测曲线或复制的连续帧, 最终成为绑定哈希的教材资源。这不是实测轨迹。

训练程序完全正确地执行, 也可能学错行为; reward 上升, 平衡可能变差; ONNX 可能不同于训练 actor; viewer 可能加载错误场景。因此最终结论要经过多层独立检查: 模型和输入正确、参数确实更新、导出一致、无辅助评估诚实、实际观看画面。

独立很重要。如果把“成功”定义成 reward 高, 再展示同一个 reward 证明成功, 只是把一个定义检查了两次。接触合法性、无摔倒时长、方向位移和完整过桥, 才是对用户物理目标的独立验证。

7.2 阅读真实 reward 和 loss 曲线

篮球固定指令 `summary.json` 有 100 条更新, 混合指令有 150 条。包含 total loss、policy loss、value loss、entropy、approximate KL 和 gradient norm, **没有保存历史每次更新 reward 或 episode return**。这是证据缺口, 不能补画一条漂亮上升曲线。图中的 reward 缺失提示是有意保留的。

两次训练都从同一个成熟 actor 起步, 并新建 critic 与 optimizer。如果把固定指令后面接上混合指令, 画成累计曲线, 就伪造了连续训练关系。正确横轴分别到 102,400 与 153,600 个 transition。固定指令每次更新对应 $8 \times 128 = 1024$ 步新数据。

本版另附的篮球 reward 诊断图记录现有策略确定性执行时的逐步奖励。它是**新增诊断轨迹**, 不是找回的历史训练曲线, 也不是进一步优化的证据。CSV 与凭据记录策略、条件与实际时长, 不能把它拼接接到旧训练历史里。

独木桥日志包含 collection、update、completed episode 事件。collection 的 `mean_reward` 是每个 transition 的平均奖励; `mean_raw_return` 是一个回合奖励之和, 再对该事件中结束的回合求均值。回合更长, 即使每步表现没变, return 也可能更高。update 的 `mean_loss` 是优化器小批次目标的均值。这三者不可互换。

独木桥 total / value loss 图使用对数纵轴: 相同纵向距离表示相同倍数变化, 而不是相同差值。1,000 降到 100, 与 100 降到 10, 在对数轴上距离相同。负 policy loss 不能放到普通对数轴, 应单独显示。

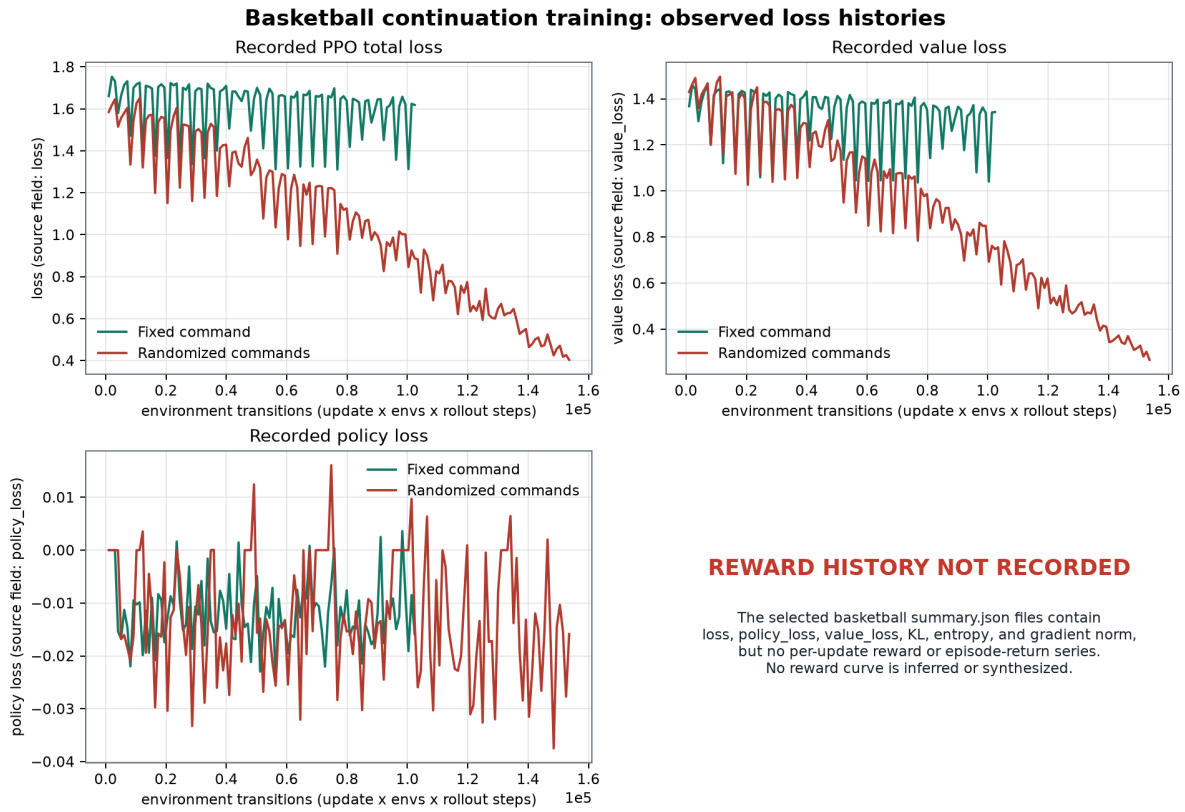


图 7: 真实篮球训练 loss, 明确标出历史 reward 缺失。两个策略是独立 warm start, 不是一条连续训练曲线。

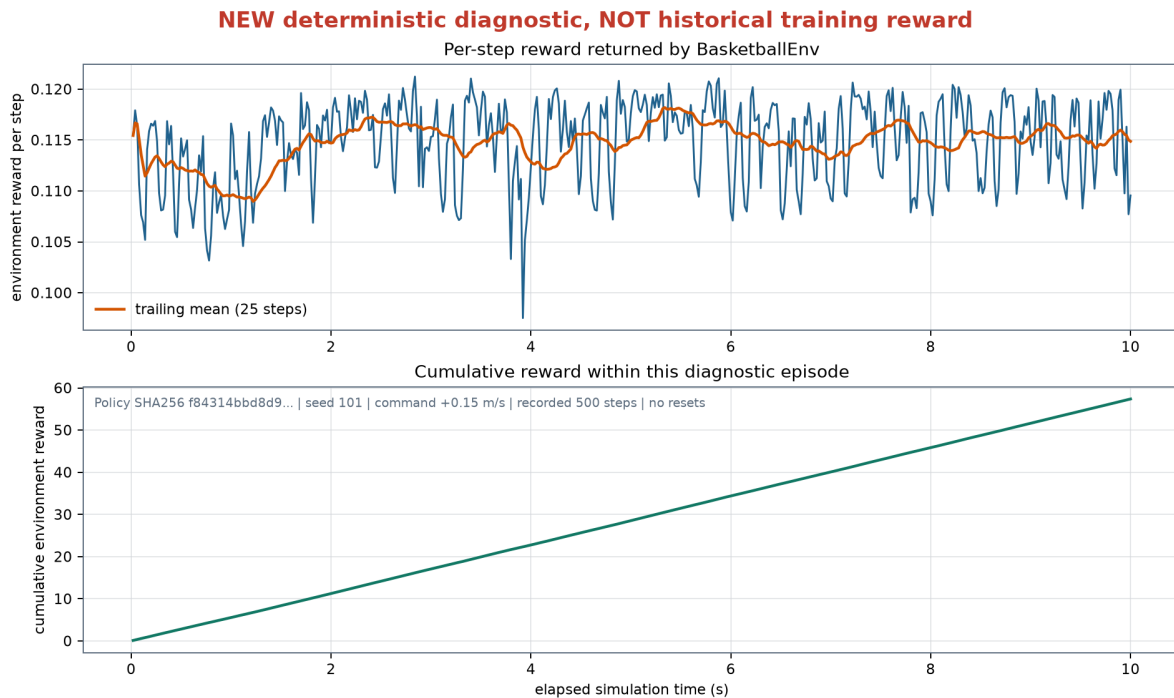


图 8: 新增确定性篮球 reward 诊断, 不是历史 PPO 训练奖励; 源策略没有改变。

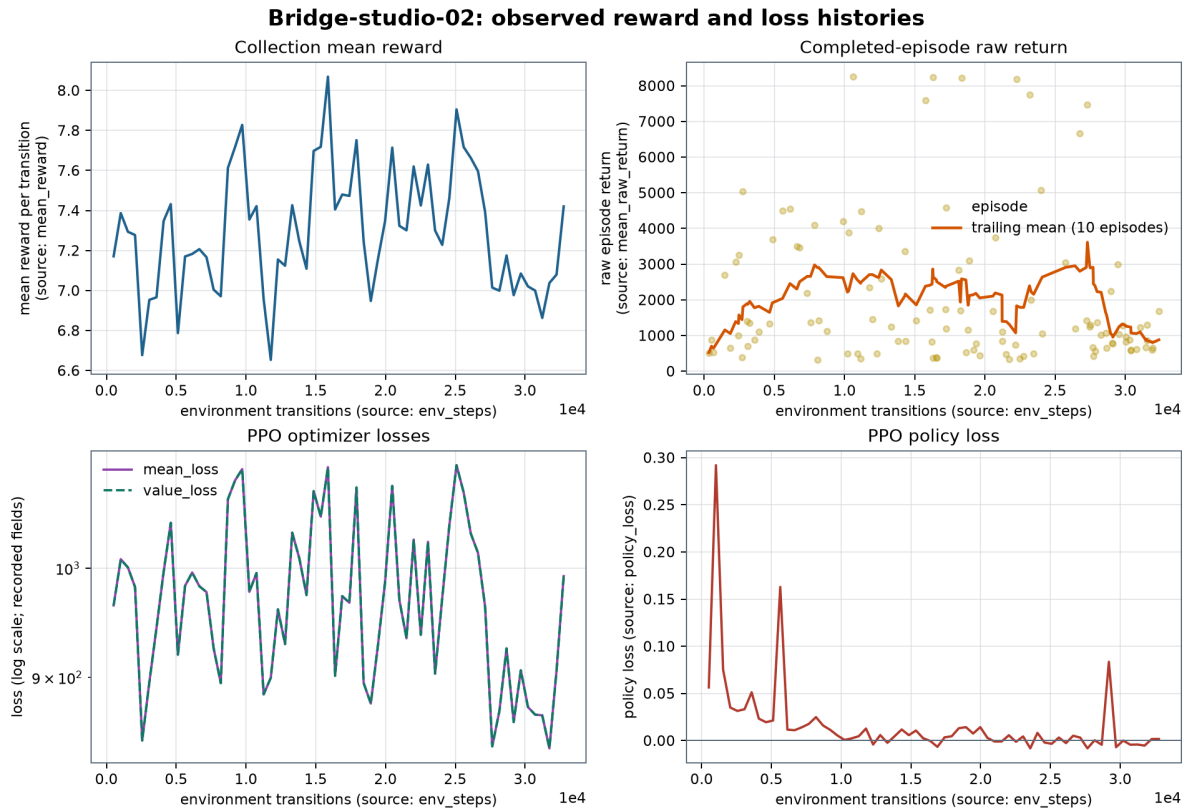


图 9: 真实独木桥 pilot 日志：每步平均 reward、完整回合 return 与 PPO loss 分开呈现。

Policy loss 很小不等于技能学会。优势归一化使均值接近零，而概率比接近一时，平均 surrogate 本来就可能接近零。早期 value loss 较大也不意外，因为 actor 成熟但 critic 是新建的。连续分布的 entropy 可以为负，见 PPO 章。Target KL 也不能保证单次优化绝不超限。

好图注说明五件事：哪次运行、哪个原始字段、横轴单位、是否变换 / 平滑、不能证明什么。本书 CSV 保留未经平滑的值，重新生成图不需要重新训练。

7.3 确定性评估：究竟通过了什么？

实测实验	结果	正确理解
篮球固定指令微调，六次 60 秒 同候选，三次前进指令 篮球混合指令微调 最终独木桥 pilot	6/6 持续存活 0/3 滚动通过 6/6 存活、0/3 滚动 0 次合格过桥	名义条件下自由球平衡已验证。 有运动不等于可靠受控滚动。 独立候选，不是固定指令的后续步数。 训练和导出工作正常；物理任务尚未完成。
七案例 API / Studio 检查	七场景可访问和显示	产品集成，不是七张技能证书。

直接篮球报告用种子 101、202、303；后来的 Studio 评估用 101、102、103。两者都有零指令和前进指令，但属于不同证据集。不能用另一个报告的截屏重标种子。直接报告的最小脚一球接触覆盖率约 99.93%，最大倾斜约 4.74 度；只描述该报告对应策略和条件。

历史源码边界：保留的直接篮球报告早于当前源码快照。报告中 evaluator / environment 哈希前缀为 fcfa65 / 2eda2a，本版提供的对应文件为 bbe735 / 6bff54。我们保留旧报告，但不把它说成用当前这些源码字节重新执行的结果。新增 10 秒 reward 诊断绑定当前环境哈希，却不能替代六次 60 秒

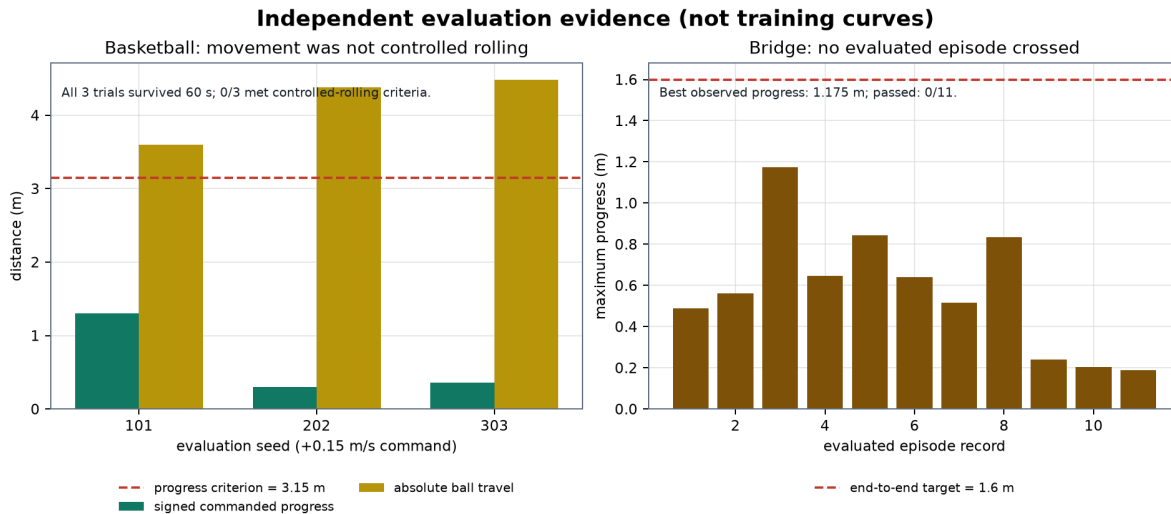


图 10: 记录中的评估结果，而非训练分数。篮球持续平衡通过；指令滚动与合格过桥没有通过。

验收。验证当前快照时，应按书中命令输出到新目录，保留新的哈希。复现协议和逐字节重建历史，是两种不同承诺。

固定指令 run 的前进跟踪平均绝对误差约 0.138 米 / 秒，目标为 0.15 米 / 秒；混合指令约 0.160 米 / 秒。即使平衡良好，这仍是较差的指令跟踪。与源策略约 0.140 米 / 秒相比的一点差异，不能证明稳健改善。

独木桥最大进度 1.174916 米相对于出发点计算，不能说“桥只有 1.10 米，已经走了 1.174916 米，所以完成了”。鸭子必须离开起点平台，产生合法木板接触，双脚到终点，避免非法支撑，并满足完整时长。最终评估含三个并行环境，总计 3,000 transition；reset 产生多次尝试和末尾片段。这些片段不是新增的合格 20 秒回合。

名义条件下三次或六次成功，不代表广泛可靠性。当前测试没有穷尽摩擦、质量、绳索刚度、指令方向、推力、传感器噪声、执行器温度、制造差异或硬件条件。左右侧向和 yaw 指令评估还没有实现。应该报告实际测了什么，而不是编造总体成功率。

7.4 视觉验证：看连续画面

不要只看鸭子是否在橙色球上方。躯干是否直立？脚是否真正支撑？地面或身体其他部位是否在提供支撑？纹理转动是否与运动一致？时钟是否连续、没有 reset？相机是否跟随得太紧，以致漂移被隐藏？

连续帧拼图仍是视频采样，可能漏掉快速抖脚或帧间事件。不确定时，按原帧率看完整 MP4，或以 50 Hz 控制频率渲染。结合接触与终止数据。接受技能前，“看起来成功”应与“数值满足规则”一致。

保留的独木桥视频长 20 秒，包含两次 reset，后续一次尝试走到木板中途。Reset 将机器人放回新出生状态，不能隐藏后算成连续进度。本书保留失败序列，是因为看清在哪个接触转换处失去平衡，对课程设计很有价值。

视频保留在工作区，未嵌入 PDF：

- docs/basketball-showcase/evidence/render/cmd-p0_000-p0_000-p0_000/seed-101/rollout.mp4
- docs/basketball-showcase/evidence/render/cmd-p0_150-p0_000-p0_000/seed-101/rollout.mp4
- rlx/runs/studio/bridge/bridge-studio-02/render/ep0.mp4

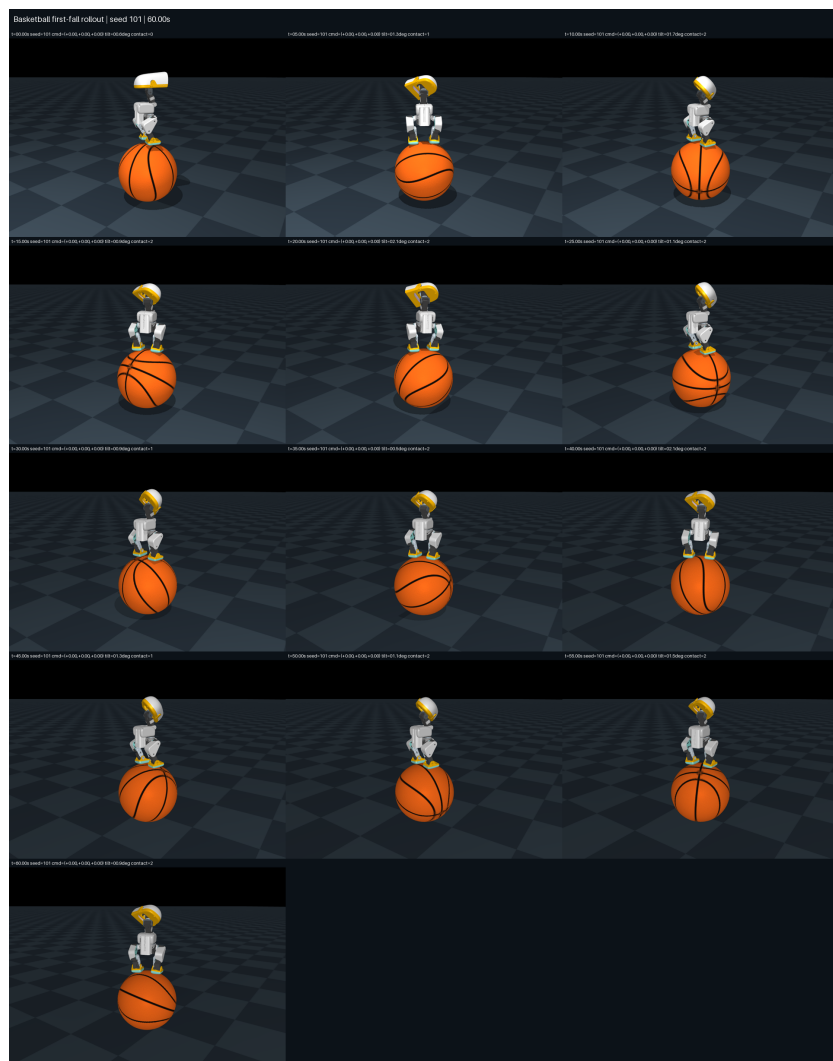


图 11: 种子 101 的实测零指令篮球序列。这证明该次持续平衡, 不证明受控行走。

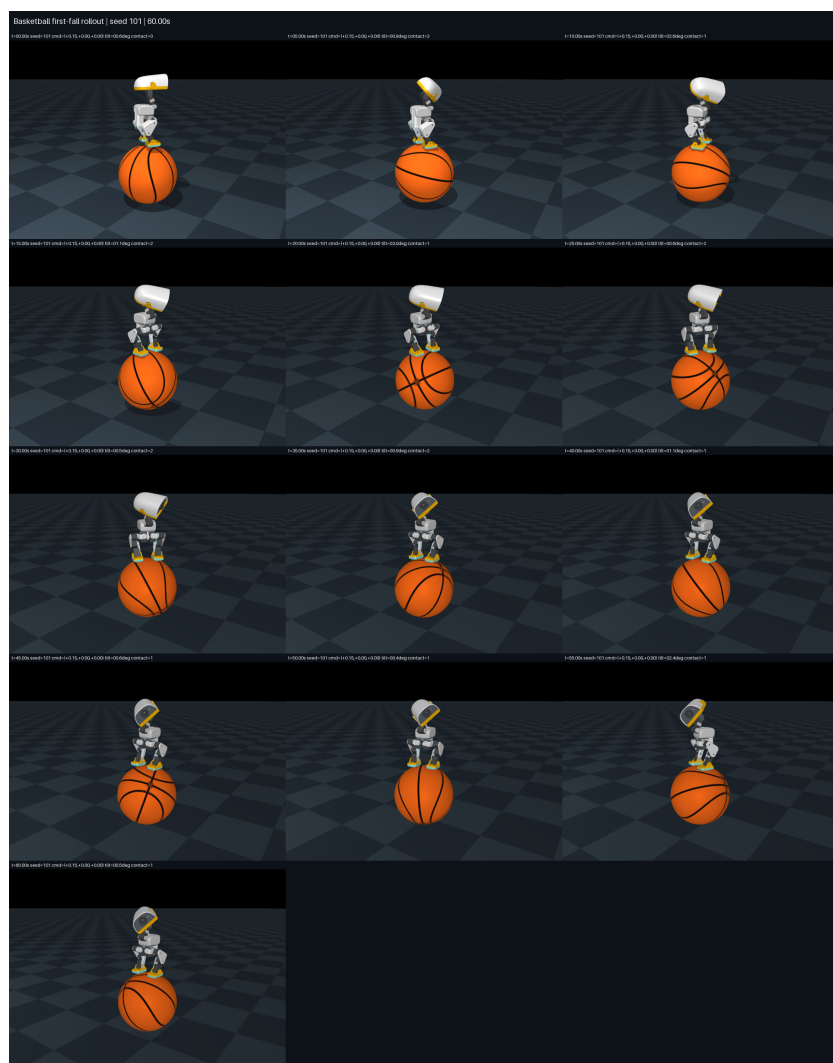


图 12: 实测前进指令尝试。鸭子保持平衡、球在动，但方向滚动门槛未通过。

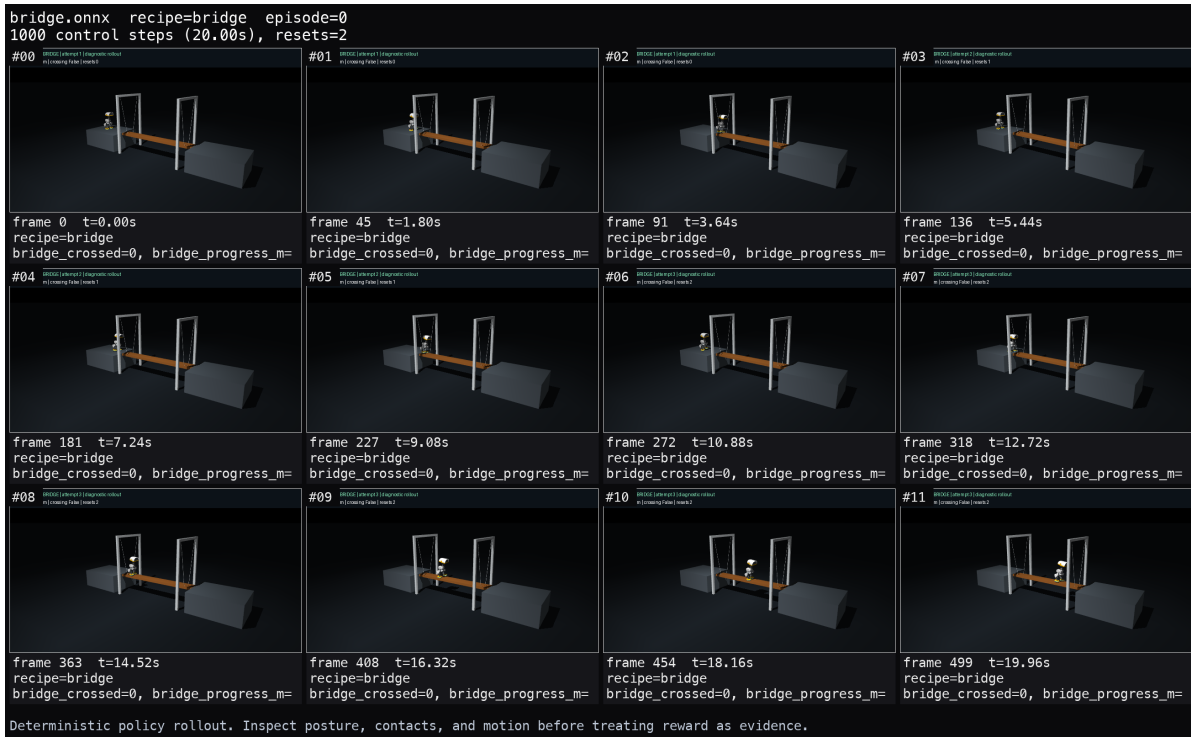


图 13: 最终独木桥 pilot 诊断序列。20 秒视频含 reset，不是一次不间断成功过桥。

7.5 在 Duck Viewer 运行两个场景

七案例是 Dance、Swing、Running、Stilts、Backflip、Basketball、Bridge。篮球在滚动未通过时应标明 balance-only；独木桥不能把试训视频变成成功过桥预览。

已有合适服务时优先使用，不要为了占一个端口停掉别人的服务。需要新服务时，每个长期进程放一个独立终端，使用空闲端口：

```
export MICRODUCK_STUDIO_PYTHON_DIRECT="$PWD/rlx/.venv-microduck/bin/python"
cd duck-viewer
node node_modules/next/dist/bin/next dev --hostname 127.0.0.1 -p 63317
```

第二个终端从工作区根目录执行：

```
cd microduck_local
MICRODUCK_ACTUATOR=bam .venv/bin/duck-lab --port 8788 \
  runs/first-gait ../microduck/policies/alpha_walking.onnx
```

第二条命令需要 harness 自己的虚拟环境和基线路径。新仓库若没有 runs/first-gait，省略该位置参数，只使用现成 walking ONNX。必要时在 microduck_local 内运行 uv sync --python 3.12 建立环境。每策略场景参数决定篮球或独木桥；默认执行器环境变量不能替代场景路由。

打开 <http://127.0.0.1:63317>，选择实验和导出策略，检查 live 场景。浏览器能打开还不够：保存的预览视频与实时物理场景是两种不同界面。

此前两个集成问题很有教学价值。其一，lab 提取 group-2 视觉几何，物理对象若只在 group 0，就会“存在但不可见”。其二，CLI 加载策略时必须传入该策略的自定义环境参数，否则名字叫 bridge 的策略也可能运行在错误场景。修复应是正确路由和视觉提取，而不是添加 CSS 装饰。LSTM 缓存也必须保证每只鸭子的 h/c 私有。

服务运行时可以使用 readiness 和 API 检查：

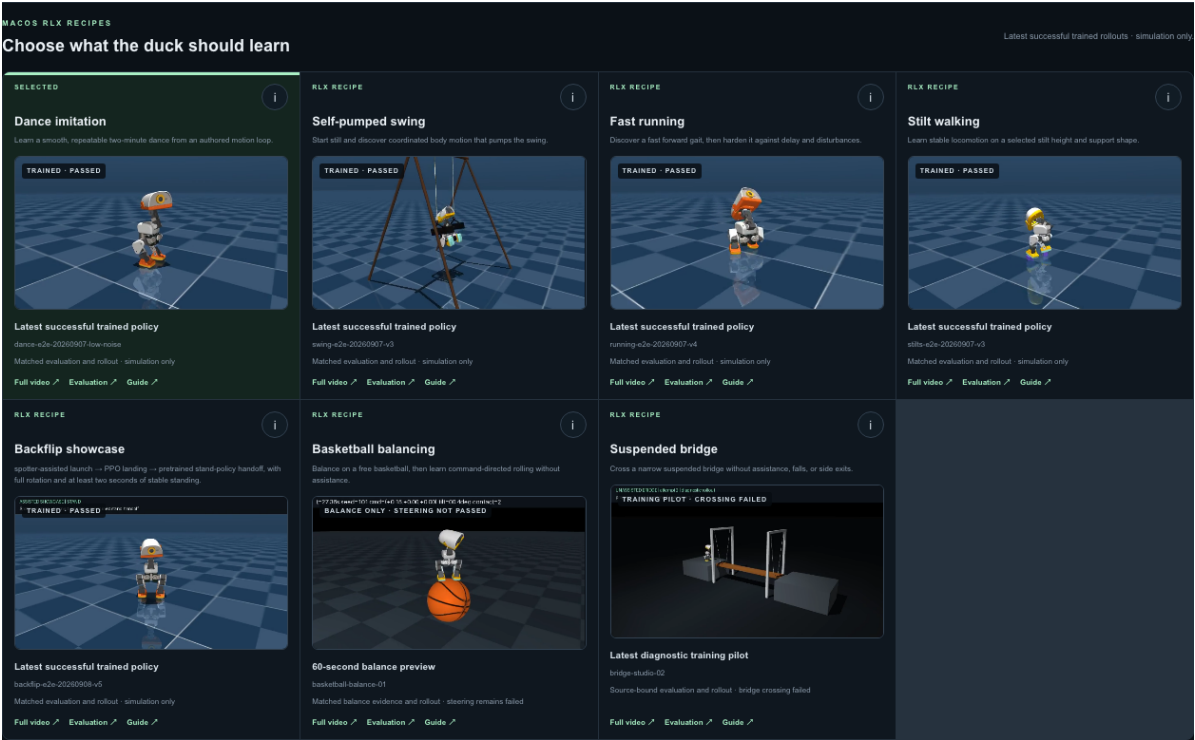


图 14: 实测七案例 Studio。卡片可用与技能验收是不同概念。



图 15: 真实 live 场景：左侧篮球，右侧独木桥、平台和支架。

```
bash restart-lab.sh --readiness-only
node scripts/verify-seven-cases.mjs
node scripts/verify-balance-studio-api.mjs
```

API 验证器调用真实评估 / 渲染并可能创建证据；可选 `--smoke-train` 会运行真实微型训练，不是只读检查。这些与静态教材构建不同，可能需要浏览器 / 图形权限和时间。

7.6 尚未完成技能的研究计划

先问：是否有任何 rollout 曾出现目标行为？如果完全没有，换奖励系数无法教会从未访问过的状态。独木桥应研究早期出生练习、木板稳定辅助、落脚位置、起点离台过渡；篮球则分别研究保留平衡、前进跟踪、侧向 / yaw 覆盖。

冻结验收规则，每次只改一种训练条件。记录每次尝试的辅助程度，评估时关闭辅助。比较源策略、零动作基线和新策略。站在起点平台的基线只证明站平台，不证明平衡于木板；中途出生的过桥只证明探索中出现行为，不证明从起点完整成功。

不要承诺“再训练多少步一定成功”。训练时间取决于探索、动力学、初始化和优化，而不仅是总步数。有价值的下一次实验也可能失败，却准确暴露课程必须解决的接触转换。

7.7 重建并检查教材

```
rlx/.venv-microduck/bin/python docs/basketball-bridge-book/tools/prepare_assets.py
rlx/.venv-microduck/bin/python docs/basketball-bridge-book/tools/build_book.py
rlx/.venv-microduck/bin/python docs/basketball-bridge-book/tools/validate_book.py
```

构建使用已有依赖，不安装软件，不重新训练策略。Markdown 是可复制源码版；PDF 增加代码换行、页码、公式和目录。`supporting-source.zip` 与 `source-manifest.json` 绑定实现快照。验证报告记录语法、源码 / 图像哈希、PDF 页数 / 文本检查和残留告警。

7.8 综合实践考核

1. 为什么 61 维观测不等于 61 个独立传感器？
2. 导出归一化在哪里执行？改分母为什么破坏 parity？
3. 算出一批 rollout 的 transition 数，区分仿真时间与现实用时。
4. 为什么 timeout 需要 value bootstrap，却不能让 GAE 跨 reset 传播？
5. 指出一张实测 loss、一张实测 reward、一张示意图。
6. 分开报告篮球平衡和滚动，并列出种子和时长。
7. 为什么独木桥最大进度不能证明过桥？
8. 不修改他人策略地启动两个 viewer 场景。
9. 哪些外部资产阻止从空文件夹离线复现？
10. 提出一个不改验收门槛的有界后续实验。

参考答案：观测含派生重力、指令和历史动作；归一化属于 actor 合同；总步数等于环境数乘每环境步数；超时不代表物理失败，但 reset 必须截断优势链；图表需来源；平衡通过而滚动失败；进度含进桥前接近距离；按策略路由场景且独立保留循环记忆；现成权重、网格和依赖仍为前提；改变一个课程或物理条件，保持确定性完整时长评估不变。

8 参考文献与来源

本地实现快照决定本书案例实际执行什么。算法论文和仿真器文档解释原理，但不证明两个策略学会技能。

1. John Schulman、Filip Wolski、Prafulla Dhariwal、Alec Radford、Oleg Klimov, **Proximal Policy Optimization Algorithms**, 2017, arXiv:1707.06347。PPO 原论文，解释裁剪 surrogate 和重复利用采样。本版核对官方记录：<https://arxiv.org/abs/1707.06347>。
2. John Schulman、Philipp Moritz、Sergey Levine、Michael Jordan、Pieter Abbeel, **High-Dimensional Continuous Control Using Generalized Advantage Estimation**, arXiv:1506.02438。GAE 参考；本书另外解释仓库中 termination / truncation 掩码。<https://arxiv.org/abs/1506.02438>。
3. **MuJoCo XML Reference**, 官方文档：body、free joint、geom、material、spatial tendon、springlength 与 limit。应对照所装版本；本快照是 MuJoCo 3.10.0。<https://mujoco.readthedocs.io/en/stable/XMLreference.html>。
4. `microduck_local/AGENTS.md`：固定观测合同、物理课程、导出—评估—观看的本地训练纪律。
5. `microduck-playground/experiments/basketball/`、`microduck-playground/artifacts/basketball/`：用户提供的设计和成熟策略，含来源与许可说明。
6. `microduck-playground/experiments/swing/README.md`：悬挂支撑参考；独木桥不是简单改名的 swing。
7. `docs/basketball-showcase/RESULTS.md`、`docs/bridge-showcase/RESULTS.md`：保留的本地结果及限制。
8. `assets/provenance.json`、`assets/SHA256SUMS`、`environment.json`、`source-manifest.json`：图像 / 数据、运行环境和代码的机器可读指纹。

本书没有整篇复制外部论文。公式、说明和代码讨论以本地实现为中心。复制的现有源码保留原文和相应许可说明；教材快照不改变第三方许可。

9 附录：完整核心源码与版本指纹

以下七个核心模块按本版快照完整收录；注释、导入与实现保持原样。代码源文件与 SHA-256 清单随书提供。其他适配器、完整 Studio CLI、篮球评估器、共享模型、viewer 接口和测试位于 `source/` 与 `supporting-source.zip`。快照不是独立软件包；运行需要完整工作区、机器人资产和所列依赖。复制代码请使用 Markdown 或 `.py` 文件，PDF 中的视觉换行不是额外语法。

9.1 A1. `rlx/rlx/environments/basketball.py`

SHA-256: 6bff54a8b1f7c856ec00cce59bb2b9b3c528d3c16054d1f730446fe68c65c757

```
"""CPU MuJoCo basketball prototype grounded in the playground b11 release."""

from __future__ import annotations

import math
from pathlib import Path
from types import MappingProxyType

import mujoco
import numpy as np

from microduck_local import contract as C
from microduck_local.walk_env import MicroduckWalkEnv

ROOT = Path(__file__).resolve().parents[3]
REFERENCE = ROOT / "microduck-playground"
BALL_RADIUS = 0.12
BALL_MASS = 0.62
CTRL_DT = C.CTRL_DT
HOLD_LEVELS = (1.0, 0.5, 0.25, 0.1, 0.03, 0.0)
REWARD_WEIGHTS = MappingProxyType({
    "linear_tracking": 1.0,
    "yaw_tracking": 0.5,
    "upright": 1.0,
    "centered": 2.0,
    "feet_on_ball": 1.0,
    "height": 1.0,
    "ball_speed_penalty": 0.05,
    "angular_velocity_penalty": 0.05,
    "action_rate_penalty": 0.2,
})

def basketball_model(actuator: str = "bam") -> mujoco.MjModel:
    if actuator not in ("bam", "xml"):
        raise ValueError("actuator must be bam or xml")
    robot_dir = Path(__file__).resolve().parents[1] / "mjlab_microduck/robot/microduck"
    asset_dir = REFERENCE / "src/mjlab_microduck/robot/assets/basketball"
    spec = mujoco.MjSpec.from_file(str(robot_dir / "robot_allcollisions.xml"))
    spec.option.timestep = C.PHYSICS_DT
    spec.option.integrator = mujoco.mjtIntegrator.mjINT_IMPLICITFAST
    spec.option.cone = mujoco.mjtCone.mjCONE_PYRAMIDAL
    spec.option.iterations = 10
    spec.option.ls_iterations = 20
    ground_texture = spec.add_texture(name="basketball_ground")
    ground_texture.type = mujoco.mjtTexture.mjTEXTURE_2D
    ground_texture.builtin = mujoco.mjtBuiltin.mjBUILTIN_CHECKER
    ground_texture.rgb1 = [.22, .27, .31]
    ground_texture.rgb2 = [.16, .20, .24]
    ground_texture.width = 256
    ground_texture.height = 256
    ground_material = spec.add_material(name="basketball_ground_mat")
    ground_textures = list(ground_material.textures)
    ground_textures[int(mujoco.mjtTextureRole.mjTEXTROLE_RGB)] = "basketball_ground"
    ground_material.textures = ground_textures
```

```

ground_material.texuniform = True
ground_material.texrepeat = [4, 4]
spec.worldbody.add_geom(
    name="floor", type=mujoco.mjtGeom.mjGEOM_PLANE,
    size=[0, 0, 0.05], material="basketball_ground_mat",
    friction=[1, 0.005, 0.0001],
)
spec.worldbody.add_light(pos=[0, -1, 3], dir=[0, 0, -1])
texture = spec.add_texture(name="basketball_tex")
texture.type = mujoco.mjtTexture.mjTEXTURE_2D
texture.file = str(asset_dir / "basketball.png")
material = spec.add_material(name="basketball_mat")
textures = list(material.textures)
textures[int(mujoco.mjtTextureRole.mjTEXTROLE_RGB)] = "basketball_tex"
material.textures = textures
material.specular = 0.15
spec.add_mesh(name="basketball_mesh", file=str(asset_dir / "basketball.obj"))
ball = spec.worldbody.add_body(name="basketball", pos=[0, 0, BALL_RADIUS + 0.001])
ball.add_freejoint(name="basketball_freejoint")
ball.add_geom(
    name="ball_sphere", type=mujoco.mjtGeom.mjGEOM_SPHERE,
    size=[BALL_RADIUS, 0, 0], mass=BALL_MASS, friction=[1.2, 0.01, 0.001],
    priority=1, condim=4, rgba=[1, 0.45, 0.02, 0], group=3,
)
ball.add_geom(
    name="ball_visual", type=mujoco.mjtGeom.mjGEOM_MESH,
    meshname="basketball_mesh", material="basketball_mat",
    mass=0, contype=0, conaffinity=0, group=2,
)
spec.add_key(
    name="STAND", qpos=np.concatenate((
        [0, 0, 2 * BALL_RADIUS + 0.128, 1, 0, 0, 0],
        C.DEFAULT_POSE, [0, 0, BALL_RADIUS + 0.001, 1, 0, 0, 0],
    )), ctrl=C.DEFAULT_POSE,
)
model = spec.compile()
model.vis.global_.offwidth = 1280
model.vis.global_.offheight = 960
return model

def curriculum_level(stage: int, seconds: float) -> int:
    if not 0 <= stage < len(HOLD_LEVELS) or not math.isfinite(seconds) or seconds < 0:
        raise ValueError("invalid curriculum stage or episode duration")
    if seconds >= 6:
        return min(stage + 1, len(HOLD_LEVELS) - 1)
    if seconds < 1.5:
        return max(stage - 1, 0)
    return stage

class BasketballEnv(MicroduckWalkEnv):
    """A blind 61D actor steps on a sphere; all assistance is explicit in info.

    This is a local actuator/physics adaptation, not a bitwise mjlab port.
    The sealed bounded reward subset is unchanged across curriculum stages.
    """

    def __init__(
        self, *, max_episode_s: float = 10, hold: float = 0,
        curriculum: bool = False, command: tuple[float, float, float] | None = None,
        seed: int | None = None, actuator: str = "bam", obs_noise: bool = False,
        domain_rand: bool = False, pushes: bool = False, random_yaw: bool = True,
        model: mujoco.MjModel | None = None,
    ):
        if not math.isfinite(hold) or not 0 <= hold <= 1:
            raise ValueError("hold must be finite and between zero and one")
        if not math.isfinite(max_episode_s) or max_episode_s <= 0:
            raise ValueError("episode length must be positive and finite")
        if curriculum and hold not in HOLD_LEVELS:
            raise ValueError("curriculum hold must be one of HOLD_LEVELS")

```

```

self.hold = float(hold)
self.curriculum = curriculum
self.stage = HOLD_LEVELS.index(hold) if hold in HOLD_LEVELS else 0
self.fixed_command = self._validate_command(command)
self.pushes = pushes
self._episode_finished = False
super().__init__(
    max_episode_s=max_episode_s, command_resample_s=5,
    obs_noise=obs_noise, domain_rand=domain_rand, action_delay=True,
    random_yaw=random_yaw, seed=seed, actuator_force=actuator,
    model=model if model is not None else basketball_model(actuator),
    terminate_on_fall=False, height_termination=False,
)
self.ball_body_id = self.model.body("basketball").id
self.ball_geom_id = self.model.geom("ball_sphere").id
self.ball_qpos_adr = int(self.model.joint("basketball_freejoint").qposadr[0])
self.ball_qvel_adr = int(self.model.joint("basketball_freejoint").dofadr[0])
self.foot_site_ids = [self.model.site(name).id for name in ("left_foot", "right_foot")]
self._ball_anchor = np.array([0, 0, BALL_RADIUS + 0.001])
self._next_push = math.inf

@staticmethod
def _validate_command(command):
    if command is None:
        return None
    array = np.asarray(command, dtype=np.float32)
    if array.shape != (3,) or not np.isfinite(array).all():
        raise ValueError("command must contain three finite numbers")
    return array.copy()

def set_command(self, command: tuple[float, float, float]) -> None:
    validated = self._validate_command(command)
    if validated is None:
        raise ValueError("set_command requires three numbers")
    self.fixed_command = validated
    self.twist_cmd[:] = validated

def _sample_commands(self) -> None:
    if self.fixed_command is not None:
        self.twist_cmd[:] = self.fixed_command
    elif self._rng.random() < 0.25:
        self.twist_cmd[:] = 0
    else:
        self.twist_cmd[:] = self._rng.uniform([-0.15, -0.1, -0.5], [0.15, 0.1, 0.5])
    self.head_cmd[:] = 0
    self.body_cmd[:] = 0

def reset(self, *, seed: int | None = None, options: dict | None = None):
    if self.curriculum and self._episode_finished:
        self.stage = curriculum_level(self.stage, self.step_count * CTRL_DT)
        self.hold = HOLD_LEVELS[self.stage]
    self._episode_finished = False
    super().reset(seed=seed, options=options)
    self.data.qpos[2] = self._rng.uniform(-0.01, 0.01, 2)
    self.data.qpos[2] = 2 * BALL_RADIUS + 0.128
    yaw = self._rng.uniform(-math.pi, math.pi) if self.random_yaw else 0
    roll, pitch = self._rng.uniform(-math.radians(2), math.radians(2), 2)
    yaw_quat = np.array([math.cos(yaw / 2), 0, 0, math.sin(yaw / 2)])
    pitch_quat = np.array([math.cos(pitch / 2), 0, math.sin(pitch / 2), 0])
    roll_quat = np.array([math.cos(roll / 2), math.sin(roll / 2), 0, 0])
    rotation = np.empty(4)
    mujoco.mju_mulQuat(rotation, yaw_quat, pitch_quat)
    mujoco.mju_mulQuat(self.data.qpos[3:7], rotation, roll_quat)
    self.data.qpos[self.joint_qpos_adr] = C.DEFAULT_POSE + self._rng.uniform(-0.05, 0.05, 14)
    self.data.qpos[self.ball_qpos_adr:self.ball_qpos_adr + 7] = [0, 0, BALL_RADIUS + 0.001,
        ↪ 1, 0, 0, 0]
    self.data.qvel[:] = 0
    self.data.xfrc_applied[:] = 0
    self.data.ctrl[:] = self.data.qpos[self.joint_qpos_adr]
    mujoco.mj_forward(self.model, self.data)

```

```

    if self.bam is not None:
        self.bam.reset()
    self.prev_joint_vel[:] = 0
    self._next_push = self._rng.uniform(1.5, 3) if self.pushes else math.inf
    return self._get_obs(), self.metrics()

def _apply_assistance(self) -> None:
    self.data.xfrc_applied[:] = 0
    if self.hold == 0:
        return
    ball_pos = self.data.xpos[self.ball_body_id]
    ball_vel = self.data.qvel[self.ball_qvel_adr:self.ball_qvel_adr + 6]
    ball_force = -400 * (ball_pos - self._ball_anchor) - 20 * ball_vel[:3]
    ball_force[2] = -20 * ball_vel[2]
    self.data.xfrc_applied[self.ball_body_id, :3] = self.hold * ball_force
    ball_rotation = self.data.xmat[self.ball_body_id].reshape(3, 3)
    self.data.xfrc_applied[self.ball_body_id, 3:] = -self.hold * .2 * (ball_rotation @
    ↪ ball_vel[:3])
    rotation = self.data.xmat[self.trunk_body_id].reshape(3, 3)
    up = rotation[:, 2]
    angular_world = rotation @ self.data.qvel[3:6]
    duck_force = -40 * (self.data.xpos[self.trunk_body_id] - ball_pos) - 4 *
    ↪ self.data.qvel[:3]
    duck_force[2] = 0
    self.data.xfrc_applied[self.trunk_body_id, :3] = self.hold * duck_force
    self.data.xfrc_applied[self.trunk_body_id, 3:] = self.hold * (
        3 * np.cross(up, [0, 0, 1]) - .08 * angular_world
    )

def step(self, action):
    action = np.asarray(action, dtype=np.float32)
    if action.shape != (14,) or not np.isfinite(action).all():
        raise ValueError("actions must be finite with shape (14,)")
    if self._episode_finished:
        raise RuntimeError("reset is required after a terminal episode")
    self._apply_assistance()
    if self.step_count * CTRL_DT >= self._next_push:
        self.data.qvel[:2] = self._rng.uniform(-.09, .09, 2)
        self._next_push += self._rng.uniform(1.5, 3)
    observation, reward, terminated, truncated, info = super().step(action)
    metrics = self.metrics()
    reasons = []
    if metrics["root_above_ball_m"] < BALL_RADIUS + .05:
        reasons.append("height")
    if metrics["root_ball_offset_m"] > .16:
        reasons.append("offset")
    if metrics["tilt_deg"] > 55:
        reasons.append("tilt")
    if metrics["robot_floor_contact"]:
        reasons.append("robot_floor_contact")
    if metrics["body_ball_contact"]:
        reasons.append("body_ball_contact")
    if not np.isfinite(self.data.qpos).all() or not np.isfinite(self.data.qvel).all():
        reasons.append("nonfinite_physics")
    terminated = terminated or bool(reasons)
    self._episode_finished = terminated or truncated
    info.update(metrics)
    info["termination_reasons"] = reasons
    if self._episode_finished:
        info["episode_rewards"] = dict(self.reward_sums)
    return observation, reward, terminated, truncated, info

def _compute_reward(self):
    relative = self.data.xpos[self.trunk_body_id] - self.data.xpos[self.ball_body_id]
    gravity = self._projected_gravity()
    feet = self.data.site_xpos[self.foot_site_ids]
    gaps = np.linalg.norm(feet - self.data.xpos[self.ball_body_id], axis=1) - (BALL_RADIUS
    ↪ + .012)
    ball_velocity = self.data.qvel[self.ball_qvel_adr:self.ball_qvel_adr + 2]

```



```

terms = {
    "linear_tracking": math.exp(-float(np.sum((self.body_lin_vel()[2] -
        ↪ self.twist_cmd[2]) ** 2)) / .1),
    "yaw_tracking": math.exp(-float((self._gyro[2] - self.twist_cmd[2]) ** 2) / .5),
    "upright": math.exp(-float(np.sum(gravity[:2] ** 2)) / .09),
    "centered": math.exp(-float(np.sum(relative[:2] ** 2)) / .0016),
    "feet_on_ball": float(np.mean(np.exp(-gaps ** 2 / .0004))),
    "height": math.exp(-float((relative[2] - BALL_RADIUS - .125) ** 2) / .0009),
    "ball_speed_penalty": -float(min(np.sum(ball_velocity ** 2), 100)),
    "angular_velocity_penalty": -float(min(np.sum(self._gyro[:2] ** 2), 100)),
    "action_rate_penalty": -float(min(np.sum((self.last_action - self.prev_action) **
        ↪ 2), 100)),
}
weighted = {key: value * REWARD_WEIGHTS[key] * CTRL_DT for key, value in terms.items()}
reward = sum(weighted.values())
if not math.isfinite(reward):
    raise FloatingPointError("nonfinite basketball reward")
return reward, weighted

def metrics(self) -> dict:
    ball_pos = self.data.xpos[self.ball_body_id]
    relative = self.data.xpos[self.trunk_body_id] - ball_pos
    foot_geoms = set(self.foot_geoms.values())
    touching = set()
    floor_contact = False
    body_ball_contact = False
    for contact in self.data.contact:
        pair = {int(contact.geom1), int(contact.geom2)}
        if self.ball_geom_id in pair:
            other = next(iter(pair - {self.ball_geom_id}), self.ball_geom_id)
            if other in foot_geoms:
                touching.add(other)
            elif self.model.geom_bodyid[other] not in (0, self.ball_body_id):
                body_ball_contact = True
        if self.floor_geom in pair:
            other = next(iter(pair - {self.floor_geom}), self.floor_geom)
            if self.model.geom_bodyid[other] not in (0, self.ball_body_id):
                floor_contact = True
    ball_velocity = self.data.qvel[self.ball_qvel_adr:self.ball_qvel_adr + 6]
    return {
        "tilt_deg": math.degrees(math.acos(float(np.clip(-self._projected_gravity()[2], -1,
            ↪ 1)))),
        "root_ball_offset_m": float(np.linalg.norm(relative[:2])),
        "root_above_ball_m": float(relative[2]),
        "ball_speed_mps": float(np.linalg.norm(ball_velocity[:2])),
        "ball_position": ball_pos.tolist(),
        "ball_angular_speed_rad_s": float(np.linalg.norm(ball_velocity[3:])),
        "body_forward_mps": float(self.body_lin_vel()[0]),
        "body_lateral_mps": float(self.body_lin_vel()[1]),
        "body_yaw_rate_rad_s": float(self._gyro[2]),
        "foot_ball_contacts": len(touching),
        "body_ball_contact": body_ball_contact,
        "robot_floor_contact": floor_contact,
        "hold": self.hold,
        "elapsed_s": self.step_count * CTRL_DT,
    }

```

9.2 A2. rlx/rlx/models/basketball.py

SHA-256: df79b783e643ed40ea19fe583d05dec7a5a116cf74cced0e48776149f8ea4e06

"""Torch recurrent policy support for the released blind basketball actor."""

from __future__ import annotations

import hashlib


```

from pathlib import Path
from typing import Any

import numpy as np
import torch
from torch import nn

OBSERVATION_DIM = 61
ACTION_DIM = 14
HIDDEN_SIZE = 256
HIDDEN_DIMS = (512, 256, 128)
NORMALIZER_EPSILON = 0.01
SOURCE_CHECKPOINT_SHA256 = (
    "57a322eff092cb71e7cba831791232f0cc4fd45ede0441cad6d87813b0033e41"
)
SOURCE_ONNX_SHA256 = (
    "e105148b160b3a86170621648215be38dd5018e955930ce182394f4513c7569b"
)
LOCAL_CHECKPOINT_FORMAT = "rlx.basketball.local_recurrent_ppo.v1"

class ObservationNormalizer(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.register_buffer("_mean", torch.zeros(1, OBSERVATION_DIM))
        self.register_buffer("_var", torch.ones(1, OBSERVATION_DIM))
        self.register_buffer("_std", torch.ones(1, OBSERVATION_DIM))
        self.register_buffer("count", torch.tensor(0.0))

    def forward(self, observations: torch.Tensor) -> torch.Tensor:
        return (observations - self._mean) / (self._std + NORMALIZER_EPSILON)

class RecurrentCore(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.rnn = nn.LSTM(OBSERVATION_DIM, HIDDEN_SIZE, num_layers=1)

class GaussianDistribution(nn.Module):
    def __init__(self, initial_std: float) -> None:
        super().__init__()
        if not np.isfinite(initial_std) or initial_std <= 0.0:
            raise ValueError("initial_std must be finite and positive")
        self.std_param = nn.Parameter(torch.full((ACTION_DIM,), float(initial_std)))

class BasketballActor(nn.Module):
    def __init__(self, *, initial_std: float = 0.03) -> None:
        super().__init__()
        self.obs_normalizer = ObservationNormalizer()
        self.distribution = GaussianDistribution(initial_std)
        self.rnn = RecurrentCore()
        layers: list[nn.Module] = []
        previous = HIDDEN_SIZE
        for width in HIDDEN_DIMS:
            layers.extend((nn.Linear(previous, width), nn.ELU()))
            previous = width
        layers.append(nn.Linear(previous, ACTION_DIM))
        self.mlp = nn.Sequential(*layers)

    def initial_state(
        self,
        batch_size: int,
        *,
        device: torch.device | str | None = None,
        dtype: torch.dtype | None = None,
    ) -> tuple[torch.Tensor, torch.Tensor]:
        if batch_size < 1:
            raise ValueError("batch_size must be positive")
        parameter = next(self.parameters())

```

```

        target_device = parameter.device if device is None else device
        target_dtype = parameter.dtype if dtype is None else dtype
        shape = (1, batch_size, HIDDEN_SIZE)
        return (
            torch.zeros(shape, device=target_device, dtype=target_dtype),
            torch.zeros(shape, device=target_device, dtype=target_dtype),
        )

    def forward(
        self,
        observations: torch.Tensor,
        h_in: torch.Tensor,
        c_in: torch.Tensor,
    ) -> tuple[torch.Tensor, torch.Tensor, torch.Tensor]:
        normalized = self.obs_normalizer(observations)
        recurrent, (h_out, c_out) = self.rnn.rnn(
            normalized.unsqueeze(0),
            (h_in, c_in),
        )
        actions = self.mlp(recurrent.squeeze(0))
        return actions, h_out, c_out

    def forward_sequence(
        self,
        observations: torch.Tensor,
        h_in: torch.Tensor,
        c_in: torch.Tensor,
        episode_starts: torch.Tensor,
    ) -> tuple[torch.Tensor, torch.Tensor, torch.Tensor]:
        if observations.ndim != 3 or observations.shape[-1] != OBSERVATION_DIM:
            raise ValueError("observations must have shape [time, batch, 61]")
        if episode_starts.shape != observations.shape[:2]:
            raise ValueError("episode_starts must have shape [time, batch]")
        hidden = h_in
        cell = c_in
        outputs: list[torch.Tensor] = []
        for step in range(observations.shape[0]):
            carry = (~episode_starts[step].bool()).to(observations.dtype)
            carry = carry.view(1, -1, 1)
            hidden = hidden * carry
            cell = cell * carry
            actions, hidden, cell = self(observations[step], hidden, cell)
            outputs.append(actions)
        return torch.stack(outputs), hidden, cell

    def action_std(self) -> torch.Tensor:
        return self.distribution.std_param.clamp_min(1e-6)

class BasketballCritic(nn.Module):
    def __init__(self, normalizer: ObservationNormalizer) -> None:
        super().__init__()
        self.normalizer = normalizer
        layers: list[nn.Module] = []
        previous = OBSERVATION_DIM
        for width in HIDDEN_DIMS:
            layers.extend((nn.Linear(previous, width), nn.ELU()))
            previous = width
        layers.append(nn.Linear(previous, 1))
        self.mlp = nn.Sequential(*layers)

    def forward(self, observations: torch.Tensor) -> torch.Tensor:
        with torch.no_grad():
            normalized = self.normalizer(observations)
        return self.mlp(normalized).squeeze(-1)

def sha256_file(path: str | Path) -> str:
    digest = hashlib.sha256()
    with Path(path).open("rb") as stream:
        for block in iter(lambda: stream.read(1024 * 1024), b''):

```

```

        digest.update(block)
    return digest.hexdigest()

def verify_source_artifacts(
    checkpoint: str | Path,
    onnx_path: str | Path | None = None,
) -> dict[str, str]:
    checkpoint_path = Path(checkpoint)
    checkpoint_hash = sha256_file(checkpoint_path)
    if checkpoint_hash != SOURCE_CHECKPOINT_SHA256:
        raise ValueError(
            f"unexpected basketball checkpoint SHA256: {checkpoint_hash}"
        )
    result = {"checkpoint_sha256": checkpoint_hash}
    candidate = Path(onnx_path) if onnx_path is not None else checkpoint_path.with_name(
        "policy.onnx"
    )
    if not candidate.is_file():
        raise FileNotFoundError(f"required source ONNX is missing: {candidate}")
    onnx_hash = sha256_file(candidate)
    if onnx_hash != SOURCE_ONNX_SHA256:
        raise ValueError(f"unexpected basketball ONNX SHA256: {onnx_hash}")
    result["onnx_sha256"] = onnx_hash
    return result

def load_source_actor(
    checkpoint: str | Path,
    *,
    initial_std: float = 0.03,
    verify_hash: bool = True,
    allow_local: bool = False,
) -> tuple[BasketballActor, dict[str, Any]]:
    checkpoint_path = Path(checkpoint)
    local_candidate = (
        allow_local and verify_hash
        and sha256_file(checkpoint_path) != SOURCE_CHECKPOINT_SHA256
    )
    if local_candidate:
        policy_path = checkpoint_path.with_name("policy.onnx")
        hashes = {
            "checkpoint_sha256": sha256_file(checkpoint_path),
            "onnx_sha256": sha256_file(policy_path),
        }
    else:
        hashes = verify_source_artifacts(checkpoint_path) if verify_hash else {}
    payload = torch.load(checkpoint_path, map_location="cpu", weights_only=True)
    if not isinstance(payload, dict) or "actor_state_dict" not in payload:
        raise ValueError("basketball checkpoint is missing actor_state_dict")
    source_steps = 0
    if local_candidate:
        if payload.get("format") != LOCAL_CHECKPOINT_FORMAT:
            raise ValueError("unsupported local basketball checkpoint format")
        if (
            payload.get("normalization_frozen") is not True
            or payload.get("actor_observation_dim") != OBSERVATION_DIM
            or payload.get("critic_observation_dim") != OBSERVATION_DIM
        ):
            raise ValueError("invalid local basketball normalization/observation contract")
        source_steps = payload.get("total_steps", payload.get("local_steps"))
        if type(source_steps) is not int or source_steps < 0:
            raise ValueError("local basketball checkpoint requires nonnegative training steps")
    actor = BasketballActor(initial_std=initial_std)
    actor.load_state_dict(payload["actor_state_dict"], strict=True)
    if any(not torch.isfinite(value).all() for value in actor.state_dict().values()):
        raise ValueError("basketball actor contains non-finite parameters")
    parity = None
    if local_candidate:
        parity = compare_actor_to_onnx(actor, policy_path)

```

```

        error = max(float(parity["max_abs_error"]), float(parity["max_state_abs_error"]))
        if not np.isfinite(error) or error > 3e-5:
            raise ValueError(f"local basketball checkpoint/ONNX parity failed: {error}")
        if hashes != {
            "checkpoint_sha256": sha256_file(checkpoint_path),
            "onnx_sha256": sha256_file(policy_path),
        }:
            raise RuntimeError("local basketball source changed during validation")
    with torch.no_grad():
        actor.distribution.std_param.fill_(float(initial_std))
    actor.obs_normalizer.requires_grad_(False)
    metadata = {
        "source_iteration": int(payload.get("iter", -1)),
        "source_hashes": hashes,
        "source_actor_keys": sorted(payload["actor_state_dict"]),
        "source_kind": "local_actor_warm_start" if local_candidate else "released_actor",
        "source_training_steps": source_steps,
        "source_onnx_parity": parity,
    }
    return actor, metadata

def export_recurrent_onnx(
    actor: BasketballActor,
    output: str | Path,
) -> Path:
    target = Path(output)
    target.parent.mkdir(parents=True, exist_ok=True)
    actor = actor.cpu().eval()
    h_in, c_in = actor.initial_state(1, device="cpu", dtype=torch.float32)
    torch.onnx.export(
        actor,
        (torch.zeros(1, OBSERVATION_DIM), h_in, c_in),
        target,
        input_names=["obs", "h_in", "c_in"],
        output_names=["actions", "h_out", "c_out"],
        opset_version=17,
        do_constant_folding=True,
        dynamo=False,
    )
    return target

def compare_actor_to_onnx(
    actor: BasketballActor,
    onnx_path: str | Path,
    *,
    steps: int = 40,
    reset_at: int = 20,
    seed: int = 42,
) -> dict[str, float | int]:
    import onnxruntime as ort

    if steps < 1 or not 0 <= reset_at < steps:
        raise ValueError("reset_at must identify a step in the comparison")
    generator = torch.Generator().manual_seed(seed)
    options = ort.SessionOptions()
    options.intra_op_num_threads = 1
    options.inter_op_num_threads = 1
    session = ort.InferenceSession(
        str(onnx_path),
        sess_options=options,
        providers=["CPUExecutionProvider"],
    )
    if [item.name for item in session.get_inputs()] != ["obs", "h_in", "c_in"]:
        raise ValueError("unexpected recurrent ONNX input contract")
    if [item.name for item in session.get_outputs()] != [
        "actions",
        "h_out",
        "c_out",
    ]:

```

```

        raise ValueError("unexpected recurrent ONNX output contract")
    actor = actor.cpu().eval()
    hidden, cell = actor.initial_state(1, device="cpu", dtype=torch.float32)
    onnx_hidden = hidden.numpy()
    onnx_cell = cell.numpy()
    max_error = 0.0
    max_state_error = 0.0
    with torch.inference_mode():
        for step in range(steps):
            if step == reset_at:
                hidden.zero_()
                cell.zero_()
                onnx_hidden.fill(0.0)
                onnx_cell.fill(0.0)
            observation = torch.randn(
                1,
                OBSERVATION_DIM,
                generator=generator,
            ) * 0.2
            observation[:, -10:] = 0.0
            expected, hidden, cell = actor(observation, hidden, cell)
            actual, onnx_hidden, onnx_cell = session.run(
                None,
                {
                    "obs": observation.numpy(),
                    "h_in": onnx_hidden,
                    "c_in": onnx_cell,
                },
            )
            for index, (torch_value, onnx_value) in enumerate((
                (expected, actual), (hidden, onnx_hidden), (cell, onnx_cell),
            )):
                expected_array = torch_value.numpy()
                if not np.isfinite(expected_array).all() or not np.isfinite(onnx_value).all():
                    raise ValueError("non-finite recurrent ONNX parity output")
                error = float(np.max(np.abs(onnx_value - expected_array)))
                if index == 0:
                    max_error = max(max_error, error)
                else:
                    max_state_error = max(max_state_error, error)
    return {
        "steps": steps, "reset_at": reset_at, "max_abs_error": max_error,
        "max_state_abs_error": max_state_error,
    }

```

9.3 A3. rlx/examples/ppo_microduck_basketball.py

SHA-256: d465af0f34751d61ed6b8ea233178f9b70472ab839c5724faef811e4
206b9b71

"""Continue the released recurrent basketball actor with local CPU PP0."""

```

from __future__ import annotations

import argparse
from dataclasses import dataclass
from datetime import datetime, timezone
import json
import math
from pathlib import Path
from typing import Any, Sequence

import numpy as np
import torch
from torch import nn

from rlx.models.basketball import (
    LOCAL_CHECKPOINT_FORMAT,

```

```
OBSERVATION_DIM,
BasketballActor,
BasketballCritic,
compare_actor_to_onnx,
export_recurrent_onnx,
load_source_actor,
sha256_file,
)

CTRL_DT = 0.02
DEFAULT_CHECKPOINT = (
    Path(__file__).resolve().parents[2]
    / "microduck-playground"
    / "artifacts"
    / "basketball"
    / "checkpoint.pt"
)

@dataclass
class Rollout:
    observations: torch.Tensor
    actions: torch.Tensor
    old_log_probs: torch.Tensor
    old_values: torch.Tensor
    rewards: torch.Tensor
    terminated: torch.Tensor
    truncated: torch.Tensor
    bootstrap_values: torch.Tensor
    episode_starts: torch.Tensor
    initial_hidden: torch.Tensor
    initial_cell: torch.Tensor
    next_hidden: torch.Tensor
    next_cell: torch.Tensor
    next_episode_starts: torch.Tensor
    infos: list[list[dict[str, Any]]]

def gaussian_log_prob(
    actions: torch.Tensor,
    means: torch.Tensor,
    std: torch.Tensor,
) -> torch.Tensor:
    variance = std.square()
    return (
        -0.5
        * (
            (actions - means).square() / variance
            + 2.0 * torch.log(std)
            + math.log(2.0 * math.pi)
        )
    ).sum(dim=-1)

def gaussian_entropy(std: torch.Tensor) -> torch.Tensor:
    return (torch.log(std) + 0.5 * math.log(2.0 * math.pi * math.e)).sum()

def compute_gae(
    rewards: torch.Tensor,
    values: torch.Tensor,
    bootstrap_values: torch.Tensor,
    terminated: torch.Tensor,
    truncated: torch.Tensor,
    *,
    gamma: float = 0.99,
    gae_lambda: float = 0.95,
) -> tuple[torch.Tensor, torch.Tensor]:
    if not (
        rewards.shape
        == values.shape
        == bootstrap_values.shape
    ):
```

```

        == terminated.shape
        == truncated.shape
    ):
        raise ValueError("GAE tensors must share [time, env] shape")
    advantages = torch.zeros_like(rewards)
    trace = torch.zeros_like(rewards[0])
    for step in range(rewards.shape[0] - 1, -1, -1):
        bootstrap_mask = (~terminated[step]).to(rewards.dtype)
        continuation_mask = (~(terminated[step] | truncated[step])).to(
            rewards.dtype
        )
        delta = (
            rewards[step]
            + gamma * bootstrap_values[step] * bootstrap_mask
            - values[step]
        )
        trace = delta + gamma * gae_lambda * continuation_mask * trace
        advantages[step] = trace
    return advantages, advantages + values

def _validate_observations(observations: Any, num_envs: int) -> np.ndarray:
    array = np.asarray(observations, dtype=np.float32)
    expected = (num_envs, OBSERVATION_DIM)
    if array.shape != expected:
        raise ValueError(f"observations must have shape {expected}, got {array.shape}")
    if not np.isfinite(array).all():
        raise RuntimeError("basketball observations contain non-finite values")
    if np.count_nonzero(array[:, 51:61]):
        raise RuntimeError("basketball actor head/body padding must remain zero")
    return array

def _validate_step(
    observation: Any,
    reward: Any,
    terminated: Any,
    truncated: Any,
    info: Any,
) -> tuple[np.ndarray, float, bool, bool, dict[str, Any]]:
    obs = np.asarray(observation, dtype=np.float32)
    if obs.shape != (OBSERVATION_DIM,) or not np.isfinite(obs).all():
        raise RuntimeError("basketball step returned an invalid observation")
    if np.count_nonzero(obs[51:61]):
        raise RuntimeError("basketball actor received nonzero head/body padding")
    value = float(reward)
    if not math.isfinite(value):
        raise RuntimeError("basketball step returned a non-finite reward")
    if not isinstance(info, dict):
        raise TypeError("basketball info must be a dictionary")
    return obs, value, bool(terminated), bool(truncated), info

def make_environments(args: argparse.Namespace) -> list[Any]:
    from rlx.environments.basketball import BasketballEnv, basketball_model

    shared_model = (
        basketball_model(actuator=args.actuator) if args.actuator == "xml" else None
    )
    command = None if args.command is None else tuple(float(x) for x in args.command)
    environments = []
    for index in range(args.num_envs):
        kwargs = {
            "max_episode_s": 10.0,
            "hold": args.hold,
            "curriculum": args.curriculum,
            "command": command,
            "seed": args.seed + index,
            "actuator": args.actuator,
            "obs_noise": False,
            "domain_rand": False,

```

```

        "pushes": args.pushes,
    }
    if shared_model is not None:
        kwargs["model"] = shared_model
    environments.append(
        BasketballEnv(**kwargs)
    )
    return environments

def reset_environments(
    environments: Sequence[Any],
    *,
    seed: int,
) -> tuple[np.ndarray, list[dict[str, Any]]]:
    observations: list[np.ndarray] = []
    infos: list[dict[str, Any]] = []
    for index, environment in enumerate(environments):
        observation, info = environment.reset(seed=seed + index)
        observations.append(np.asarray(observation, dtype=np.float32))
        if not isinstance(info, dict):
            raise TypeError("basketball reset info must be a dictionary")
        infos.append(info)
    return _validate_observations(observations, len(environments)), infos

def collect_rollout(
    actor: BasketballActor,
    critic: BasketballCritic,
    environments: Sequence[Any],
    observations: np.ndarray,
    hidden: torch.Tensor,
    cell: torch.Tensor,
    episode_starts: torch.Tensor,
    *,
    steps: int,
    generator: torch.Generator,
    seed: int,
    reset_counts: np.ndarray,
) -> tuple[Rollout, np.ndarray, torch.Tensor, torch.Tensor, torch.Tensor]:
    if steps < 1:
        raise ValueError("steps must be positive")
    num_envs = len(environments)
    initial_hidden = hidden.detach().clone()
    initial_cell = cell.detach().clone()
    stored: dict[str, list[torch.Tensor]] = {
        name: []
        for name in (
            "observations",
            "actions",
            "old_log_probs",
            "old_values",
            "rewards",
            "terminated",
            "truncated",
            "bootstrap_values",
            "episode_starts",
        )
    }
    all_infos: list[list[dict[str, Any]]] = []
    actor.eval()
    critic.eval()
    for _ in range(steps):
        observation_tensor = torch.from_numpy(observations)
        with torch.inference_mode():
            carry = (~episode_starts.bool()).to(observation_tensor.dtype).view(
                1, num_envs, 1
            )
            hidden = hidden * carry
            cell = cell * carry
            means, next_hidden, next_cell = actor(

```



```

        observation_tensor,
        hidden,
        cell,
    )
    std = actor.action_std()
    actions = means + torch.randn(
        means.shape,
        generator=generator,
        dtype=means.dtype,
    ) * std
    log_probs = gaussian_log_prob(actions, means, std)
    values = critic(observation_tensor)
    next_observations: list[np.ndarray] = []
    rewards: list[float] = []
    terminations: list[bool] = []
    truncations: list[bool] = []
    step_infos: list[dict[str, Any]] = []
    final_observations: list[np.ndarray] = []
    for index, environment in enumerate(environments):
        result = _validate_step(*environment.step(actions[index].numpy()))
        next_observation, reward, terminated, truncated, info = result
        final_observations.append(next_observation)
        rewards.append(reward)
        terminations.append(terminated)
        truncations.append(truncated)
        step_infos.append(info)
        if terminated or truncated:
            reset_counts[index] += 1
            reset_observation, reset_info = environment.reset(
                seed=seed + index + int(reset_counts[index]) * num_envs
            )
            reset_observation = np.asarray(reset_observation, dtype=np.float32)
            if not isinstance(reset_info, dict):
                raise TypeError("basketball reset info must be a dictionary")
            info = {**info, "reset_info": reset_info}
            step_infos[-1] = info
            next_observations.append(reset_observation)
        else:
            next_observations.append(next_observation)
    next_batch = _validate_observations(next_observations, num_envs)
    final_batch = _validate_observations(final_observations, num_envs)
    terminated_tensor = torch.tensor(terminations, dtype=torch.bool)
    truncated_tensor = torch.tensor(truncations, dtype=torch.bool)
    with torch.inference_mode():
        bootstrap_values = critic(torch.from_numpy(final_batch))
        bootstrap_values = torch.where(
            terminated_tensor,
            torch.zeros_like(bootstrap_values),
            bootstrap_values,
        )
    stored["observations"].append(observation_tensor)
    stored["actions"].append(actions)
    stored["old_log_probs"].append(log_probs)
    stored["old_values"].append(values)
    stored["rewards"].append(torch.tensor(rewards, dtype=torch.float32))
    stored["terminated"].append(terminated_tensor)
    stored["truncated"].append(truncated_tensor)
    stored["bootstrap_values"].append(bootstrap_values)
    stored["episode_starts"].append(episode_starts.clone())
    all_infos.append(step_infos)
    observations = next_batch
    hidden = next_hidden
    cell = next_cell
    episode_starts = terminated_tensor | truncated_tensor
    rollout = Rollout(
        **{name: torch.stack(values) for name, values in stored.items()},
        initial_hidden=initial_hidden,
        initial_cell=initial_cell,
        next_hidden=hidden.detach(),
        next_cell=cell.detach(),

```

```

        next_episode_starts=episode_starts,
        infos=all_infos,
    )
    return rollout, observations, hidden.detach(), cell.detach(), episode_starts

def ppo_update(
    actor: BasketballActor,
    critic: BasketballCritic,
    optimizer: torch.optim.Optimizer,
    rollout: Rollout,
    *,
    epochs: int = 5,
    clip_coefficient: float = 0.2,
    value_coefficient: float = 1.0,
    entropy_coefficient: float = 0.01,
    max_grad_norm: float = 1.0,
    target_kl: float = 0.02,
) -> dict[str, float]:
    advantages, returns = compute_gae(
        rollout.rewards,
        rollout.old_values,
        rollout.bootstrap_values,
        rollout.terminated,
        rollout.truncated,
    )
    advantages = (advantages - advantages.mean()) / (
        advantages.std(unbiased=False) + 1e-8
    )
    actor.train()
    critic.train()
    metrics: dict[str, float] = {}
    for epoch in range(epochs):
        means, _, _ = actor.forward_sequence(
            rollout.observations,
            rollout.initial_hidden,
            rollout.initial_cell,
            rollout.episode_starts,
        )
        std = actor.action_std()
        log_probs = gaussian_log_prob(rollout.actions, means, std)
        log_ratio = log_probs - rollout.old_log_probs
        ratio = torch.exp(log_ratio)
        approximate_kl = ((ratio - 1) - log_ratio).mean()
        if not torch.isfinite(approximate_kl):
            raise RuntimeError("PP0 KL became non-finite")
        if float(approximate_kl.detach()) > target_kl:
            metrics["approx_kl"] = float(approximate_kl.detach())
            metrics["epochs_completed"] = epoch
            metrics["kl_early_stop"] = 1.0
            break
        unclipped = ratio * advantages
        clipped = torch.clamp(
            ratio,
            1.0 - clip_coefficient,
            1.0 + clip_coefficient,
        ) * advantages
        policy_loss = -torch.minimum(unclipped, clipped).mean()
        values = critic(rollout.observations.reshape(-1, OBSERVATION_DIM)).reshape(
            rollout.rewards.shape
        )
        value_loss = 0.5 * (values - returns).square().mean()
        entropy = gaussian_entropy(std)
        loss = (
            policy_loss
            + value_coefficient * value_loss
            - entropy_coefficient * entropy
        )
        if not torch.isfinite(loss):
            raise RuntimeError("PP0 loss became non-finite")
        optimizer.zero_grad(set_to_none=True)

```

```

    loss.backward()
    gradient_norm = nn.utils.clip_grad_norm_(
        list(actor.parameters()) + list(critic.parameters()),
        max_grad_norm,
    )
    if not torch.isfinite(torch.as_tensor(gradient_norm)):
        raise RuntimeError("PPO gradient norm became non-finite")
    optimizer.step()
    if not all(
        torch.isfinite(parameter).all()
        for parameter in list(actor.parameters()) + list(critic.parameters())
    ):
        raise RuntimeError("PPO parameters became non-finite")
    with torch.no_grad():
        updated_means, _, _ = actor.forward_sequence(
            rollout.observations, rollout.initial_hidden, rollout.initial_cell,
            rollout.episode_starts,
        )
        updated_log_probs = gaussian_log_prob(rollout.actions, updated_means,
        ↪ actor.action_std())
        updated_log_ratio = updated_log_probs - rollout.old_log_probs
        final_kl = (torch.exp(updated_log_ratio) - 1 - updated_log_ratio).mean()
        if not torch.isfinite(final_kl):
            raise RuntimeError("post-update PPO KL became non-finite")
        metrics = {
            "loss": float(loss.detach()),
            "policy_loss": float(policy_loss.detach()),
            "value_loss": float(value_loss.detach()),
            "entropy": float(entropy.detach()),
            "gradient_norm": float(gradient_norm),
            "approx_kl": float(final_kl),
            "epochs_completed": epoch + 1,
            "kl_early_stop": 0.0,
        }
        if float(final_kl) > target_kl:
            metrics["kl_early_stop"] = 1.0
            break
    return metrics

def actor_weight_change(
    actor: BasketballActor,
    initial_state: dict[str, torch.Tensor],
) -> float:
    total = torch.tensor(0.0)
    current = actor.state_dict()
    for name, source in initial_state.items():
        if name.startswith("obs_normalizer.") or name == "distribution.std_param":
            continue
        total += (current[name].cpu() - source.cpu()).square().sum()
    return float(torch.sqrt(total))

def save_results(
    output: Path,
    actor: BasketballActor,
    critic: BasketballCritic,
    optimizer: torch.optim.Optimizer,
    *,
    args: argparse.Namespace,
    source_metadata: dict[str, Any],
    initial_actor_state: dict[str, torch.Tensor],
    local_steps: int,
    update_metrics: list[dict[str, float]],
) -> dict[str, Any]:
    from rlx.environments import basketball as basketball_environment

    output.mkdir(parents=True, exist_ok=True)
    checkpoint_path = output / "checkpoint.pt"
    onnx_path = output / "policy.onnx"

```

```

checkpoint = {
    "format": LOCAL_CHECKPOINT_FORMAT,
    "actor_state_dict": actor.state_dict(),
    "critic_state_dict": critic.state_dict(),
    "optimizer_state_dict": optimizer.state_dict(),
    "local_steps": local_steps,
    "total_steps": source_metadata.get("source_training_steps", 0) + local_steps,
    "local_updates": int(update_metrics[-1]["update"]) if update_metrics else 0,
    "source": source_metadata,
    "normalization_frozen": True,
    "actor_observation_dim": OBSERVATION_DIM,
    "critic_observation_dim": OBSERVATION_DIM,
}
torch.save(checkpoint, checkpoint_path)
export_recurrent_onnx(actor, onnx_path)
parity = compare_actor_to_onnx(actor, onnx_path)
summary = {
    "created_at": datetime.now(timezone.utc).isoformat(),
    "continuation_kind": "actor warm-start with fresh local critic and optimizer",
    "full_upstream_resume": False,
    "source_kind": source_metadata.get("source_kind", "released_actor"),
    "source_training_steps": source_metadata.get("source_training_steps", 0),
    "source_checkpoint": str(args.checkpoint.resolve()),
    "source_checkpoint_sha256": source_metadata["source_hashes"][
        "checkpoint_sha256"
    ],
    "source_onnx_sha256": source_metadata["source_hashes"].get("onnx_sha256"),
    "source_iteration": source_metadata["source_iteration"],
    "checkpoint": str(checkpoint_path.resolve()),
    "checkpoint_sha256": sha256_file(checkpoint_path),
    "onnx": str(onnx_path.resolve()),
    "onnx_sha256": sha256_file(onnx_path),
    "onnx_parity": parity,
    "local_steps": local_steps,
    "total_steps": source_metadata.get("source_training_steps", 0) + local_steps,
    "local_updates": int(update_metrics[-1]["update"]) if update_metrics else 0,
    "actor_weight_change_l2": actor_weight_change(actor, initial_actor_state),
    "normalization_frozen": True,
    "actor_observation_dim": OBSERVATION_DIM,
    "ball_state_actor": False,
    "environment": {
        "exact_upstream_recipe": False,
        "reward_weights": dict(basketball_environment.REWARD_WEIGHTS),
        "reward_scale": CTRL_DT,
        "omitted_upstream_terms": ["pose", "angular_momentum", "joint_limits",
            ↪ "self_collision"],
        "hold_levels": list(basketball_environment.HOLD_LEVELS),
        "source_sha256": sha256_file(basketball_environment.__file__),
    },
    "settings": {
        "num_envs": args.num_envs,
        "steps": args.steps,
        "learning_rate": args.learning_rate,
        "initial_std": args.initial_std,
        "hold": args.hold,
        "curriculum": args.curriculum,
        "actuator": args.actuator,
        "seed": args.seed,
        "command": args.command,
        "pushes": args.pushes,
        "control_dt": CTRL_DT,
        "target_kl": args.target_kl,
        "obs_noise": False,
        "domain_rand": False,
        "randomized_commands": args.command is None,
    },
    "updates": update_metrics,
}
(output / "summary.json").write_text(
    json.dumps(summary, indent=2, sort_keys=True, allow_nan=False) + "\n"
)

```

`return` summary

```
def train(args: argparse.Namespace) -> dict[str, Any]:
    if args.output.exists() and (not args.output.is_dir() or any(args.output.iterdir())):
        raise FileExistsError(f"refusing to overwrite an existing run: {args.output}")
    torch.set_num_threads(1)
    torch.manual_seed(args.seed)
    np.random.seed(args.seed)
    actor, source_metadata = load_source_actor(
        args.checkpoint,
        initial_std=args.initial_std,
        verify_hash=True,
        allow_local=True,
    )
    initial_actor_state = {
        name: value.detach().clone() for name, value in actor.state_dict().items()
    }
    critic = BasketballCritic(actor.obs_normalizer)
    parameters = list(actor.parameters()) + [
        parameter
        for name, parameter in critic.named_parameters()
        if not name.startswith("normalizer.")
    ]
    optimizer = torch.optim.Adam(parameters, lr=args.learning_rate)
    environments = make_environments(args)
    generator = torch.Generator().manual_seed(args.seed)
    reset_counts = np.zeros(args.num_envs, dtype=np.int64)
    update_metrics: list[dict[str, float]] = []
    local_steps = 0
    try:
        observations, _ = reset_environments(environments, seed=args.seed)
        hidden, cell = actor.initial_state(args.num_envs)
        episode_starts = torch.ones(args.num_envs, dtype=torch.bool)
        for update in range(args.updates):
            rollout, observations, hidden, cell, episode_starts = collect_rollout(
                actor,
                critic,
                environments,
                observations,
                hidden,
                cell,
                episode_starts,
                steps=args.steps,
                generator=generator,
                seed=args.seed,
                reset_counts=reset_counts,
            )
            metrics = ppo_update(actor, critic, optimizer, rollout, target_kl=args.target_kl)
            metrics["hold_min"] = min(environment.hold for environment in environments)
            metrics["hold_max"] = max(environment.hold for environment in environments)
            metrics["completed_episodes"] = int(reset_counts.sum())
            local_steps += args.num_envs * args.steps
            update_metrics.append({"update": update + 1, **metrics})
            print(
                json.dumps(
                    {
                        "event": "basketball_ppo_update",
                        "update": update + 1,
                        "updates": args.updates,
                        "local_steps": local_steps,
                        **metrics,
                    },
                    allow_nan=False,
                ),
                flush=True,
            )
        if (update + 1) % args.save_interval == 0 and update + 1 < args.updates:
            save_results(
                args.output / f"update-{update + 1:04d}", actor, critic, optimizer,
                args=args, source_metadata=source_metadata,
```

```

        initial_actor_state=initial_actor_state,
        local_steps=local_steps, update_metrics=update_metrics,
    )
    finally:
        for environment in environments:
            environment.close()
    return save_results(
        args.output,
        actor,
        critic,
        optimizer,
        args=args,
        source_metadata=source_metadata,
        initial_actor_state=initial_actor_state,
        local_steps=local_steps,
        update_metrics=update_metrics,
    )

def _positive_int(value: str) -> int:
    parsed = int(value)
    if parsed < 1:
        raise argparse.ArgumentTypeError("must be positive")
    return parsed

def _positive_float(value: str) -> float:
    parsed = float(value)
    if not math.isfinite(parsed) or parsed <= 0.0:
        raise argparse.ArgumentTypeError("must be finite and positive")
    return parsed

def _nonnegative_float(value: str) -> float:
    parsed = float(value)
    if not math.isfinite(parsed) or parsed < 0.0:
        raise argparse.ArgumentTypeError("must be finite and nonnegative")
    return parsed

def parse_args(argv: Sequence[str] | None = None) -> argparse.Namespace:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--checkpoint", type=Path, default=DEFAULT_CHECKPOINT)
    parser.add_argument(
        "--output",
        type=Path,
        default=Path("runs/basketball/local-recurrent-ppo"),
    )
    parser.add_argument("--num-envs", type=_positive_int, default=4)
    parser.add_argument("--updates", type=_positive_int, default=5)
    parser.add_argument("--steps", type=_positive_int, default=32)
    parser.add_argument("--learning-rate", type=_positive_float, default=2e-5)
    parser.add_argument("--target-kl", type=_positive_float, default=0.02)
    parser.add_argument("--save-interval", type=_positive_int, default=25)
    parser.add_argument("--initial-std", type=_positive_float, default=0.03)
    parser.add_argument("--hold", type=_nonnegative_float, default=0.0)
    parser.add_argument(
        "--curriculum",
        action=argparse.BooleanOptionalAction,
        default=False,
    )
    parser.add_argument("--actuator", default="bam")
    parser.add_argument("--seed", type=int, default=42)
    commands = parser.add_mutually_exclusive_group()
    commands.add_argument(
        "--command",
        type=float,
        nargs=3,
        metavar=("VX", "VY", "WZ"),
        default=(0.08, 0.0, 0.0),
    )

```

```

commands.add_argument("--randomized-commands", action="store_true")
parser.add_argument(
    "--pushes",
    action=argparse.BooleanOptionalAction,
    default=False,
)
args = parser.parse_args(argv)
if args.randomized_commands:
    args.command = None
if not args.checkpoint.is_file():
    parser.error(f"checkpoint does not exist: {args.checkpoint}")
if args.command is not None and not all(math.isfinite(x) for x in args.command):
    parser.error("--command values must be finite")
return args

def main(argv: Sequence[str] | None = None) -> int:
    args = parse_args(argv)
    summary = train(args)
    print(json.dumps(summary, allow_nan=False), flush=True)
    return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

9.4 A4. rlx/rlx/environments/bridge.py

SHA-256: 49bd21b6260841bb06ab1d04fcca18cd79c478e0484d8d722775353a04a41f37

```

"""Suspended narrow-bridge environment for Microduck locomotion prototypes."""

from __future__ import annotations

import math
from pathlib import Path
from types import MappingProxyType

import mujoco
import numpy as np

from microduck_local import contract as C
from microduck_local.walk_env import MicroduckWalkEnv

ROBOT_DIR = Path(__file__).resolve().parents[1] / "mjlab_microduck/robot/microduck"

CTRL_DT = C.CTRL_DT
GROUND_Z = 0.0
PLATFORM_TOP_Z = 0.24
PLATFORM_HALF_LENGTH = 0.30
PLATFORM_HALF_WIDTH = 0.24
START_PLATFORM_X = -0.85
END_PLATFORM_X = 0.85
BRIDGE_HALF_LENGTH = 0.55
BRIDGE_HALF_WIDTH = 0.065
BRIDGE_HALF_THICKNESS = 0.015
BRIDGE_MASS = 0.45
BRIDGE_CENTER_Z = PLATFORM_TOP_Z - BRIDGE_HALF_THICKNESS
FOOT_CONTACT_SETTLE_M = 0.003
ASSISTANCE_LEVELS = (1.0, 0.6, 0.3, 0.1, 0.0)
SPAWN_X_LEVELS = (0.0, -0.30, -0.55, -0.75, START_PLATFORM_X - 0.05)

CURRICULUM_KNOBS = MappingProxyType({
    "assistance": "plank restoring force and level torque only",
    "spawn_x": "progressive physical start position from plank to launch platform",
})

```

```

def _suspension_length(x: float, y: float, z: float) -> float:
    return math.sqrt(x * x + y * y + z * z)

def bridge_model(actuator: str = "xml") -> mujoco.MjModel:
    """Build a free suspended plank between two fixed safe platforms."""
    if actuator not in ("xml", "bam"):
        raise ValueError("actuator must be xml or bam")

    spec = mujoco.MjSpec.from_file(str(ROBOT_DIR / "robot_allcollisions.xml"))
    spec.option.timestep = C.PHYSICS_DT
    spec.option.integrator = mujoco.mjtIntegrator.mjINT_IMPLICITFAST
    spec.option.cone = mujoco.mjtCone.mjCONE_PYRAMIDAL
    spec.option.iterations = 12
    spec.option.ls_iterations = 24

    ground_mat = spec.add_material(name="bridge_ground_mat", rgba=(0.10, 0.12, 0.14, 1))
    platform_mat = spec.add_material(name="bridge_platform_mat", rgba=(0.28, 0.31, 0.34, 1))
    plank_mat = spec.add_material(name="bridge_plank_mat", rgba=(0.52, 0.31, 0.13, 1))
    cable_mat = spec.add_material(name="bridge_cable_mat", rgba=(0.75, 0.77, 0.78, 1))
    del ground_mat, platform_mat, plank_mat, cable_mat

    spec.worldbody.add_geom(
        name="floor",
        type=mujoco.mjtGeom.mjGEOM_PLANE,
        pos=(0, 0, GROUND_Z),
        size=(0, 0, 0.05),
        material="bridge_ground_mat",
        friction=(1.0, 0.005, 0.0001),
    )
    for name, x in (
        ("start_platform", START_PLATFORM_X),
        ("end_platform", END_PLATFORM_X),
    ):
        spec.worldbody.add_geom(
            name=name,
            type=mujoco.mjtGeom.mjGEOM_BOX,
            group=2,
            pos=(x, 0, PLATFORM_TOP_Z / 2),
            size=(PLATFORM_HALF_LENGTH, PLATFORM_HALF_WIDTH, PLATFORM_TOP_Z / 2),
            material="bridge_platform_mat",
            friction=(1.1, 0.005, 0.0001),
            priority=1,
        )

    plank = spec.worldbody.add_body(
        name="bridge_plank",
        pos=(0, 0, BRIDGE_CENTER_Z),
    )
    plank.add_freejoint(name="bridge_freejoint")
    plank.add_geom(
        name="bridge_surface",
        type=mujoco.mjtGeom.mjGEOM_BOX,
        group=2,
        size=(BRIDGE_HALF_LENGTH, BRIDGE_HALF_WIDTH, BRIDGE_HALF_THICKNESS),
        mass=BRIDGE_MASS,
        material="bridge_plank_mat",
        friction=(1.15, 0.006, 0.0002),
        priority=1,
        condim=4,
    )

    anchor_z = 0.77
    anchor_y = 0.17
    attach_y = BRIDGE_HALF_WIDTH - 0.008
    attach_x = BRIDGE_HALF_LENGTH - 0.10
    vertical = anchor_z - BRIDGE_CENTER_Z
    cable_length = _suspension_length(0.0, anchor_y - attach_y, vertical)
    for longitudinal, x in (("start", -attach_x), ("end", attach_x)):
        spec.worldbody.add_geom(
            name=f"bridge_{longitudinal}_crossbar", type=mujoco.mjtGeom.mjGEOM_BOX,

```



```

        pos=(x, 0, anchor_z + 0.014), size=(0.014, 0.24, 0.014),
        material="bridge_cable_mat", contype=0, conaffinity=0, group=2,
    )
    for post_side in (-1, 1):
        spec.worldbody.add_geom(
            name=f"bridge_{longitudinal}_post_{post_side}", type=mujoco.mjtGeom.mjGEOM_BOX,
            pos=(x, post_side * 0.225, anchor_z / 2),
            size=(0.014, 0.014, anchor_z / 2), material="bridge_cable_mat",
            contype=0, conaffinity=0, group=2,
        )
    for lateral, anchor_side, attach_side in (
        ("left", anchor_y, attach_y),
        ("right", -anchor_y, -attach_y),
    ):
        name = f"bridge_{longitudinal}_{lateral}"
        spec.worldbody.add_site(
            name=f"{name}_anchor",
            pos=(x, anchor_side, anchor_z),
            size=(0.004,),
            rgba=(0.75, 0.77, 0.78, 1),
        )
        plank.add_site(
            name=f"{name}_attach",
            pos=(x, attach_side, 0),
            size=(0.004,),
            rgba=(0.75, 0.77, 0.78, 1),
        )
        tendon = spec.add_tendon(
            name=f"{name}_cable",
            stiffness=320.0,
            damping=2.0,
            springlength=(0.0, cable_length - 0.0015),
            limited=True,
            range=(0.0, cable_length + 0.008),
            width=0.0015,
            rgba=(0.75, 0.77, 0.78, 1),
            solref_limit=(0.015, 1.0),
            solimp_limit=(0.90, 0.95, 0.001, 0.5, 2.0),
        )
        tendon.wrap_site(f"{name}_anchor")
        tendon.wrap_site(f"{name}_attach")

    spec.worldbody.add_light(pos=(0, -1.5, 3), dir=(0, 0.3, -1),
                             diffuse=(0.85, 0.85, 0.85), ambient=(0.3, 0.3, 0.3))
    spec.add_key(
        name="STAND",
        qpos=np.concatenate((
            (0, 0, 0.12, 1, 0, 0, 0),
            C.DEFAULT_POSE,
            (0, 0, BRIDGE_CENTER_Z, 1, 0, 0, 0),
        )),
        ctrl=C.DEFAULT_POSE,
    )
    model = spec.compile()
    model.vis.global_.offwidth = 1280
    model.vis.global_.offheight = 960
    return model

def curriculum_level(stage: int, *, crossed: bool, elapsed_s: float) -> int:
    """Advance after an honest crossing; retreat after an immediate failure."""
    if (
        not 0 <= stage < len(ASSISTANCE_LEVELS)
        or not math.isfinite(elapsed_s)
        or elapsed_s < 0
    ):
        raise ValueError("invalid curriculum stage or elapsed time")
    if crossed:
        return min(stage + 1, len(ASSISTANCE_LEVELS) - 1)
    if elapsed_s < 2.0:
        return max(stage - 1, 0)

```

```
return stage
```

```
class BridgeEnv(MicroduckWalkEnv):
    """Gymnasium environment for crossing a physically suspended narrow plank.

    The policy receives the standard 61-float Microduck observation only.
    Bridge pose, world progress, contacts, and crossing state are privileged
    metrics. Assistance acts on the plank, never on the robot, and the inherited
    locomotion reward is identical at every curriculum level.
    """

    def __init__(
        self,
        *,
        max_episode_s: float = 20.0,
        seed: int | None = None,
        obs_noise: bool = False,
        domain_rand: bool = False,
        pushes: bool = False,
        random_yaw: bool = False,
        actuator: str = "xml",
        action_delay: bool = False,
        command: tuple[float, float, float] | None = (0.25, 0.0, 0.0),
        navigation: bool = True,
        assistance: float | None = None,
        curriculum: bool = False,
        model: mujoco.MjModel | None = None,
    ) -> None:
        if not math.isfinite(max_episode_s) or max_episode_s <= 0:
            raise ValueError("episode length must be positive and finite")
        if assistance is None:
            assistance = 1.0 if curriculum else 0.0
        if not math.isfinite(assistance) or not 0 <= assistance <= 1:
            raise ValueError("assistance must be finite and between zero and one")
        if curriculum and assistance not in ASSISTANCE_LEVELS:
            raise ValueError("curriculum assistance must be one of ASSISTANCE_LEVELS")

        self.fixed_command = self._validate_command(command)
        self.navigation = bool(navigation)
        self.assistance = float(assistance)
        self.curriculum = bool(curriculum)
        self.stage = (
            ASSISTANCE_LEVELS.index(assistance)
            if assistance in ASSISTANCE_LEVELS
            else len(ASSISTANCE_LEVELS) - 1
        )
        self.pushes = bool(pushes)
        self._bridge_random_yaw = bool(random_yaw)
        self._episode_finished = False
        self._crossed = False
        self._bridge_contact_observed = False
        self._next_push = math.inf
        self._last_push_force = 0.0

        super().__init__(
            max_episode_s=max_episode_s,
            command_resample_s=5.0,
            obs_noise=obs_noise,
            domain_rand=domain_rand,
            action_delay=action_delay,
            random_yaw=False,
            seed=seed,
            actuator_force=actuator,
            model=model if model is not None else bridge_model(actuator),
            terminate_on_fall=False,
            height_termination=False,
        )

        self.bridge_body_id = self.model.body("bridge_plank").id
        self.bridge_geom_id = self.model.geom("bridge_surface").id
```

```

self.bridge_qpos_adr = int(self.model.joint("bridge_freejoint").qposadr[0])
self.bridge_qvel_adr = int(self.model.joint("bridge_freejoint").dofadr[0])
self.start_platform_geom = self.model.geom("start_platform").id
self.end_platform_geom = self.model.geom("end_platform").id
self.support_geoms = {
    self.start_platform_geom,
    self.bridge_geom_id,
    self.end_platform_geom,
}
self._bridge_nominal_position = np.array((0.0, 0.0, BRIDGE_CENTER_Z))

@staticmethod
def _validate_command(command):
    if command is None:
        return None
    array = np.asarray(command, dtype=np.float32)
    if array.shape != (3,) or not np.isfinite(array).all():
        raise ValueError("command must contain three finite numbers")
    return array.copy()

def set_command(self, command: tuple[float, float, float]) -> None:
    validated = self._validate_command(command)
    if validated is None:
        raise ValueError("set_command requires three numbers")
    self.fixed_command = validated
    self.twist_cmd[:] = validated

def _sample_commands(self) -> None:
    if self.fixed_command is not None:
        self.twist_cmd[:] = self.fixed_command
    else:
        self.twist_cmd[:] = (
            self._rng.uniform(0.12, 0.24),
            self._rng.uniform(-0.025, 0.025),
            self._rng.uniform(-0.12, 0.12),
        )
    self.head_cmd[:] = 0
    self.body_cmd[:] = 0

def _update_navigation_command(self) -> None:
    if not self.navigation or self.fixed_command is None:
        return
    rotation = self.data.xmat[self.trunk_body_id].reshape(3, 3)
    heading = rotation[:, 0].astype(np.float64)
    heading[2] = 0
    norm = float(np.linalg.norm(heading))
    heading = heading / norm if norm > 1e-9 else np.array((1.0, 0.0, 0.0))
    side = np.array((-heading[1], heading[0], 0.0))
    reached_destination = (
        self.data.xpos[self.trunk_body_id, 0] >= END_PLATFORM_X - 0.05
    )
    desired_world = np.array((
        0.0 if reached_destination else float(self.fixed_command[0]),
        float(np.clip(-1.2 * self.data.xpos[self.trunk_body_id, 1], -0.12, 0.12)),
        0.0,
    ))
    yaw_error = math.atan2(float(heading[1]), float(heading[0]))
    self.twist_cmd[:] = (
        float(desired_world @ heading),
        float(desired_world @ side),
        float(np.clip(-1.5 * yaw_error, -0.6, 0.6)),
    )

def _get_obs(self) -> np.ndarray:
    self._update_navigation_command()
    return super()._get_obs()

def reset(self, *, seed: int | None = None, options: dict | None = None):
    if self.curriculum and self._episode_finished:
        self.stage = curriculum_level(

```

```

        self.stage,
        crossed=self._crossed,
        elapsed_s=self.step_count * CTRL_DT,
    )
    self.assistance = ASSISTANCE_LEVELS[self.stage]

    self._episode_finished = False
    self._crossed = False
    self._bridge_contact_observed = False
    super().reset(seed=seed, options=options)

    spawn_x = (
        SPAWN_X_LEVELS[self.stage]
        if self.curriculum
        else START_PLATFORM_X - 0.05
    )
    self.data.qpos[:3] = (
        spawn_x,
        self._rng.uniform(-0.015, 0.015),
        self.stand_z + PLATFORM_TOP_Z - FOOT_CONTACT_SETTLE_M,
    )
    yaw = (
        self._rng.uniform(-math.radians(5), math.radians(5))
        if self._bridge_random_yaw
        else 0.0
    )
    self.data.qpos[3:7] = (math.cos(yaw / 2), 0, 0, math.sin(yaw / 2))
    self.data.qpos[self.joint_qpos_adr] = (
        C.DEFAULT_POSE + self._rng.uniform(-0.025, 0.025, C.NUM_JOINTS)
    )

    roll = self._rng.uniform(-math.radians(1.5), math.radians(1.5))
    pitch = self._rng.uniform(-math.radians(0.8), math.radians(0.8))
    rotation = np.empty(4)
    roll_quat = np.array((math.cos(roll / 2), math.sin(roll / 2), 0, 0))
    pitch_quat = np.array((math.cos(pitch / 2), 0, math.sin(pitch / 2), 0))
    mujoco.mju_mulQuat(rotation, pitch_quat, roll_quat)
    self.data.qpos[
        self.bridge_qpos_adr:self.bridge_qpos_adr + 7
    ] = (*self._bridge_nominal_position, *rotation)
    self.data.qvel[:] = 0
    self.data.qvel[self.bridge_qvel_adr + 3:self.bridge_qvel_adr + 6] = (
        self._rng.uniform(-0.025, 0.025, 3)
    )
    self.data.xfrc_applied[:] = 0
    self.data.ctrl[:] = self.data.qpos[self.joint_qpos_adr]
    mujoco.mj_forward(self.model, self.data)
    if self.bam is not None:
        self.bam.reset(self.data.qpos[self.joint_qpos_adr])
    self.prev_joint_vel[:] = self._joint_vel()
    self._next_push = self._rng.uniform(1.5, 3.0) if self.pushes else math.inf
    self._last_push_force = 0.0
    return self._get_obs(), self.metrics()

def _apply_physics_curriculum(self) -> None:
    self.data.xfrc_applied[:] = 0
    self._last_push_force = 0.0
    if self.assistance:
        pos = self.data.xpos[self.bridge_body_id]
        vel = self.data.qvel[self.bridge_qvel_adr:self.bridge_qvel_adr + 3]
        rotation = self.data.xmat[self.bridge_body_id].reshape(3, 3)
        angular_world = (
            rotation
            @ self.data.qvel[self.bridge_qvel_adr + 3:self.bridge_qvel_adr + 6]
        )
        displacement = pos - self._bridge_nominal_position
        restoring_force = -45.0 * displacement - 4.0 * vel
        restoring_force[2] = -25.0 * displacement[2] - 2.5 * vel[2]
        restoring_torque = (
            3.5 * np.cross(rotation[:, 2], np.array((0.0, 0.0, 1.0)))

```

```

        - 0.18 * angular_world
    )
    self.data.xfrc_applied[self.bridge_body_id, :3] = (
        self.assistance * restoring_force
    )
    self.data.xfrc_applied[self.bridge_body_id, 3:] = (
        self.assistance * restoring_torque
    )

    if self.step_count * CTRL_DT >= self._next_push:
        self._last_push_force = float(self._rng.choice((-3.0, 3.0)))
        self.data.xfrc_applied[self.trunk_body_id, 1] = self._last_push_force
        self._next_push += self._rng.uniform(1.5, 3.0)

def step(self, action):
    action = np.asarray(action, dtype=np.float32)
    if action.shape != (C.NUM_JOINTS,) or not np.isfinite(action).all():
        raise ValueError("actions must be finite with shape (14,)")
    if self._episode_finished:
        raise RuntimeError("reset is required after a terminal episode")

    self._apply_physics_curriculum()
    observation, reward, terminated, truncated, info = super().step(action)
    metrics = self.metrics()
    reasons = []
    if metrics["robot_ground_contact"]:
        reasons.append("robot_ground_contact")
    if metrics["tilt_deg"] > 70:
        reasons.append("tilt")
    if metrics["trunk_height_m"] < 0.10:
        reasons.append("height")
    if (
        abs(metrics["trunk_lateral_m"]) > PLATFORM_HALF_WIDTH + 0.12
        and metrics["supported_feet"] == 0
    ):
        reasons.append("fell_off_side")
    if not np.isfinite(self.data.qpos).all() or not np.isfinite(self.data.qvel).all():
        reasons.append("nonfinite_physics")

    terminated = terminated or bool(reasons)
    self._episode_finished = terminated or truncated
    info.update(metrics)
    info["termination_reasons"] = reasons
    if self._episode_finished:
        info["episode_rewards"] = dict(self.reward_sums)
    return observation, reward, terminated, truncated, info

def _contact_state(self) -> dict:
    feet = self.foot_geoms
    foot_supports: dict[str, set[int]] = {"left": set(), "right": set()}
    foot_ground = {"left": False, "right": False}
    robot_ground = False
    robot_bridge = False
    nonfoot_support = False
    foot_geom_ids = set(feet.values())
    for contact in self.data.contact:
        geom1 = int(contact.geom1)
        geom2 = int(contact.geom2)
        pair = {geom1, geom2}
        for side, foot in feet.items():
            if foot not in pair:
                continue
            other = geom2 if geom1 == foot else geom1
            if other in self.support_geoms:
                foot_supports[side].add(other)
            if other == self.floor_geom:
                foot_ground[side] = True
    if self.floor_geom in pair:
        other = geom2 if geom1 == self.floor_geom else geom1
        body_id = int(self.model.geom_bodyid[other])

```

```

        if body_id not in (0, self.bridge_body_id):
            robot_ground = True
        if self.bridge_geom_id in pair:
            other = geom2 if geom1 == self.bridge_geom_id else geom1
            body_id = int(self.model.geom_bodyid[other])
            if body_id not in (0, self.bridge_body_id):
                robot_bridge = True
        if pair & self.support_geoms:
            support = next(iter(pair & self.support_geoms))
            other = geom2 if geom1 == support else geom1
            body_id = int(self.model.geom_bodyid[other])
            if body_id not in (0, self.bridge_body_id) and other not in foot_geom_ids:
                nonfoot_support = True
    return {
        "foot_supports": foot_supports,
        "foot_ground": foot_ground,
        "robot_ground": robot_ground,
        "robot_bridge": robot_bridge,
        "nonfoot_support": nonfoot_support,
    }

def _foot_contacts(self) -> dict[str, bool]:
    contacts = self._contact_state()["foot_supports"]
    return {side: bool(surfaces) for side, surfaces in contacts.items()}

def metrics(self) -> dict:
    contacts = self._contact_state()
    foot_supports = contacts["foot_supports"]
    feet_on_bridge = (
        int(self.bridge_geom_id in foot_supports["left"])
        + int(self.bridge_geom_id in foot_supports["right"])
    )
    self._bridge_contact_observed = (
        self._bridge_contact_observed or feet_on_bridge > 0
    )
    left_end = self.end_platform_geom in foot_supports["left"]
    right_end = self.end_platform_geom in foot_supports["right"]
    full_traversal = float(self.data.xpos[self.trunk_body_id, 0]) >= (
        END_PLATFORM_X - 0.08
    )
    accepted = (
        full_traversal
        and left_end
        and right_end
        and not any(contacts["foot_ground"].values())
        and not contacts["robot_ground"]
        and not contacts["nonfoot_support"]
        and self._bridge_contact_observed
    )
    self._crossed = self._crossed or accepted

    rotation = self.data.xmat[self.bridge_body_id].reshape(3, 3)
    plank_up = rotation[:, 2]
    roll = math.atan2(float(rotation[2, 1]), float(rotation[2, 2]))
    pitch = math.atan2(
        float(-rotation[2, 0]),
        math.sqrt(float(rotation[2, 1]**2 + rotation[2, 2]**2)),
    )
    gravity = self._projected_gravity()
    return {
        "elapsed_s": self.step_count * CTRL_DT,
        "assistance": self.assistance,
        "curriculum_stage": self.stage,
        "command": self.twist_cmd.tolist(),
        "trunk_x_m": float(self.data.xpos[self.trunk_body_id, 0]),
        "trunk_lateral_m": float(self.data.xpos[self.trunk_body_id, 1]),
        "trunk_height_m": float(self.data.xpos[self.trunk_body_id, 2]),
        "tilt_deg": math.degrees(
            math.acos(float(np.clip(-gravity[2], -1.0, 1.0)))
        ),
    },

```

```

        "bridge_position": self.data.xpos[self.bridge_body_id].tolist(),
        "bridge_roll_deg": math.degrees(roll),
        "bridge_pitch_deg": math.degrees(pitch),
        "bridge_up_z": float(plank_up[2]),
        "bridge_speed_mps": float(
            np.linalg.norm(
                self.data.qvel[
                    self.bridge_qvel_adr:self.bridge_qvel_adr + 3
                ]
            )
        ),
        "left_support": sorted(foot_supports["left"]),
        "right_support": sorted(foot_supports["right"]),
        "supported_feet": int(bool(foot_supports["left"]))
        + int(bool(foot_supports["right"])),
        "feet_on_bridge": feet_on_bridge,
        "feet_on_end_platform": int(left_end) + int(right_end),
        "foot_ground_contacts": int(contacts["foot_ground"]["left"])
        + int(contacts["foot_ground"]["right"]),
        "robot_ground_contact": contacts["robot_ground"],
        "robot_bridge_contact": contacts["robot_bridge"],
        "nonfoot_support": contacts["nonfoot_support"],
        "full_traversal": full_traversal,
        "bridge_contact_observed": self._bridge_contact_observed,
        "crossing_accepted": self._crossed,
        "push_force_n": self._last_push_force,
    }

def recipe_metrics(self) -> dict[str, float]:
    """Numeric-only privileged metrics for training logs and evaluation."""
    metrics = self.metrics()
    return {
        "bridge_crossed": float(metrics["crossing_accepted"]),
        "bridge_progress_m": float(
            metrics["trunk_x_m"] - (START_PLATFORM_X - 0.05)
        ),
        "bridge_assistance": float(self.assistance),
        "robot_floor_contact": float(metrics["robot_ground_contact"]),
        "nonfoot_support": float(metrics["nonfoot_support"]),
        "feet_support": float(metrics["supported_feet"]),
        "bridge_contact_observed": float(
            metrics["bridge_contact_observed"]
        ),
        "feet_on_bridge": float(metrics["feet_on_bridge"]),
        "feet_on_end_platform": float(metrics["feet_on_end_platform"]),
        "bridge_roll_deg": float(metrics["bridge_roll_deg"]),
        "bridge_pitch_deg": float(metrics["bridge_pitch_deg"]),
        "bridge_speed_mps": float(metrics["bridge_speed_mps"]),
        "robot_tilt_deg": float(metrics["tilt_deg"]),
        "robot_lateral_m": float(metrics["trunk_lateral_m"]),
    }

```

9.5 A5. rlx/rlx/environments/bridge_evaluation.py

SHA-256: dd3b116933777d789e860e6a1398204ddfb60ae28e77cf3f351275049936078a

```

"""Strict per-episode acceptance for an unassisted suspended-bridge crossing."""

from __future__ import annotations

from dataclasses import asdict, dataclass, field
import math
from numbers import Integral, Real
from typing import Any

CONTROL_HZ = 50

```

```

REQUIRED_METRICS = (
    "bridge_crossed",
    "bridge_progress_m",
    "bridge_assistance",
    "robot_floor_contact",
    "nonfoot_support",
    "feet_support",
    "bridge_contact_observed",
)
_BINARY_METRICS = (
    "bridge_crossed",
    "robot_floor_contact",
    "nonfoot_support",
    "bridge_contact_observed",
)

@dataclass(frozen=True)
class BridgeEvaluationCriteria:
    version: int = 1
    required_steps: int = 1000
    control_hz: int = CONTROL_HZ
    min_progress_m: float = 1.60
    required_end_feet: int = 2
    requires_unassisted: bool = True
    requires_no_falls: bool = True
    all_episodes_required: bool = True

    def __post_init__(self) -> None:
        if (
            isinstance(self.required_steps, bool)
            or not isinstance(self.required_steps, Integral)
            or self.required_steps < 1
        ):
            raise ValueError("bridge required_steps must be a positive integer")

    def to_dict(self) -> dict[str, Any]:
        return asdict(self)

def _numeric_metrics(metrics: Any) -> dict[str, float] | None:
    if not isinstance(metrics, dict):
        return None
    values: dict[str, float] = {}
    for key in REQUIRED_METRICS:
        value = metrics.get(key)
        if isinstance(value, bool) or not isinstance(value, Real):
            return None
        converted = float(value)
        if not math.isfinite(converted):
            return None
        values[key] = converted
    return values

@dataclass
class _Episode:
    env_index: int
    index: int
    steps: int = 0
    measured_steps: int = 0
    max_progress_m: float = 0.0
    crossed: bool = False
    crossing_step: int | None = None
    start_support_observed: bool = False
    end_feet_at_crossing: int = 0
    assisted_steps: int = 0
    floor_contact_steps: int = 0
    nonfoot_support_steps: int = 0
    previous_crossed: float | None = None
    bridge_contact_observed: bool = False
    failures: set[str] = field(default_factory=set)

```



```

def observe(
    self,
    metrics: dict[str, Any],
    criteria: BridgeEvaluationCriteria,
) -> None:
    self.steps += 1
    values = _numeric_metrics(metrics)
    if values is None:
        self.failures.add("missing, malformed, or non-finite bridge metrics")
        return
    if any(values[key] not in (0.0, 1.0) for key in _BINARY_METRICS):
        self.failures.add("invalid binary bridge metrics")
        return
    if (
        values["bridge_progress_m"] < -0.5
        or values["bridge_assistance"] < 0
        or values["bridge_assistance"] > 1
        or values["feet_support"] not in (0.0, 1.0, 2.0)
    ):
        self.failures.add("invalid physical bridge metrics")
        return
    if self.previous_crossed == 1.0 and values["bridge_crossed"] == 0.0:
        self.failures.add("bridge_crossed regressed within an episode")

    self.measured_steps += 1
    self.max_progress_m = max(
        self.max_progress_m,
        values["bridge_progress_m"],
    )
    self.start_support_observed = (
        self.start_support_observed
        or (
            values["bridge_progress_m"] <= 0.10
            and values["feet_support"] >= 1.0
        )
    )
    self.assisted_steps += int(values["bridge_assistance"] > 0.0)
    self.floor_contact_steps += int(values["robot_floor_contact"] == 1.0)
    self.nonfoot_support_steps += int(values["nonfoot_support"] == 1.0)
    self.bridge_contact_observed = (
        self.bridge_contact_observed
        or values["bridge_contact_observed"] == 1.0
    )

    crossing_now = values["bridge_crossed"] == 1.0 and not self.crossed
    if crossing_now:
        self.crossed = True
        self.crossing_step = self.steps
        self.end_feet_at_crossing = int(values["feet_support"])
        if self.end_feet_at_crossing != criteria.required_end_feet:
            self.failures.add("crossing lacked two-foot landing support")
        if not self.bridge_contact_observed:
            self.failures.add(
                "crossing lacked suspended-plank foot-contact history"
            )
    self.previous_crossed = values["bridge_crossed"]

def finish(
    self,
    criteria: BridgeEvaluationCriteria,
    *,
    terminated: bool,
    truncated: bool,
) -> dict[str, Any]:
    failures = set(self.failures)
    complete = (
        truncated
        and not terminated
        and self.steps >= criteria.required_steps
    )

```

```

if terminated:
    failures.add("episode terminated or fell before evaluation completion")
if not complete:
    failures.add(
        f"episode did not complete the required "
        f"{criteria.required_steps / criteria.control_hz:g}-second horizon"
    )
if self.steps == 0 or self.measured_steps != self.steps:
    failures.add("incomplete bridge metric coverage")
if not self.start_support_observed:
    failures.add("launch-platform foot support was not observed")
if not self.crossed:
    failures.add("contact-validated end-to-end crossing was not observed")
if not self.bridge_contact_observed:
    failures.add("suspended-plank foot contact was not observed")
if self.max_progress_m < criteria.min_progress_m:
    failures.add("bridge progress remained below end-to-end target")
if self.assisted_steps:
    failures.add("bridge assistance was active during evaluation")
if self.floor_contact_steps:
    failures.add("robot contacted the floor")
if self.nonfoot_support_steps:
    failures.add("non-foot robot support contact occurred")

return {
    "env_index": self.env_index,
    "episode_index": self.index,
    "steps": self.steps,
    "measured_steps": self.measured_steps,
    "terminated": bool(terminated),
    "truncated": bool(truncated),
    "complete": complete,
    "max_progress_m": self.max_progress_m,
    "crossed": self.crossed,
    "crossing_step": self.crossing_step,
    "start_support_observed": self.start_support_observed,
    "end_feet_at_crossing": self.end_feet_at_crossing,
    "bridge_contact_observed": self.bridge_contact_observed,
    "assisted_steps": self.assisted_steps,
    "floor_contact_steps": self.floor_contact_steps,
    "nonfoot_support_steps": self.nonfoot_support_steps,
    "passed": not failures,
    "failures": sorted(failures),
}

```

```

class BridgeEvaluation:
    """Accumulate strict unassisted crossing evidence per vector lane."""

    criteria: BridgeEvaluationCriteria
    _current: list[_Episode]

    def __init__(self, num_envs: int, required_steps: int) -> None:
        if (
            isinstance(num_envs, bool)
            or not isinstance(num_envs, Integral)
            or num_envs < 1
        ):
            raise ValueError("num_envs must be positive")
        if (
            isinstance(required_steps, bool)
            or not isinstance(required_steps, Integral)
            or required_steps < 1
        ):
            raise ValueError("required_steps must be a positive integer")
        self.criteria = BridgeEvaluationCriteria(required_steps=int(required_steps))
        self._current = [_Episode(index, 0) for index in range(int(num_envs))]
        self._episodes: list[dict[str, Any]] = []

    def observe(
        self,

```

```

        env_index: int,
        metrics: dict[str, Any],
        terminated: bool = False,
        truncated: bool = False,
    ) -> None:
        if not 0 <= env_index < len(self._current):
            raise IndexError(f"env_index out of range: {env_index}")
        episode = self._current[env_index]
        episode.observe(metrics, self.criteria)
        if terminated or truncated:
            self._episodes.append(
                episode.finish(
                    self.criteria,
                    terminated=bool(terminated),
                    truncated=bool(truncated),
                )
            )
            self._current[env_index] = _Episode(env_index, episode.index + 1)

    def report(self) -> dict[str, Any]:
        episodes = [
            *self._episodes,
            *(
                episode.finish(
                    self.criteria,
                    terminated=False,
                    truncated=False,
                )
                for episode in self._current
                if episode.steps
            ),
        ]
        covered_lanes = {episode["env_index"] for episode in episodes}
        failures = {
            failure
            for episode in episodes
            for failure in episode["failures"]
        }
        if set(range(len(self._current))) - covered_lanes or not episodes:
            failures.add("missing evaluation lane coverage")
        return {
            "passed": bool(episodes) and not failures,
            "failures": sorted(failures),
            "episodes": episodes,
            "criteria": self.criteria.to_dict(),
            "completed_episodes": sum(
                bool(episode["complete"]) for episode in episodes
            ),
            "passed_episodes": sum(
                bool(episode["passed"]) for episode in episodes
            ),
            "measurement_scope": (
                "50 Hz per-episode numeric recipe metrics; crossing credit "
                "requires full progress and contact-validated two-foot landing"
            ),
            "hardware_safe": False,
        }

__all__ = [
    "BridgeEvaluation",
    "BridgeEvaluationCriteria",
    "REQUIRED_METRICS",
]

```

9.6 A6. rlx/rlx/models/bridge_bootstrap.py

SHA-256: f1dd30fbc474dcd00080e9dfa1026dcf1528cb946e640a94865882566584ca7a

```

"""Create a bridge-training warm start from the shipped walking actor."""

from __future__ import annotations

import hashlib
import math
from pathlib import Path
from typing import Any

import numpy as np

WALK_POLICY = (
    Path(__file__).resolve().parents[3] / "microduck" / "policies" / "alpha_walking.onnx"
)
EXPECTED_NODE_TYPES = (
    "Sub",
    "Div",
    "Gemm",
    "Elu",
    "Gemm",
    "Elu",
    "Gemm",
    "Elu",
    "Gemm",
)
EXPECTED_INITIALIZERS = {
    "obs_normalizer._mean": (1, 61),
    "onnx::Div_24": (1, 61),
    "mlp.0.weight": (512, 61),
    "mlp.0.bias": (512,),
    "mlp.2.weight": (256, 512),
    "mlp.2.bias": (256,),
    "mlp.4.weight": (128, 256),
    "mlp.4.bias": (128,),
    "mlp.6.weight": (14, 128),
    "mlp.6.bias": (14,),
}
PARITY_SAMPLES = 32
PARITY_THRESHOLD = 1e-4

def _fixed_shape(value_info: Any) -> tuple[int, ...]:
    return tuple(dimension.dim_value for dimension in value_info.tensor_type.shape.dim)

def _validated_source() -> tuple[Any, dict[str, np.ndarray], str]:
    import onnx
    from onnx import numpy_helper

    if not WALK_POLICY.is_file():
        raise FileNotFoundError(f"walking source policy not found: {WALK_POLICY}")
    source_hash = hashlib.sha256(WALK_POLICY.read_bytes()).hexdigest()
    source = onnx.load(WALK_POLICY)
    onnx.checker.check_model(source)

    if len(source.graph.input) != 1 or source.graph.input[0].name != "obs":
        raise ValueError("Walking teacher input contract changed")
    if _fixed_shape(source.graph.input[0]) != (1, 61):
        raise ValueError("Walking teacher observation shape changed")
    if len(source.graph.output) != 1 or source.graph.output[0].name != "actions":
        raise ValueError("Walking teacher output contract changed")
    if _fixed_shape(source.graph.output[0]) != (1, 14):
        raise ValueError("Walking teacher action shape changed")
    if tuple(node.op_type for node in source.graph.node) != EXPECTED_NODE_TYPES:

```

```

        raise ValueError("Walking teacher graph changed; exact transfer needs a parity review")

tensors = {
    value.name: np.asarray(numpy_helper.to_array(value))
    for value in source.graph.initializer
}
shapes = {name: value.shape for name, value in tensors.items()}
if shapes != EXPECTED_INITIALIZERS:
    raise ValueError("Walking teacher initializer contract changed")
if any(value.dtype != np.float32 for value in tensors.values()):
    raise ValueError("Walking teacher initializers must be float32")

mean = tensors["obs_normalizer._mean"]
standard_deviation = tensors["onnx::Div_24"]
if not np.all(np.isfinite(mean)):
    raise ValueError("Walking teacher normalizer mean is invalid")
if not np.all(np.isfinite(standard_deviation)) or not np.all(
    standard_deviation > 0
):
    raise ValueError("Walking teacher normalizer standard deviation is invalid")
return source, tensors, source_hash

def _representative_observations(
    mean: np.ndarray, standard_deviation: np.ndarray, seed: int
) -> np.ndarray:
    rng = np.random.default_rng(seed)
    normalized = rng.uniform(-3.0, 3.0, size=(PARITY_SAMPLES - 1, 61)).astype(
        np.float32
    )
    normalized = np.concatenate(
        (np.zeros((1, 61), dtype=np.float32), normalized), axis=0
    )
    observations = mean.reshape(1, 61) + normalized * standard_deviation.reshape(
        1, 61
    )
    if not np.all(np.isfinite(observations)):
        raise RuntimeError("Seeded parity observations are not finite")
    return observations.astype(np.float32)

def initialize_bridge(
    output: Path, *, seed: int = 7, initial_std: float = 0.03
) -> Path:
    """Transfer alpha_walking exactly into an MLX actor with fresh training state."""
    import mlx.core as mx
    import onnxruntime as ort

    from rlx.models.microduck import (
        create_actor_critic,
        normalize_observations,
        save_checkpoint,
    )

    if not math.isfinite(initial_std) or initial_std <= 0:
        raise ValueError("initial_std must be finite and positive")

    _, tensors, source_hash = _validated_source()
    mean = tensors["obs_normalizer._mean"].reshape(61).copy()
    standard_deviation = tensors["onnx::Div_24"].reshape(61).copy()
    variance = np.square(standard_deviation, dtype=np.float32)

    mx.random.seed(seed)
    network = create_actor_critic(initial_std=initial_std)
    actor_weights = [
        (
            f"actor_mean.layers.{index}.{kind}",
            mx.array(tensors[f"mlp.{index}.{kind}"].copy()),
        )
        for index in (0, 2, 4, 6)
        for kind in ("weight", "bias")
    ]

```

```

]
network.load_weights(actor_weights, strict=False)
mx.eval(network.parameters())

observations = _representative_observations(mean, standard_deviation, seed)
session = ort.InferenceSession(
    str(WALK_POLICY), providers=["CPUExecutionProvider"]
)
expected = np.concatenate(
    [session.run(["actions"], {"obs": item[None]})[0] for item in observations],
    axis=0,
)
normalized = normalize_observations(
    mx.array(observations), mean, variance, epsilon=0.0, clip=1e6
)
actual = np.asarray(network.deterministic(normalized), dtype=np.float32)
parity_error = float(np.max(np.abs(actual - expected)))
if not math.isfinite(parity_error) or parity_error > PARITY_THRESHOLD:
    raise RuntimeError(
        f"Exact walking actor transfer failed parity: {parity_error}"
    )
if hashlib.sha256(WALK_POLICY.read_bytes()).hexdigest() != source_hash:
    raise RuntimeError("Walking source policy changed during initialization")

return save_checkpoint(
    output,
    network,
    mean,
    variance,
    1.0,
    epsilon=0.0,
    clip=1e6,
    metadata={
        "recipe": "bridge",
        "steps": 0,
        "bootstrap": (
            "exact alpha_walking actor and normalizer transfer; "
            "fresh critic and optimizer"
        ),
        "initialization_method": "exact_weight_transfer",
        "continuation_kind": (
            "actor warm-start with fresh local critic and optimizer"
        ),
        "source_policy": str(WALK_POLICY.resolve()),
        "source_policy_sha256": source_hash,
        "walking_policy_sha256": source_hash,
        "parity_samples": len(observations),
        "parity_max_absolute_action_error": parity_error,
        "parity_threshold": PARITY_THRESHOLD,
        "critic_initialization": "fresh_seeded_mlx",
        "optimizer_initialization": "fresh_by_training_caller",
        "freeze_observation_normalization_by_caller": True,
        "seed": seed,
        "initial_std": initial_std,
        "trained": False,
    },
)

```

9.7 A7. rlx/rlx/algorithms/ppo.py

SHA-256: 126783ef9600ad7b27030d45d87bdf9a4f890cb09839316717ad46f5778b2f83

```

from dataclasses import dataclass
from time import perf_counter
from typing import Callable, Optional

import numpy as np

```

```

import mlx.core as mx
import mlx.nn as nn
import mlx.optimizers as optim
from mlx.utils import tree_flatten, tree_map

from rlx.environments.environment import Environment
from rlx.buffers.rollout_buffer import RolloutBuffer
from rlx.utils import compute_generalized_advantage_estimate, flatten

```

```

LOG_RATIO_MIN = -20.0
LOG_RATIO_MAX = 20.0
VALUE_LOSS_DELTA = 10.0

```

```

@dataclass
class PPOConfig:
    num_envs: int = 4096
    num_steps: int = 16
    gamma: float = 0.99
    gae_lambda: float = 0.95
    num_minibatches: int = 16
    update_epochs: int = 2
    normalize_advantages: bool = True
    clip_coefficient: float = 0.2
    clip_value_loss: bool = True
    entropy_coefficient: float = 0.01
    value_coefficient: float = 0.5
    max_grad_norm: float = 0.5

@dataclass
class PPO:
    config: PPOConfig
    env: Environment
    network: nn.Module
    optimizer: optim.Optimizer
    buffer: RolloutBuffer
    key: mx.array
    step: int = 0

    def __post_init__(self):
        self._constrain_actor_log_std()
        mx.eval(self.network.state)
        self._assert_finite("model parameters", self.network.parameters())
        state = [self.network.state, self.optimizer.state]
        self.update_step = mx.compile(self.update_step, inputs=state, outputs=state)
        self._update_step_with_metrics = mx.compile(
            self._update_step_with_metrics,
            inputs=state,
            outputs=state,
        )

    def _constrain_actor_log_std(self):
        constrain = getattr(self.network, "constrain_actor_log_std", None)
        if constrain is not None:
            constrain()

    @staticmethod
    def _all_finite(tree):
        checks = [mx.all(mx.isfinite(value)) for _, value in tree_flatten(tree)]
        return mx.all(mx.stack(checks))

    @classmethod
    def _assert_finite(cls, label, tree):
        check = cls._all_finite(tree)
        mx.eval(check)
        if not bool(np.asarray(check)):
            raise RuntimeError(f"PPO {label} contains non-finite values")

    @staticmethod

```

```

def _raise_if_non_finite(label, check):
    if not bool(np.asarray(check)):
        raise RuntimeError(f"PP0 {label} contains non-finite values")

def _gradient_failure_details(
    self,
    observations,
    actions,
    old_log_probabilities,
    old_values,
    advantages,
    returns,
) -> str:
    """Recompute one failed minibatch eagerly and name unstable tensors."""
    loss, grads = nn.value_and_grad(self.network, self.loss_fn)(
        self.network,
        observations,
        actions,
        old_log_probabilities,
        old_values,
        advantages,
        returns,
    )
    flat_grads = tree_flatten(grads)
    mx.eval(loss, *(value for _, value in flat_grads))
    bad = [
        name for name, value in flat_grads
        if not np.isfinite(np.asarray(value)).all()
    ]

    def summary(name, value):
        array = np.asarray(value, dtype=np.float64)
        finite = array[np.isfinite(array)]
        if finite.size:
            bounds = f"{finite.min():.6g}..{finite.max():.6g}"
        else:
            bounds = "no finite values"
        return f"{name}[{bounds}]; nonfinite={array.size - finite.size}"

    inputs = ", ".join(
        summary(name, value)
        for name, value in (
            ("observations", observations),
            ("actions", actions),
            ("old_log_probabilities", old_log_probabilities),
            ("old_values", old_values),
            ("advantages", advantages),
            ("returns", returns),
        )
    )
    tensors = ", ".join(bad) if bad else "not reproduced during eager diagnosis"
    return f"gradient tensors={tensors}; {inputs}"

def warmup(self, num_steps: int):
    pass

def _truncation_values(self, terminated, truncated, info):
    """Evaluate only final timeout states, never autoreset observations."""
    selected = np.flatnonzero(np.asarray(truncated) & ~np.asarray(terminated))
    result = mx.zeros(self.config.num_envs)
    if not selected.size:
        return result
    if "terminal_observation" not in info or "_terminal_observation" not in info:
        raise ValueError("PP0 timeout requires terminal_observation and its validity mask")
    terminals = mx.array(info["terminal_observation"])
    mask = np.asarray(info["_terminal_observation"])
    expected = (self.config.num_envs, *self.env.observation_space.shape)
    if terminals.shape != expected or mask.shape != (self.config.num_envs,) or mask.dtype != bool:
        raise ValueError("PP0 terminal observations or validity mask have invalid shape or dtype")

```



```

if not np.all(mask[selected]):
    raise ValueError("PPO timeout is missing a valid terminal observation")
indices = mx.array(selected.astype(np.int32))
_, values = self.network(terminals[indices])
self._assert_finite("timeout critic values", {"values": values})
return result.at[indices].add(values.squeeze(-1))

def train(
    self,
    num_steps: int,
    callback: Optional[Callable] = None,
    *,
    observer: Optional[Callable[[dict], None]] = None,
):
    """Train to the step target, optionally observing completed phases.

    The episode callback keeps its ``callback(info, step)`` convention.
    Observer dictionaries contain per-phase ``phase``, ``steps`` and wall
    ``seconds``; updates also contain ``optimizer_steps`` and ``mean_loss``
    plus the mean actual-minibatch ``policy_loss``, ``value_loss``,
    ``entropy``, ``approximate_kl``, ``clip_fraction`` and
    ``explained_variance``.
    Collection includes buffer reset and rollout; update includes last-value
    evaluation, GAE and deferred critic work. Observer time is excluded;
    these intervals do not isolate device execution or transfer costs.
    Observer exceptions propagate. A failed phase emits no completion event.
    Observers must not mutate training state or consume training RNGs.
    """
    self.key, reset_key = mx.random.split(self.key)
    keys = mx.random.split(reset_key, self.config.num_envs)
    observation, state, _ = self.env.reset(keys)
    mx.eval(observation, *state.values())

    next_done = mx.zeros(self.config.num_envs)
    while self.step < num_steps:
        if observer is not None:
            collection_step = self.step
            collection_start = perf_counter()
        self.buffer.reset()
        timeout_values = []
        for _ in range(self.config.num_steps):
            self._constrain_actor_log_std()
            distribution, value = self.network(observation)
            action = distribution.sample()
            log_probability = distribution.log_prob(action)

            self.key, step_key = mx.random.split(self.key)
            keys = mx.random.split(step_key, self.config.num_envs)
            next_observation, state, reward, terminated, truncated, info = (
                self.env.step(keys, state, action)
            )
            mx.eval(
                next_observation, reward, terminated, truncated, *state.values()
            )

            timeout_values.append(self._truncation_values(terminated, truncated, info))
            if callback is not None and "episode" in info:
                callback(info, self.step)

            self.buffer.add(
                observation,
                next_observation,
                action,
                reward,
                terminated,
                truncated,
                value=value,
                log_prob=log_probability,
            )

            observation = next_observation

```

```

        next_done = mx.logical_or(terminated, truncated).astype(
            mx.float32
        )
        self.step += self.config.num_envs

    if observer is not None:
        collection_seconds = perf_counter() - collection_start
        phase_steps = self.step - collection_step
        observer({
            "phase": "collection",
            "steps": phase_steps,
            "seconds": collection_seconds,
        })
        update_start = perf_counter()

    _, last_value = self.network(observation)
    advantages = compute_generalized_advantage_estimate(
        self.buffer.rewards,
        self.buffer.values,
        self.buffer.terminations,
        last_value.squeeze(-1),
        next_done,
        self.config.gamma,
        self.config.gae_lambda,
        truncations=self.buffer.truncations,
        truncation_values=mx.stack(timeout_values),
    )
    returns = advantages + self.buffer.values

    if observer is None:
        self.update(advantages, returns)
    else:
        metrics = self.update(advantages, returns, collect_metrics=True)
        update_seconds = perf_counter() - update_start
        observer({
            "phase": "update",
            "steps": phase_steps,
            "seconds": update_seconds,
            **metrics,
        })

def loss_fn(
    self,
    network,
    observations,
    actions,
    old_log_probabilities,
    old_values,
    advantages,
    returns,
):
    return self._loss_and_metrics(
        network,
        observations,
        actions,
        old_log_probabilities,
        old_values,
        advantages,
        returns,
    )[0]

def _loss_and_metrics(
    self,
    network,
    observations,
    actions,
    old_log_probabilities,
    old_values,
    advantages,
    returns,
):
    distribution, new_values = network(observations)

```

```

new_log_probabilities = distribution.log_prob(actions)
entropy = distribution.entropy()

# PPO's clipped objective does not need an unbounded probability ratio.
# Large but finite log-probability differences can overflow exp() during
# backpropagation before gradient clipping has a chance to act.
log_ratio = mx.clip(
    new_log_probabilities - old_log_probabilities,
    LOG_RATIO_MIN,
    LOG_RATIO_MAX,
)
ratio = mx.exp(log_ratio)

if self.config.normalize_advantages:
    advantages = (advantages - mx.mean(advantages)) / (
        mx.std(advantages) + 1e-8
    )

policy_loss = mx.maximum(
    -advantages * ratio,
    -advantages
    * mx.clip(
        ratio,
        1 - self.config.clip_coefficient,
        1 + self.config.clip_coefficient,
    ),
)
policy_loss = mx.mean(policy_loss)

new_values = new_values.squeeze(-1)

def value_error_loss(predictions):
    errors = predictions - returns
    absolute_errors = mx.abs(errors)
    quadratic = mx.minimum(absolute_errors, VALUE_LOSS_DELTA)
    linear = absolute_errors - quadratic
    return 0.5 * mx.square(quadratic) + VALUE_LOSS_DELTA * linear

if self.config.clip_value_loss:
    value_loss_unclipped = value_error_loss(new_values)
    clipped_values = old_values + mx.clip(
        new_values - old_values,
        -self.config.clip_coefficient,
        self.config.clip_coefficient,
    )
    value_loss_clipped = value_error_loss(clipped_values)
    value_loss = mx.mean(
        mx.maximum(value_loss_unclipped, value_loss_clipped)
    )
else:
    value_loss = mx.mean(value_error_loss(new_values))

entropy_loss = mx.mean(entropy)

loss = (
    policy_loss
    - self.config.entropy_coefficient * entropy_loss
    + self.config.value_coefficient * value_loss
)
approximate_kl = mx.mean((ratio - 1) - log_ratio)
clip_fraction = mx.mean(
    mx.abs(ratio - 1) > self.config.clip_coefficient
)
value_errors = returns - new_values
return (
    loss,
    policy_loss,
    value_loss,
    entropy_loss,
    approximate_kl,
    clip_fraction,
)

```

```

        mx.sum(returns),
        mx.sum(mx.square(returns)),
        mx.sum(value_errors),
        mx.sum(mx.square(value_errors)),
    )

def _apply_update(self, loss, grads):
    loss_finite = mx.all(mx.isfinite(loss))
    gradients_finite = self._all_finite(grads)
    update_finite = mx.logical_and(loss_finite, gradients_finite)
    grads = tree_map(
        lambda grad: mx.where(update_finite, grad, mx.zeros_like(grad)),
        grads,
    )
    grads, _ = optim.clip_grad_norm(grads, self.config.max_grad_norm)
    self.optimizer.update(self.network, grads)
    self._constrain_actor_log_std()
    parameters_finite = self._all_finite(self.network.parameters())
    return loss_finite, gradients_finite, parameters_finite

def update_step(
    self,
    observations,
    actions,
    old_log_probabilities,
    old_values,
    advantages,
    returns,
):
    loss, grads = nn.value_and_grad(self.network, self.loss_fn)(
        self.network,
        observations,
        actions,
        old_log_probabilities,
        old_values,
        advantages,
        returns,
    )
    loss_finite, gradients_finite, parameters_finite = self._apply_update(
        loss, grads
    )
    return loss, loss_finite, gradients_finite, parameters_finite

def _update_step_with_metrics(
    self,
    observations,
    actions,
    old_log_probabilities,
    old_values,
    advantages,
    returns,
):
    metrics, grads = nn.value_and_grad(
        self.network, self._loss_and_metrics
    )(
        self.network,
        observations,
        actions,
        old_log_probabilities,
        old_values,
        advantages,
        returns,
    )
    loss_finite, gradients_finite, parameters_finite = self._apply_update(
        metrics[0], grads
    )
    return (
        *metrics,
        loss_finite,
        gradients_finite,
        parameters_finite,
    )

```

```

    )

def update(
    self,
    advantages: mx.array,
    returns: mx.array,
    *,
    collect_metrics: bool = False,
):
    """Update minibatches, returning metrics only when requested.

    Metrics count completed optimizer steps and average the values produced
    by their actual minibatch objective evaluations.
    Metrics require at least one optimizer step; exceptions propagate without
    returning partial metrics. The default return value remains ``None``.
    """
    if collect_metrics:
        optimizer_steps = 0
        totals = {
            "mean_loss": 0.0,
            "policy_loss": 0.0,
            "value_loss": 0.0,
            "entropy": 0.0,
            "approximate_kl": 0.0,
            "clip_fraction": 0.0,
        }
        return_sum = 0.0
        return_square_sum = 0.0
        value_error_sum = 0.0
        value_error_square_sum = 0.0
        metric_samples = 0
        observations = flatten(self.buffer.observations)
        actions = flatten(self.buffer.actions)
        log_probabilities = flatten(self.buffer.log_probs)
        values = flatten(self.buffer.values)
        advantages = flatten(advantages)
        returns = flatten(returns)
        self._assert_finite(
            "update inputs",
            {
                "observations": observations,
                "actions": actions,
                "log_probabilities": log_probabilities,
                "values": values,
                "advantages": advantages,
                "returns": returns,
            },
        )

    batch_size = self.config.num_steps * self.config.num_envs
    minibatch_size = batch_size // self.config.num_minibatches

    for _ in range(self.config.update_epochs):
        permutation = np.random.permutation(batch_size)
        for start in range(0, batch_size, minibatch_size):
            minibatch_indices = mx.array(
                permutation[start : start + minibatch_size]
            )
            if collect_metrics:
                result = self._update_step_with_metrics(
                    observations[minibatch_indices],
                    actions[minibatch_indices],
                    log_probabilities[minibatch_indices],
                    values[minibatch_indices],
                    advantages[minibatch_indices],
                    returns[minibatch_indices],
                )
                metric_values = result[:6]
                metric_moments = result[6:10]
                loss_finite, gradients_finite, parameters_finite = result[10:]
                loss = metric_values[0]

```

```

else:
    loss, loss_finite, gradients_finite, parameters_finite = (
        self.update_step(
            observations[minibatch_indices],
            actions[minibatch_indices],
            log_probabilities[minibatch_indices],
            values[minibatch_indices],
            advantages[minibatch_indices],
            returns[minibatch_indices],
        )
    )
mx.eval(
    loss,
    loss_finite,
    gradients_finite,
    parameters_finite,
    self.network.state,
    self.optimizer.state,
)
self._raise_if_non_finite("loss", loss_finite)
if not bool(np.asarray(gradients_finite)):
    details = self._gradient_failure_details(
        observations[minibatch_indices],
        actions[minibatch_indices],
        log_probabilities[minibatch_indices],
        values[minibatch_indices],
        advantages[minibatch_indices],
        returns[minibatch_indices],
    )
    raise RuntimeError(
        f"PP0 gradients contains non-finite values: {details}"
    )
self._raise_if_non_finite(
    "model parameters", parameters_finite
)
if collect_metrics:
    for name, value in zip(totals, metric_values):
        totals[name] += float(value.item())
    return_sum += float(metric_moments[0].item())
    return_square_sum += float(metric_moments[1].item())
    value_error_sum += float(metric_moments[2].item())
    value_error_square_sum += float(metric_moments[3].item())
    metric_samples += int(returns[minibatch_indices].size)
    optimizer_steps += 1

if collect_metrics:
    if optimizer_steps == 0:
        raise ValueError("PP0 update metrics require at least one optimizer step")
    return_variance = (
        return_square_sum / metric_samples
        - (return_sum / metric_samples) ** 2
    )
    value_error_variance = (
        value_error_square_sum / metric_samples
        - (value_error_sum / metric_samples) ** 2
    )
    explained_variance = (
        1 - value_error_variance / return_variance
        if return_variance > 1e-8
        else 0.0
    )
    return {
        "optimizer_steps": optimizer_steps,
        **{
            name: total / optimizer_steps
            for name, total in totals.items()
        },
        "explained_variance": explained_variance,
    }

```

```

def evaluate(self, num_steps: int, callback: Optional[Callable] = None):
    self.key, reset_key = mx.random.split(self.key)
    keys = mx.random.split(reset_key, self.config.num_envs)
    observation, state, _ = self.env.reset(keys)
    mx.eval(observation, *state.values())

    for _ in range(0, num_steps, self.config.num_envs):
        self._constrain_actor_log_std()
        distribution, _ = self.network(observation)
        action = distribution.sample()

        self.key, step_key = mx.random.split(self.key)
        keys = mx.random.split(step_key, self.config.num_envs)
        observation, state, reward, terminated, truncated, info = (
            self.env.step(keys, state, action)
        )
        mx.eval(observation, reward, terminated, truncated, *state.values())

        if callback is not None and "episode" in info:
            callback(info, self.step)

```

9.8 Source inventory / 源码清单

File / 文件	Bytes / 字节	Printed / 全文
rlx/rlx/environments/ basketball.py	15053	True
rlx/rlx/models/basketba ll.py	12553	True
rlx/examples/ppo_ microduck_basketball.py	25019	True
rlx/rlx/environments/ bridge.py	23794	True
rlx/rlx/environments/ bridge_evaluation.py	10122	True
rlx/rlx/models/bridge_ bootstrap.py	6821	True
rlx/rlx/algorithms/ ppo.py	21802	True
rlx/examples/ppo_ microduck_balance.py	37084	False
rlx/examples/ppo_ microduck_studio.py	59467	False
rlx/scripts/eval_ basketball_local.py	29570	False
rlx/rlx/environments/ microduck_recipes.py	48106	False
rlx/rlx/environments/ microduck.py	16607	False
rlx/rlx/models/microduc k.py	7786	False
rlx/rlx/export/microduc k_onnx.py	5599	False
rlx/rlx/buffers/rollout_ buffer.py	3063	False
rlx/rlx/utils/utils.py	3138	False
rlx/rlx/utils/distribu tions.py	2399	False
rlx/rlx/utils/__init__ .py	522	False

File / 文件	Bytes / 字节	Printed / 全文
microduck_local/src/ microduck_local/contract .py	4067	False
microduck_local/src/ microduck_local/walk_ env.py	38041	False
microduck_local/src/ microduck_local/studio_ policies.py	16512	False
microduck_local/src/ microduck_local/viz_ server.py	185964	False
duck-viewer/lib/experime nts.ts	37262	False
duck-viewer/lib/rlx- job.ts	46969	False
duck-viewer/lib/scene- layout.ts	645	False
scripts/train-basketba ll.sh	256	False
scripts/train-bridge.sh	368	False
scripts/verify-balance- studio-api.mjs	3756	False
scripts/verify-seven- cases.mjs	5701	False
rlx/pyproject.toml	3578	False
microduck_local/pyprojec t.toml	2379	False
duck-viewer/package.json	826	False
microduck-playground/ artifacts/basketball/ NOTICE	763	False
microduck-playground/ artifacts/basketball/ LICENSE	11351	False
rlx/tests/test_basketba ll.py	4998	False
rlx/tests/test_basketba ll_evaluation.py	7185	False
rlx/tests/test_basketba ll_policy.py	16678	False
rlx/tests/test_bridge.py	10383	False
rlx/tests/test_bridge_ bootstrap.py	2546	False
rlx/tests/test_bridge_ evaluation.py	5319	False
rlx/tests/test_balance_ adapter.py	18789	False